Universidad de Oviedo
Universidá d'Uviéu
University of Oviedo

Escuela de Ingeniería Informática
Universidad de Oviedo

DEGREE IN SOFTWARE ENGINEERING

Type of TFG: Development

Year: 2025 / 2026

Population growth simulator. Simulation of population models with stochasticity

Author: Mario Orviz Viesca

Tutors: Celia Melendi Lavandera

Mario Quevedo de Anta

Version 1.0 (29/06/2026) Oviedo, Spain

Declaration of Authorship and Originality

In accordance with the provisions of Article 8.3 of the Agreement of July 17, 2020, of the Governing Council of the University of Oviedo, approving the Regulations on the preparation and defense of master's degree final projects at the University of Oviedo (BOPA No. 153 of 7-viii-2020):

Mr. **Mario Orviz Viesca**, with ID number **34314457K**,

I DECLARE THAT:

The Final Degree Project titled **Population growth simulator. Simulation of population models with stochasticity**, which I present for exhibition and defense, is original and I have duly cited all sources of information used, both in the body of the text and in the bibliography.

_____

«Digital Signature of the Student»

In Oviedo, on

_____

of

_____

of 20

___

ACKNOWLEDGEMENTS

TABLE OF CONTENTS

List of Figures

List of Tables

Chapter 1

General Description of the Project

1.1 Summary

This project is a web application developed in collaboration with the Faculty of Biology, designed to simulate population growth dynamics for educational purposes. Its main goal is to serve as an interactive teaching tool that helps university students and researchers or professors better understand and visualize how populations evolve over time.

The application is entirely web-based, meaning it requires no installation and can be accessed from any device with a browser and an internet connection. It is intended to be free and openly available to anyone in the world, with no barriers to access. This global accessibility is a core design principle of the project, as the aim is to bring quality educational resources to as wide an audience as possible, regardless of their institution or geographic location.

The project was born out of a real collaboration between the School of Engineering and the Faculty of Biology, which gives it a genuine interdisciplinary and practical context. Although the application is currently functional and actively under development, the intention is for it to eventually be deployed and used in real academic settings, both as a complement to biology courses and as a resource for independent study and research.

By combining an intuitive interface with solid scientific foundations, the application bridges the gap between theoretical population ecology and hands-on learning, making abstract mathematical models tangible and interactive for its users.

1.2 Keywords

Population dynamics, Educational simulation, Web application, Computational biology, Interactive visualization

1.3 Abstract

This project presents the design and development of a web application created in collaboration with the Faculty of Biology, aimed at simulating population growth dynamics for educational purposes. The tool allows users to interactively explore stage-structured matrix population models (Lefkovitch and Leslie matrices) sourced from the COMPADRE and COMADRE databases, through intuitive visual representations that make abstract demographic concepts more accessible and engaging.

The application is entirely browser-based, requiring no installation, and is freely available to anyone with an internet connection. It targets university students, researchers, and professors as its primary audience, serving as a complementary resource for teaching and studying population ecology.

The project emerges from a genuine interdisciplinary collaboration between the fields of software engineering and biology, with the long-term goal of deploying the tool in real academic environments worldwide. The current version is functional and under active development, with a focus on usability, scientific accuracy, and universal accessibility.

1.4 Biological Context

This section provides an overview of the biological context in which the project is situated. It explains how the application defines the concepts of **simulation** and **quasi-extinction** in the context of population dynamics.

1.4.1 Data Background

The concept of **population model** refers to a mathematical representation of how likely a population is to change over time, based on field data collected from real populations.

A population itself has several stages, depending on the species; these stages may include: eggs, larvae, juveniles, adults, and senescent individuals. As we standardize the population to a set of M stages, we can represent a specific population as a vector $P(t) = \left(p_1(t), p_2(t), ..., p_{M(t)}\right)$, where $p_{i(t)}$ is the number of individuals in stage i at time t.

Therefore, if we want to model the population dynamics of a specific species, we need to define the transition probabilities between stages, which can be represented as an $M \times M$ matrix A, where each element $a_{\{i,j\}}$ represents the probability of an individual in stage j transitioning to stage i in the next time step. This kind of stage-structured projection matrix is known in the literature as a **Lefkovitch matrix** (a generalization of the classic age-structured **Leslie matrix**, which restricts transitions to a fecundity row and a survival sub-diagonal) [1]. The COMPADRE matrices used in this project are Lefkovitch (or, where strictly age-structured, Leslie) matrices.

1.4.2 Simulation Analysis

In the context of this project, a **simulation** refers to the process of using a mathematical model to predict the future state of a population based on its current state and the defined transition probabilities. We can use the population vector $P(t)$ and the transition matrix A to project the population into the future using the equation: $P(t+1) = A * P(t)$

This equation allows us to create the projection of a matrix A with an initial population vector $P(0)$, which represents the population at time $t = 0$. By iteratively applying the transition matrix A to the population vector, we can project how the population evolves over time.

This is known as a **population projection**, and in the application we'll refer to it as a **deterministic simulation**.

1.4.2.1 Introducing Stochasticity

The fact is that, in real life, populations are subject to random events and environmental fluctuations that can affect their growth and survival. Therefore, we also consider **stochastic simulations**, which incorporate randomness into the population projection process. The way we achieve stochastic simulations is the following one:

Instead of having a unique matrix A, we must start the simulation with a set of 2 to N matrices A of the same species, that might have been registered under different environmental conditions (ex. different seasons). We also need again the vector $P(0)$ as starting point.

The next step is defining the value of R, the total number of **iterations** (runs) to perform. On each iteration, a matrix at random is picked from the set of matrices and used for the entire projection - that is, the same matrix is applied at every step of that iteration, from $P(0)$ through $P(n)$. We continue until all R iterations have been run. In this process, we keep track of the frequency with which each matrix is used in a vector F of size N, where f_1 represents how many times the first matrix in the set has been picked. Then, with the results of all iterations, we compute the mean, minimum and maximum values and standard deviation for each stage at each t step, across iterations.

From these simulations we can also obtain other relevant information, such as the frequency-weighted mean transition matrix $\overline{A} = \frac{1}{R}\sum_{k=1}^{R} A_k$ (equivalently, $\overline{A} = \sum_{j=1}^{N}\left(\frac{f_j}{R}\right)A_j$ using the frequency vector F). Note that, because repeated matrix multiplication is non-linear, projecting $\overline{A}$ alone does **not** reproduce the exact same trajectory as the stochastic run - exact reproducibility of a given run is instead achieved by storing the random seed used to pick the matrices at each iteration.

1.4.3 Quasi-extinction analysis

Once we have our stochastic simulations defined, we introduce the concept of **quasi-extinction** analysis. The quasi-extinction analysis consists in studying how likely a population is to become extinct. In the book "Quantitative conservation biology : theory and practice of population viability analysis" [2], it's defined as: "the population falling below a quasi-extinction threshold set as the minimum number of individuals (or, often, females) below which the population is likely to be critically and immediately imperiled"

To perform a **quasi-extinction analysis**, we need the same things as for a stochastic simulation (there's no point in performing an analysis of this nature over a single deterministic projection, as we would always obtain either a 100% or a 0% probability). Then, we define a threshold vector of size M, with one entry per population stage. This vector tells us the minimum amount of individuals in that specific stage required to consider the species **quasi-extinct**. This means that, if in a projection the amount of individuals in a specific stage goes below that threshold at any step, we consider that iteration to have hit quasi-extinction.

Finally, we obtain the probability of **quasi-extinction** as the proportion of iterations that hit quasi-extinction out of the R total iterations performed:

$$P_{\text{qe}} = \frac{n_{\text{extinct}}}{R}$$

We can also obtain more relevant information from this kind of simulations, like which stage is more likely to trigger **quasi-extinction**, at which step… .

The R implementations of these methods provided by the github repository **eco3r** [3] of Mario Quevedo de Anta served as a methodological reference for the algorithms implemented in this project.

Chapter 2

Planning and Management

2.1 Project Planning

2.1.1 Identification of Stakeholders

The following table shows, identified by a unique ID and classified into external and internal, the different stakeholders identified for the system "Population growth simulator. Simulation of population models with stochasticity"

Stakeholders		
ID	**Name**	**Description**
Internal Stakeholders		
SI-1	Universidad de Oviedo	University to whom the TFG belongs
SI-2	Biology Faculty of the University of Oviedo	Faculty for whom the software is developed
SI-3	Software Engineering School of the University of Oviedo	School to whom the student that will develop the software belongs
SI-4	Celia Melendi Lavandera	TFG tutor belonging to the Software Engineering School
SI-5	Mario Quevedo de Anta	TFG co-tutor belonging to the Biology Faculty
SI-6	Mario Orviz Viesca	Student in charge of the planning, development, and documentation of this TFG
SI-7	Teachers and students from the Biology Faculty	Actual real users for the application
External Stakeholders		
SE-1	Max Planck Institute for Demographic Research (Germany)	Institution that supports and hosts the COM(P)ADRE online repository for matrix population models (MPMs) and metadata
SE-2	Other biology organisms	Some other biology organisms that might find this tool useful for their activity

SE-3	Hosting provider	This product will be a web product, it will need a hosting provider

Table 1: Internal and External Stakeholder identification

2.1.1.1 System users

The following table shows the different types of users that are expected to interact with the future system:

System Users		
ID	**Name**	**Description**
SU-1	Non-registered user	User that accesses the system without having an account or being logged in
SU-2	Registered user	User that accesses the system being logged into their account

Table 2: System Users

2.1.2 OBS and PBS

In this section, I will present the organizational breakdown structure and the product breakdown structure for this project.

2.1.2.1 OBS

This section defines the **Organizational Breakdown Structure (OBS)** for a project in which one person performs all tasks, supported by two tutors who provide guidance, feedback, and formal acceptance.

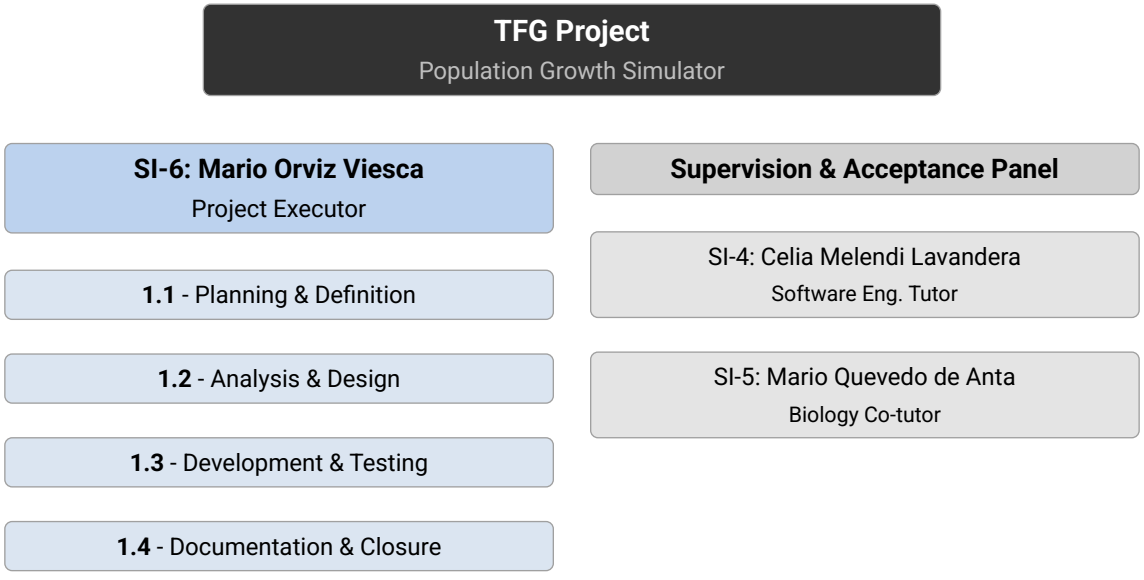

Figure 1: Organizational Breakdown Structure (OBS)

OBS Nodes		
ID	**Node**	**Description**
1. Project Executor (SI-6)		

1.1	Planning & Definition	Scope definition, scheduling, stakeholder analysis, and risk planning.
1.2	Analysis & Design	Requirements elicitation, system architecture, and interface design.
1.3	Development & Testing	Implementation of all system components and verification activities.
1.4	Documentation & Closure	TFG report writing, final delivery, and project closure tasks.
2. Supervision & Acceptance Panel		
2.1	SI-4 - Celia Melendi Lavandera	Software Engineering tutor. Provides methodological guidance and reviews technical deliverables.
2.2	SI-5 - Mario Quevedo de Anta	Biology co-tutor. Validates domain correctness and grants formal acceptance of the biological models implemented.

Table 3: OBS node descriptions

Responsibility Assignment Matrix (RACI)			
Activity	**SI-6 (Executor)**	**SI-4 (Tutor)**	**SI-5 (Co-tutor)**
Planning & Definition	R	C	I
Requirements & Analysis	R	C	A
Architecture & Design	R	C	I
Development	R	I	I
Testing & Validation	R	C	A
TFG Documentation	R	A	I
Mid-project Review	R	C	A
Final Delivery & Closure	R	A	C

*Table 4: RACI matrix - **R**: Responsible, **A**: Accountable/Accepts, **C**: Consulted, **I**: Informed*

2.1.3 Initial Planning. WBS

This project was developed following the **Scrum** agile framework. Work was organised into five development sprints plus an initial inception sprint, each lasting two to three weeks. Table 5 summarises the sprint structure, goals, and durations.

Sprint	Period	Duration	Goal
Sprint 0	Feb 11-28, 2026	2.5 weeks	Project inception: scope definition, Typst documentation template, initial Shiny prototype.
Sprint 1	Mar 1-14, 2026	2 weeks	Core architecture: FastAPI backend, PostgreSQL database, JWT authentication, COMPADRE seeder, initial test suite.

Sprint	Period	Duration	Goal
Sprint 2	Mar 15-28, 2026	2 weeks	MVP polish: matrix ownership model, frontend redesign, end-to-end tests, CI/CD pipeline setup.
Sprint 3	Mar 29 - May 8, 2026	5 weeks	Component decoupling and TFG documentation foundation: architecture diagrams, requirements analysis, technology alternatives.
Sprint 4	May 9-26, 2026	2.5 weeks	Analytics service, quasi-extinction service, jobs system, simulation export format v2.
Sprint 5	May 27 - Jun 9, 2026	2 weeks	Feature completion: stochastic simulation, internationalisation, matrix import/export, comprehensive test suite.

Table 5: Sprint overview

The Gantt chart below (Table 6) shows how the ten parallel workstreams overlap across the project timeline. Blue cells indicate development activity; green marks testing and DevOps work, which runs concurrently with most development sprints; orange marks the documentation sprint that produced this document.

Workstream	Feb (Sprint 0)	Mar (Sprints 1-2)	Apr (Sprint 3)	May (Sprints 3-4)	Jun (Sprint 5)
Project Management					
Prototype & Design					
Core Architecture					
Authentication					
Matrix Browsing					
Custom Matrices					
Simulations					
Analytics & Quasi-Extinction					
Testing & DevOps					
Documentation					

Table 6: Project Gantt chart - February to June 2026

2.1.4 Sprint Backlogs

This section details the task backlog for each sprint. Tasks are categorised by workstream and prioritised as **High**, **Med**, or **Low** relative to the sprint goal. Each task carries a point estimate on the same scale as the user-story points in Section 4.1.1.4: tasks that directly implement a story reuse its point value, while tasks with no single corresponding story (setup, architecture, DevOps, documentation) are estimated on the same scale by analogy. See Table 13 for the mapping between tasks and user stories.

Category legend: Setup | Architecture | Feature | Testing | DevOps | Documentation | Research

2.1.4.1 Sprint 0 Backlog (Feb 11-28, 2026)

> **Sprint goal:** Project inception: repository setup, Typst documentation template, initial Shiny prototype, and COMPADRE data exploration.

ID	Task	Category	Priority	Est. (pts)
S0-01	Set up project repository and version control	Setup	High	2
S0-02	Draft project scope, objectives, and initial requirements	Setup	High	3
S0-03	Create Typst documentation template with styles and structure	Documentation	High	3
S0-04	Build initial Shiny prototype with matrix input UI	Feature	High	5
S0-05	Implement basic population simulation (matrix × vector loop)	Feature	High	5
S0-06	Research COMPADRE dataset structure and data access	Research	Med	3

Table 7: Sprint 0 backlog

2.1.4.2 Sprint 1 Backlog (Mar 1-14, 2026)

> **Sprint goal:** Core architecture: FastAPI backend, PostgreSQL database, JWT authentication, COMPADRE seeder, initial test suite.

ID	Task	Category	Priority	Est. (pts)
S1-01	Set up PostgreSQL schema with SQLAlchemy ORM and Alembic migrations	Architecture	High	5
S1-02	Scaffold FastAPI project skeleton with `/v1/` prefix and router layout	Architecture	High	3
S1-03	Implement `/v1/matrices` CRUD endpoints (list, detail, create)	Feature	High	8
S1-04	Implement JWT authentication endpoints (register, login)	Feature	High	8
S1-05	Build COMPADRE seeder to populate the matrix catalogue on startup	Feature	High	8
S1-06	Write initial test suite covering auth and matrix endpoints	Testing	High	5

ID	Task	Category	Priority	Est. (pts)
S1-07	Document FastAPI technology decision, database schema, and stack	Documentation	Med	2

Table 8: Sprint 1 backlog

2.1.4.3 Sprint 2 Backlog (Mar 15-28, 2026)

> **Sprint goal:** MVP polish: matrix ownership model, Docker stack, frontend redesign, end-to-end tests, CI/CD pipeline.

ID	Task	Category	Priority	Est. (pts)
S2-01	Refactor backend to strict controller / service / repository layers	Architecture	High	8
S2-02	Create Docker Compose stack with auto-migration entrypoint script	DevOps	High	3
S2-03	Redesign Shiny frontend with four-tab layout (Browse, Simulate, My Matrices, Account)	Feature	High	8
S2-04	Implement matrix ownership model (`source_type`: custom vs. COMPADRE)	Feature	High	8
S2-05	Set up GitHub Actions CI pipeline (unit tests, integration tests)	DevOps	High	5
S2-06	Configure Bandit SAST, pip-audit, Trivy, and CodeQL security workflows	DevOps	Med	3
S2-07	Write integration tests for auth, matrix, and simulation endpoints	Testing	High	5

Table 9: Sprint 2 backlog

2.1.4.4 Sprint 3 Backlog (Mar 29 - May 8, 2026)

> **Sprint goal:** Component decoupling and TFG documentation foundation: architecture diagrams, requirements analysis, technology alternatives.

ID	Task	Category	Priority	Est. (pts)
S3-01	Complete frontend-backend decoupling (Shiny talks only to the API)	Architecture	High	5
S3-02	Add cyclic-graph visualisation and multi-matrix display in frontend	Feature	Med	5

ID	Task	Category	Priority	Est. (pts)
S3-03	Write requirements analysis chapter (use cases, scenarios, state diagrams)	Documentation	High	5
S3-04	Write solution alternatives and technology comparison chapter	Documentation	High	5
S3-05	Produce ER diagram, component diagrams, and deployment diagrams	Documentation	High	5
S3-06	Fix GitHub Actions pipeline (Trivy scanner, test runner configuration)	DevOps	Med	2
S3-07	Compile bibliographic references for the TFG document	Documentation	Low	1

Table 10: Sprint 3 backlog

2.1.4.5 Sprint 4 Backlog (May 9-26, 2026)

> **Sprint goal:** Analytics service, quasi-extinction service, async jobs system, simulation export format v2.

ID	Task	Category	Priority	Est. (pts)
S4-01	Add simulation snapshot and analytics DB columns (migration 0005)	Architecture	High	3
S4-02	Implement `AnalyticsService` (λ_1, SSD, reproductive value, elasticities, sensitivities)	Feature	High	5
S4-03	Wire `AnalyticsService` into `SimulationService` for all run types	Feature	High	3
S4-04	Add typed analytics records and simulation export format v2 schema	Feature	High	3
S4-05	Implement simulation export and import endpoints (`/v1/simulations/{id}/export`)	Feature	High	5
S4-06	Implement `JobRepository` and `QuasiExtinctionService` (Monte Carlo)	Feature	High	8
S4-07	Add jobs controller and router (`/v1/jobs/`)	Feature	High	3
S4-08	Implement matrix upload/download in the frontend	Feature	Med	5

Table 11: Sprint 4 backlog

2.1.4.6 Sprint 5 Backlog (May 27 - Jun 9, 2026)

> **Sprint goal:** Feature completion: stochastic simulation, quasi-extinction per-stage config, internationalisation, matrix import/export, comprehensive test suite.

ID	Task	Category	Priority	Est. (pts)
S5-01	Implement stochastic simulation (multiple matrices, uniform random selection, seed)	Feature	High	8
S5-02	Add stochastic analytics (λ_s, mean-matrix elasticities) to `AnalyticsService`	Feature	High	3
S5-03	Implement per-stage thresholds and exclusions in quasi-extinction algorithm	Feature	High	5
S5-04	Add internationalisation support (Galician, Basque, Catalan, Asturian)	Feature	Med	3
S5-05	Refactor matrix browse view to paginated card/list layout	Feature	Med	5
S5-06	Expand unit and integration tests (export, import, stochastic, QE)	Testing	High	5
S5-07	Write Playwright E2E tests for quasi-extinction tab and navigation	Testing	High	3
S5-08	Write planning and management chapter for TFG documentation	Documentation	High	5
S5-09	Deploy application on railway	Feature	Med	3

Table 12: Sprint 5 backlog

2.1.4.7 User Story Traceability

Table 13 maps every user story from Section 4.1.1.4 to the sprint(s) and task(s) in Table 5 that implemented it. The mapping is not one-to-one: a story is often delivered incrementally across more than one task, or even across more than one sprint (e.g. a backend endpoint in one sprint, its frontend in a later one), and several sprint tasks (setup, architecture, DevOps, documentation) support no single story and are therefore absent from this table even though they appear in Table 5.

US ID	Sprint(s)	Task(s)
US-01	Sprint 1	S1-04
US-02	Sprint 1	S1-04
US-03	Sprint 1	S1-04

US ID	Sprint(s)	Task(s)
US-04	Sprint 1, Sprint 5	S1-03, S5-05
US-05	Sprint 1, Sprint 5	S1-03, S5-05
US-06	Sprint 1, Sprint 2, Sprint 3	S1-03, S2-03, S3-02
US-07	Sprint 4, Sprint 5	S4-08, S5-06
US-08	Sprint 0, Sprint 1	S0-06, S1-05
US-09	Sprint 1, Sprint 5	S1-03, S5-05
US-10	Sprint 0, Sprint 1, Sprint 2	S0-04, S1-03, S2-03
US-11	Sprint 2	S2-03, S2-04
US-12	Sprint 2	S2-03, S2-04
US-13	Sprint 2	S2-04
US-14	Sprint 2	S2-04
US-15	Sprint 4, Sprint 5	S4-08, S5-06
US-16	Sprint 0, Sprint 2	S0-05, S2-01, S2-03
US-17	Sprint 5	S5-01
US-18	Sprint 2	S2-01, S2-03
US-19	Sprint 2	S2-01, S2-03
US-20	Sprint 4, Sprint 5	S4-02, S4-03, S5-02
US-21	Sprint 4	S4-06, S4-07
US-22	Sprint 5	S5-03
US-23	Sprint 2, Sprint 3	S2-05, S2-06, S3-06
US-24	Sprint 5	S5-04

Table 13: User Story Traceability Matrix

2.1.5 Risks

2.1.5.1 Risk Management Plan

Risk management for this project follows a Probability-Impact methodology adapted from the PMBOK Guide [4]: risks are classified by a four-branch Risk Breakdown Structure, assessed against a qualitative probability scale and a four-objective (cost, schedule, scope, quality) impact scale, and prioritised through a Probability-Impact Matrix. The full methodology, scales, and the contingency plans for every red-zone risk are described in Section 9.1.

2.1.5.2 Risk Identification

Thirteen risks were identified across the four Risk Breakdown Structure categories (Technical, External, Organizational, and Project Management). Each risk below is analysed using the scales defined in Section 9.1: probability is rated on a five-level scale, impact is rated independently against cost, schedule, scope, and quality, and the resulting exposure

score (probability multiplied by the worst-affected objective's impact) determines the risk's zone on the Probability-Impact Matrix (Table 70), classified as green, yellow, or red.

Technical Risks

T-01 - Simulation and analytics correctness errors			
Description	The deterministic and stochastic simulation engines (`api/services/simulation_service.py`) and the ecological analytics service (λ_1, stable stage distribution, reproductive value, sensitivities, elasticities) implement non-trivial linear-algebra formulas drawn from Caswell [1]. An implementation error could silently produce numerically valid but biologically meaningless results, which schema validation cannot catch and which could go unnoticed without domain review.		
Category	Technical, Quality		
Probability	Low (30%)		
Impact	Cost	Low (10%)	**0.24**
	Schedule	Moderate (20%)	
	Scope	Low (10%)	
	Quality	Very High (80%)	
Strategy	Mitigate		
Response	Unit-test the analytics service against reference matrices with hand-verified eigenvalues; require the biology co-tutor (SI-5) to review and sign off on the formulas before they are presented as validated; add a regression test pinned to the corrected values whenever a discrepancy is found.		
Status	Open, monitored via `tests/unit/test_analytics_service.py`; not materialised.		

Table 14: Risk T-01 - Simulation and analytics correctness errors

T-02 - Database performance at COMPADRE scale			
Description	The COMPADRE seed loads close to 6,000 matrices into `population_matrices`. Unindexed search and filter queries (by species, kingdom, country, source) issued from the Browse Matrices tab could degrade response times as the catalog grows, especially once user-created custom matrices accumulate on top of the seeded set.		
Category	Technical, Performance & Reliability		
Probability	Medium (50%)		
Impact	Cost	Very Low (5%)	**0.10**
	Schedule	Low (10%)	
	Scope	Low (10%)	
	Quality	Moderate (20%)	
Strategy	Mitigate		
Response	Add database indexes on the commonly filtered columns; paginate `GET /v1/matrices` responses; cache repeated search results at the API layer.		
Status	Open, indexes and pagination in place; monitored under realistic load.		

Table 15: Risk T-02 - Database performance at COMPADRE scale

T-03 - JWT / API security vulnerabilities			
Description	The REST API is the only access path to user data, custom matrices, and simulation results, authenticated via JWT Bearer tokens. A flaw in token handling, password storage, or endpoint authorisation could allow unauthorized access to another user's matrices or simulations.		
Category	Technical, Security		
Probability	Very Low (10%)		
Impact	Cost	Low (10%)	**0.08**
	Schedule	Low (10%)	
	Scope	Very Low (5%)	
	Quality	Very High (80%)	
Strategy	Mitigate		
Response	Use battle-tested libraries for password hashing and JWT signing; keep token expiry short (7 days) and never persist secrets in the database; enforce automated scanning on every push via Bandit (SAST), pip-audit (CVEs), Trivy (container scan), and GitHub CodeQL.		
Status	Open, continuously monitored by `security.yml` and `codeql.yml`; no high-severity findings to date.		

Table 16: Risk T-03 - JWT / API security vulnerabilities

T-04 - Breaking changes from upstream dependency updates			
Description	The project depends on a fast-moving Python ecosystem (FastAPI, SQLAlchemy, NumPy, Pydantic, Shiny for Python). An upstream release can introduce breaking changes that silently alter behaviour or break the build.		
Category	Technical, Technology		
Probability	Medium (50%)		
Impact	Cost	Low (10%)	**0.10**
	Schedule	Moderate (20%)	
	Scope	Very Low (5%)	
	Quality	Low (10%)	
Strategy	Mitigate		
Response	Pin dependency versions in `requirements.txt`; let Dependabot open weekly update PRs individually rather than bulk-upgrading, and require CI (unit + integration + E2E) to pass before any dependency PR is merged.		
Status	Open, Dependabot active; no unresolved breakage to date.		

Table 17: Risk T-04 - Breaking changes from upstream dependency updates

T-05 - Stochastic / Monte Carlo resource exhaustion			
Description	Stochastic simulations and the quasi-extinction Monte Carlo job (`quasi_extinction_service.py`) run many repeated matrix multiplications per request. Without limits, a request with a very large number of steps or Monte Carlo iterations could exhaust memory or block the API process.		
Category	Technical, Performance & Reliability		
Probability	Low (30%)		
Impact	Cost	Very Low (5%)	**0.06**
	Schedule	Low (10%)	
	Scope	Low (10%)	
	Quality	Moderate (20%)	
Strategy	Mitigate		
Response	Cap `n_steps` at 1000 in `SimulationCreate` validation; use NumPy arrays rather than Python lists for population history; run quasi-extinction analyses as an asynchronous, DB-backed background job (`SimulationJob`) so a long-running Monte Carlo run cannot block the request/response cycle.		
Status	Open, caps and async execution in place; not materialised.		

Table 18: Risk T-05 - Stochastic / Monte Carlo resource exhaustion

T-06 - Windows / Linux development environment inconsistency			
Description	Development happens on a Windows host while the application runs in Linux containers. Differences such as line-ending conventions or path handling can produce defects that only appear inside Docker, not in the local editor.		
Category	Technical, Complexity & Interfaces		
Probability	High (70%)		
Impact	Cost	Very Low (5%)	**0.07**
	Schedule	Low (10%)	
	Scope	Very Low (5%)	
	Quality	Low (10%)	
Strategy	Mitigate		
Response	Document known fixes in `CLAUDE.md` (e.g. `entrypoint.sh` must use LF endings); treat Docker Compose as the single source of truth for runtime behaviour rather than the host OS.		
Status	Materialised, `entrypoint.sh` was committed with CRLF endings early in development, which prevented containers from starting until corrected with `sed -i 's/\\r//' entrypoint.sh`. Logged in the Issue Log and fixed; no recurrence since.		

Table 19: Risk T-06 - Windows / Linux development environment inconsistency

External Risks

E-01 - COMPADRE data format or availability change			
Description	The platform's seeded matrix catalog depends entirely on the COMPADRE Plant Matrix Database, distributed as an external `.RData` file with a companion `.parquet` metadata file. A change to that schema, or the file becoming unavailable from its source, would break the seeding pipeline (`db/seed_compadre.py`).		
Category	External, Suppliers & External Services		
Probability	Low (30%)		
Impact	Cost	Low (10%)	**0.12**
	Schedule	Moderate (20%)	
	Scope	High (40%)	
	Quality	Moderate (20%)	
Strategy	Mitigate		
Response	Vendor a pinned snapshot of the COMPADRE `.RData`/`.parquet` files inside the repository rather than re-downloading them at build time; document the expected schema so the ETL step can be adapted quickly if a future COMPADRE release changes it.		
Status	Open, current snapshot vendored and seeding verified; not materialised.		

Table 20: Risk E-01 - COMPADRE data format or availability change

E-02 - University documentation template or format requirement changes			
Description	The TFG document follows formatting and structural guidelines set by the Escuela de Ingeniería Informática. A late change to those requirements (template, required sections, defense format) could force rework close to the submission deadline.		
Category	External, Regulation		
Probability	Very Low (10%)		
Impact	Cost	Very Low (5%)	**0.01**
	Schedule	Low (10%)	
	Scope	Very Low (5%)	
	Quality	Very Low (5%)	
Strategy	Accept		
Response	Monitor official guidance published by the School for the current academic year; keep at least one buffer day in the documentation schedule reserved for template-related adjustments.		
Status	Open, not materialised.		

Table 21: Risk E-02 - University documentation template or format requirement changes

Organizational Risks

O-01 - Sole-developer key-person risk			
Description	Per the OBS (Figure 1), one person performs every role on this project: planning, analysis, development, testing, and documentation. An extended illness or personal emergency has no fallback within the team and would stall all project activity simultaneously.		
Category	Organizational, Resources		
Probability	Low (30%)		
Impact	Cost	Moderate (20%)	**0.24**
	Schedule	Very High (80%)	
	Scope	High (40%)	
	Quality	Moderate (20%)	
Strategy	Mitigate		
Response	Commit and push work-in-progress frequently in small increments so no work is at risk of loss; keep `CLAUDE.md` and this document current enough that work could in principle be resumed after an interruption; notify both tutors proactively if availability is at risk so the schedule can be renegotiated early.		
Status	Open, not materialised.		

Table 22: Risk O-01 - Sole-developer key-person risk

O-02 - Competing coursework workload			
Description	As a 4th-year student, time available for this project competes directly with other concurrent subject deadlines, which are not under the project's control and can shift with little notice.		
Category	Organizational, Prioritization		
Probability	High (70%)		
Impact	Cost	Very Low (5%)	**0.28**
	Schedule	High (40%)	
	Scope	Low (10%)	
	Quality	Low (10%)	
Strategy	Mitigate		
Response	Built-in buffer days absorb short-term schedule pressure; user stories are prioritised with MoSCoW labels (Section 4.1.1.4) so Should/Could items can be paused first without affecting the Must-priority core; progress is tracked weekly against the plan (Section 2.2.1).		
Status	Active management, buffer days are being consumed as planned; no Must-priority scope affected to date.		

Table 23: Risk O-02 - Competing coursework workload

Project Management Risks

PM-01 - Effort estimation / schedule slippage			
Description	Several phases of this solo project (e.g. the requirements and technology research phase, the frontend reactive-state implementation) already ran longer than their initial estimate, a common risk in single-developer academic projects where there is no second estimator to sanity-check planning assumptions.		
Category	Project Management, Estimation		
Probability	High (70%)		
Impact	Cost	Low (10%)	**0.28**
	Schedule	High (40%)	
	Scope	Low (10%)	
	Quality	Very Low (5%)	
Strategy	Mitigate		
Response	Track actual versus planned duration per phase in the Tracking Plan (Section 2.2.1); re-estimate remaining phases from observed velocity rather than the original plan; trim Should/Could-priority scope before touching Must-priority requirements if slippage continues.		
Status	Partially materialised, the requirements/technology research phase and the frontend reactive-state implementation each ran about one week over their original estimate (Section 2.2.1); absorbed using buffer days, no deadline impact to date.		

Table 24: Risk PM-01 - Effort estimation / schedule slippage

PM-02 - Scope creep from stakeholder requests			
Description	The Biology Faculty stakeholders (SI-7, and co-tutor SI-5) are an active source of domain feedback. A reasonable, well-intentioned request for an additional feature outside the agreed Stakeholder Requirements (Section 3.2) baseline could expand scope beyond what the remaining schedule supports.		
Category	Project Management, Control		
Probability	Medium (50%)		
Impact	Cost	Low (10%)	**0.20**
	Schedule	Moderate (20%)	
	Scope	High (40%)	
	Quality	Low (10%)	
Strategy	Mitigate		
Response	Treat the SR-F/SR-NF baseline (Section 3.2) as frozen for the current release; log new requests in the Project Issue Log instead of implementing them immediately; route accepted-but-out-of-scope requests to the Future Work section (Section 8.2) unless a Must-priority item is explicitly descoped to make room.		
Status	Open, not materialised.		

Table 25: Risk PM-02 - Scope creep from stakeholder requests

PM-03 - Tutor / co-tutor feedback delay			
Description	Validation milestones, in particular biological-correctness review from the co-tutor (SI-5) and methodological review from the tutor (SI-4), depend on feedback turnaround that is outside the developer's direct control.		
Category	Project Management, Communication		
Probability	Medium (50%)		
Impact	Cost	Very Low (5%)	**0.10**
	Schedule	Moderate (20%)	
	Scope	Very Low (5%)	
	Quality	Low (10%)	
Strategy	Mitigate		
Response	Schedule bi-weekly check-ins with both tutors; send asynchronous written progress updates so feedback does not require a synchronous meeting; document decisions taken without feedback so they can be revisited once it arrives.		
Status	Open, not materialised.		

Table 26: Risk PM-03 - Tutor / co-tutor feedback delay

2.1.5.3 Risk Register

Table 27 summarises all identified risks. The five risks that fall in the red zone (T-01, O-01, O-02, PM-01, PM-02) each have a dedicated contingency plan in Section 9.1, to be triggered if the risk materialises.

ID	Name	Category	Prob.	Score	Strategy	Status
T-01	Simulation/analytics correctness errors	Technical	30%	0.24	Mitigate	Open
T-02	DB performance at COMPADRE scale	Technical	50%	0.10	Mitigate	Open
T-03	JWT / API security vulnerabilities	Technical	10%	0.08	Mitigate	Open
T-04	Upstream dependency breakage	Technical	50%	0.10	Mitigate	Open
T-05	Stochastic / Monte Carlo resource exhaustion	Technical	30%	0.06	Mitigate	Open
T-06	Windows / Linux environment inconsistency	Technical	70%	0.07	Mitigate	Materialised
E-01	COMPADRE data format / availability change	External	30%	0.12	Mitigate	Open
E-02	University template / format changes	External	10%	0.01	Accept	Open
O-01	Sole-developer key-person risk	Organizational	30%	0.24	Mitigate	Open
O-02	Competing coursework workload	Organizational	70%	0.28	Mitigate	Active
PM-01	Effort estimation / schedule slippage	Proj. Mgmt.	70%	0.28	Mitigate	Partial
PM-02	Scope creep from stakeholder requests	Proj. Mgmt.	50%	0.20	Mitigate	Open
PM-03	Tutor / co-tutor feedback delay	Proj. Mgmt.	50%	0.10	Mitigate	Open

Table 27: Risk register summary. Red-zone risks (score >= 0.18) have a dedicated contingency plan in the Appendix (Section 9.1).

2.1.6 Opportunities

2.1.6.1 Opportunity Identification

The same Risk Breakdown Structure, probability scale, and impact scale used for threats (Section 9.1) also classify **opportunities** (positive risks: events that would benefit the project if they occur). Six opportunities were identified across the four RBS categories, analysed the same way as the risks above, with the exposure score determining the opportunity's zone on the Opportunity Probability-Impact Matrix (Table 71). Strategy follows the PMBOK opportunity responses (Exploit, Enhance, Share, or Accept) in place of the threat responses Avoid, Mitigate, Transfer, or Accept.

Technical Opportunities

OPP-T-01 — Reusable ecological-analytics library			
Description	The analytics service (λ_1, stable stage distribution, reproductive value, sensitivities, elasticities) is already implemented as a self-contained module decoupled from FastAPI request/response concerns. If kept that way, it could be extracted into a standalone, pip-installable Python package for population ecology, useful to other students or researchers beyond this TFG.		
Category	Technical, Quality		
Probability	Low (30%)		
Impact	Cost	Very Low (5%)	**0.12**
	Schedule	Low (10%)	
	Scope	Moderate (20%)	
	Quality	High (40%)	
Strategy	Enhance		
Response	Keep `api/services/analytics_service.py` free of FastAPI-specific types and database access so it can be extracted with minimal refactoring; keep `docs/analytics.md` as a self-contained mathematical reference suitable for an external package's documentation.		
Status	Open, not pursued yet; the module is already structured to make this easy if revisited.		

Table 28: Opportunity OPP-T-01 - Reusable ecological-analytics library

OPP-T-02 — COMPADRE ETL generalises to COMADRE faster than planned			
Description	COMADRE seeding is already identified as Future Work (Section 8.2). If `db/seed_compadre.py`'s schema-driven parsing approach (`.RData` + `.parquet` to ORM rows) transfers cleanly to COMADRE's structure, that future-work item could be completed well ahead of schedule, with comparatively little additional engineering effort.		
Category	Technical, Technology		
Probability	Medium (50%)		
Impact	Cost	Very Low (5%)	**0.10**
	Schedule	Moderate (20%)	
	Scope	Moderate (20%)	
	Quality	Low (10%)	
Strategy	Exploit		
Response	Keep the ETL pipeline's parsing logic schema-driven and well documented (column mapping, stage-name extraction) rather than hard-coded to COMPADRE specifics, so adapting it to COMADRE is a configuration change, not a rewrite.		
Status	Open, not started; COMADRE remains a Future Work item (Section 8.2).		

Table 29: Opportunity OPP-T-02 - COMPADRE ETL generalises to COMADRE faster than planned

External Opportunities

OPP-E-01 — Institutional adoption interest from the Biology Faculty			
Description	The Faculty of Biology stakeholders (SI-7, and co-tutor SI-5) are actively engaged with the project. Positive reception during the TFG defense or earlier demos could turn "Institutional deployment" (currently a Future Work item, Section 8.2) into a real, faculty-sponsored next step rather than a hypothetical one.		
Category	External, Market		
Probability	Medium (50%)		
Impact	Cost	Low (10%)	**0.20**
	Schedule	Low (10%)	
	Scope	High (40%)	
	Quality	Moderate (20%)	
Strategy	Share		
Response	Demonstrate the working platform to interested faculty and students during the TFG defense; keep the Installation and Operations manuals (Section 7.2) detailed enough that a faculty-led rollout requires minimal extra documentation effort from the developer.		
Status	Open, not yet raised with the Faculty beyond the existing collaboration.		

Table 30: Opportunity OPP-E-01 - Institutional adoption interest from the Biology Faculty

OPP-E-02 — COMPADRE maintainers reference or link the tool			
Description	The COMPADRE Plant Matrix Database team (Max Planck Institute for Demographic Research) maintains a public list of tools built on their data. If they became aware of this platform, a mention or link from them would meaningfully increase its visibility and credibility.		
Category	External, Suppliers & Ext. Services		
Probability	Low (30%)		
Impact	Cost	Very Low (5%)	**0.06**
	Schedule	Very Low (5%)	
	Scope	Low (10%)	
	Quality	Moderate (20%)	
Strategy	Accept		
Response	No active outreach planned within the TFG's scope; the COMPADRE database is already credited prominently in the bibliography and in the Browse Matrices tab's source attribution, so the project is already well positioned if contacted.		
Status	Open, not pursued.		

Table 31: Opportunity OPP-E-02 - COMPADRE maintainers reference or link the tool

Organizational Opportunities

OPP-O-01 — Unused buffer days reinvested in extra features			
Description	The 14-day documentation schedule (`schedule.md`) reserves two buffer days for review and polish. If the threats that could consume them (O-02, PM-01) do not materialise, that time becomes available to pull forward Should/Could-priority items instead of just reviewing.		
Category	Organizational, Resources		
Probability	Medium (50%)		
Impact	Cost	Very Low (5%)	**0.10**
	Schedule	Low (10%)	
	Scope	Moderate (20%)	
	Quality	Low (10%)	
Strategy	Exploit		
Response	Keep the MoSCoW-prioritised backlog (Section 4.1.1.4) ranked and ready, so the next Should/Could item can be pulled in immediately if a phase finishes under its planned duration, instead of leaving the time unallocated.		
Status	Open, contingent on O-02/PM-01 not materialising.		

Table 32: Opportunity OPP-O-01 - Unused buffer days reinvested in extra features

Project Management Opportunities

OPP-PM-01 — Tutor check-ins surface co-authorship / domain insight			
Description	The bi-weekly tutor check-ins set up to mitigate the feedback-delay risk (PM-03) are a standing two-way channel, not just a status report. Domain insight volunteered by the biology co-tutor during these sessions could exceed simple validation and point toward a joint publication or a more rigorous ecological analysis than originally scoped.		
Category	Project Management, Communication		
Probability	Low (30%)		
Impact	Cost	Very Low (5%)	**0.12**
	Schedule	Very Low (5%)	
	Scope	Low (10%)	
	Quality	High (40%)	
Strategy	Enhance		
Response	Treat the check-ins as a two-way conversation rather than a one-way status update; explicitly invite and record any domain observations from the co-tutor, even ones outside the immediate requirements, for possible follow-up after the TFG.		
Status	Open, ongoing via the existing bi-weekly check-ins.		

Table 33: Opportunity OPP-PM-01 - Tutor check-ins surface co-authorship / domain insight

2.1.6.2 Opportunity Register

Table 34 summarises all identified opportunities. The one red-zone opportunity (OPP-E-01) has a dedicated exploitation plan in Section 9.1, to be activated if the opportunity arises.

ID	Name	Category	Prob.	Score	Strategy	Status
OPP-T-01	Reusable ecological-analytics library	Technical	30%	0.12	Enhance	Open
OPP-T-02	COMPADRE ETL generalises to COMADRE faster	Technical	50%	0.10	Exploit	Open
OPP-E-01	Institutional adoption interest from Biology Faculty	External	50%	0.20	Share	Open
OPP-E-02	COMPADRE maintainers reference / link the tool	External	30%	0.06	Accept	Open
OPP-O-01	Unused buffer days reinvested in extra features	Organizational	50%	0.10	Exploit	Open
OPP-PM-01	Tutor check-ins surface co-authorship / insight	Proj. Mgmt.	30%	0.12	Enhance	Open

Table 34: Opportunity register summary. The red-zone opportunity (score >= 0.18) has a dedicated exploitation plan in the Appendix (Section 9.1).

2.1.7 Initial Budget

The developer rate used throughout this budget is derived from the gross annual salary of 24,000.00€, adjusted by a Social Security multiplier of 1.3. In Spain the employer's contribution is approximately 30%, broken down as 23.6% **contingencias comunes**, 5.5% **desempleo**, 0.6% **Formación Profesional**, and 0.2% **FOGASA**. Dividing by the standard 1760 Spanish working year (220 days × 8 h/day) yields a real employer cost of **17.73€ per hour**.

The two project supervisors are costed at their gross annual salary of 37,518.88€, which already incorporates all employer Social Security contributions. Dividing by the same 1760 working year yields a supervision rate of **21.32€ per hour**.

2.1.7.1 Direct costs

Direct costs are expenses exclusively and directly generated by this project. Personnel costs represent the largest share. Railway and the domain name are both project-specific and not shared with any other activity.

Item	Unit cost	Quantity	Total
Software development	17.73€ /h	304.5 h	5,398.79€
Project supervision (2 tutors × 1 h/week × 17 weeks)	21.32€ /h	34 h	724.88€
Railway hosting (production)	15.00€ /month	1 month	15.00€
Domain name (1-year registration)	12.00€ /year	1 year	12.00€
Total direct costs			**6,150.67€**

Table 35: Direct costs

2.1.7.2 Indirect costs

Indirect costs are resources shared with other professional activities and therefore cannot be attributed 100% to this project. They are allocated proportionally: development-tool subscriptions (Anthropic Claude and GitHub Pro) are spread over the 4 project duration; laptop depreciation is allocated by time fraction (304.5 h worked out of 1760 h per year ≈ 17.3%); electricity is estimated from the average power draw of the development setup (laptop + monitor ≈ 115 W); and internet access is charged at a 30% professional-use fraction.

Item	Allocation basis	Total
Anthropic Claude Pro	$20/month × 0.92 USD/EUR × 4 months	73.60€
GitHub Pro	$4/month × 0.92 USD/EUR × 4 months	14.72€
Laptop depreciation (€1,200 / 4 years)	304.5 h / 1760 h per year	51.90€
Electricity (115 W average draw)	304.5 h × 0.115 kW × €0.18/kWh	6.30€
Internet (30% professional use)	€40/month × 30% × 4 months	48.00€
Total indirect costs		**194.52€**

Table 36: Indirect costs

2.1.7.3 Labour cost by sprint

Story-point totals from the sprint backlogs are used as a proxy for effort, allocating the 304.5 total development hours (at 1.5 h per story point) across the six sprints proportionally to their point weight (203 pts in total). This reflects the pace variation across the project: Sprint 3 accumulated fewer points over a longer calendar period, consistent with its documentation-heavy focus.

The TFG template recommends applying different hourly rates to different roles (analyst, developer, documenter). Since a single developer performed all roles throughout this project, the rate of **17.73€/h** is applied uniformly.

Sprint	Period	Story pts	Hours	Labour cost
Sprint 0	Feb 11–28, 2026	21	31.5 h	558.50€
Sprint 1	Mar 1–14, 2026	39	58.5 h	1,037.21€
Sprint 2	Mar 15–28, 2026	40	60 h	1,063.80€
Sprint 3	Mar 29 – May 8, 2026	28	42 h	744.66€
Sprint 4	May 9–26, 2026	35	52.5 h	930.83€
Sprint 5	May 27 – Jun 9, 2026	40	60 h	1,063.80€
Total		**203 pts**	**304.5 h**	**5,398.79€**

Table 37: Labour cost by sprint

2.1.7.4 Budget summary

The **Presupuesto de Ejecución Material** (PEM) is the sum of direct and indirect costs and represents the bare execution cost of the project.

General overheads (15% on PEM) cover costs not attributable to any single project: administrative tasks, workspace, general tooling, and accounting. The 15% rate follows standard practice in Spanish engineering project budgets.

A **contingency reserve** (10% on the post-overhead subtotal) covers unforeseen events during development: unexpected technical complexity, third-party API changes, scope adjustments, or estimation errors. It is applied to the subtotal (not the PEM) so that the reserve also accounts for the overhead cost of any additional work.

Direct costs (Table 35)	6,150.67€
Indirect costs (Table 36)	194.52€
Presupuesto de Ejecución Material (PEM)	**6,345.19€**
General overheads (15% on PEM)	951.78€
Subtotal	7,296.97€
Contingency reserve (10% on subtotal)	729.70€
TOTAL COST	**8,026.67€**

Table 38: Budget summary

2.1.7.5 Client budget

The client budget translates the internal cost into the price offered to the commissioning party. It starts from the cost budget total (excl. VAT) and adds the **benefit**, the developer's

profit margin, set at 6% on the PEM following the standard rate in Spanish engineering project budgets. VAT at 21% is then applied to the full amount.

Cost budget excl. VAT (Table 38)	8,026.67€
Benefit (6% on PEM)	380.71€
Total excl. VAT	**8,407.38€**
VAT / IVA (21%)	1,765.55€
CLIENT BUDGET TOTAL	**10,172.93€**

Table 39: Client budget

2.2 Project Execution

2.2.1 Planning Tracking Plan

Progress was tracked against three baselines: the **initial** plan (Sprint 0, scope and sprint structure as defined above), a **mid-project** checkpoint (end of Sprint 3, after component decoupling and the start of TFG documentation), and the **final** baseline (end of Sprint 5, feature-complete). The per-sprint tracking tables and Figure 8 below compare planned vs. actual duration and remaining points per sprint.

This section tracks what actually happened against the plan laid out in Table 5. Three baselines are used to judge progress:

- **Initial baseline**: The WBS estimate at project start: 203 pts spread across 44 tasks in 6 sprints (Table 5), of which 122 pts trace directly to the 24 user stories in Section 4.1.1.4 and the remainder covers setup, architecture, DevOps, and documentation work.
- **Mid-project baseline**: Checkpoint at the end of Sprint 2 (Mar 28, 2026): 100 of the 203 planned points were complete (49%), broadly on track with the cumulative ideal line in Figure 8, with Sprint 3 already showing the widest actual/ideal gap due to its much longer (5.5-week) period.
- **Final baseline**: End of Sprint 5 (Jun 9, 2026): all 203 points were completed; every sprint's actual line in Figure 8 reaches zero by its period end.

2.2.1.1 Sprint 0

Task	Start	End	Status
S0-01	Feb 11, 2026	Feb 15, 2026	Done
S0-02	Feb 14, 2026	Feb 18, 2026	Done
S0-03	Feb 17, 2026	Feb 21, 2026	Done
S0-04	Feb 20, 2026	Feb 24, 2026	Done
S0-05	Feb 23, 2026	Feb 27, 2026	Done
S0-06	Feb 26, 2026	Feb 28, 2026	Done

Table 40: Sprint 0 actual tracking

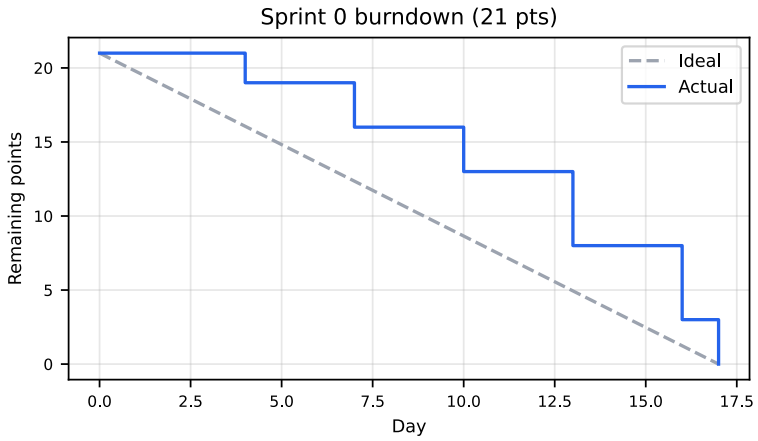

Figure 2: Sprint 0 burndown: ideal vs. actual

2.2.1.2 SPRINT 1

Task	Start	End	Status
S1-01	Mar 1, 2026	Mar 4, 2026	Done
S1-02	Mar 3, 2026	Mar 6, 2026	Done
S1-03	Mar 5, 2026	Mar 8, 2026	Done
S1-04	Mar 7, 2026	Mar 10, 2026	Done
S1-05	Mar 9, 2026	Mar 12, 2026	Done
S1-06	Mar 11, 2026	Mar 14, 2026	Done
S1-07	Mar 13, 2026	Mar 14, 2026	Done

Table 41: Sprint 1 actual tracking

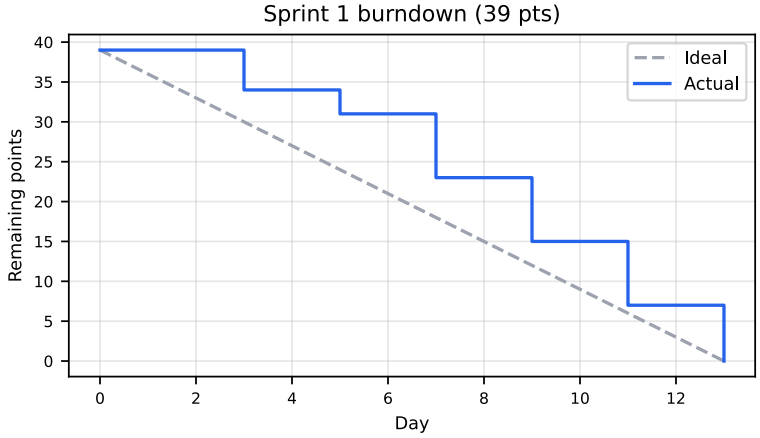

Figure 3: Sprint 1 burndown: ideal vs. actual

2.2.1.3 SPRINT 2

Task	Start	End	Status
S2-01	Mar 15, 2026	Mar 18, 2026	Done
S2-02	Mar 17, 2026	Mar 20, 2026	Done

Task	Start	End	Status
S2-03	Mar 19, 2026	Mar 22, 2026	Done
S2-04	Mar 21, 2026	Mar 24, 2026	Done
S2-05	Mar 23, 2026	Mar 26, 2026	Done
S2-06	Mar 25, 2026	Mar 28, 2026	Done
S2-07	Mar 27, 2026	Mar 28, 2026	Done

Table 42: Sprint 2 actual tracking

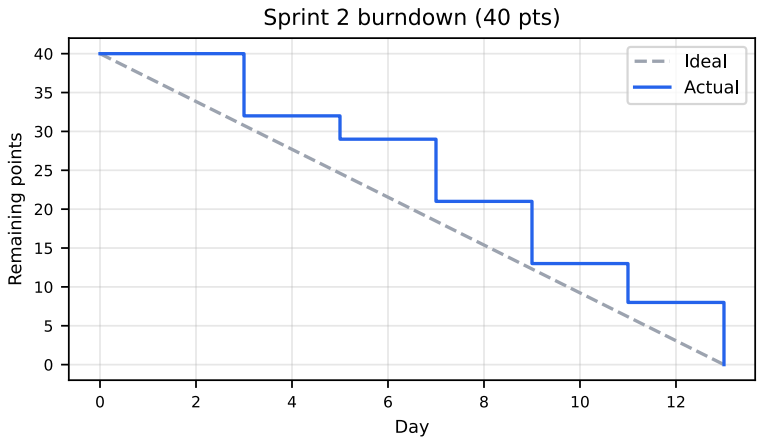

Figure 4: Sprint 2 burndown: ideal vs. actual

2.2.1.4 Sprint 3

Task	Start	End	Status
S3-01	Mar 29, 2026	Apr 6, 2026	Done
S3-02	Apr 3, 2026	Apr 12, 2026	Done
S3-03	Apr 9, 2026	Apr 18, 2026	Done
S3-04	Apr 15, 2026	Apr 24, 2026	Done
S3-05	Apr 21, 2026	Apr 30, 2026	Done
S3-06	Apr 27, 2026	May 6, 2026	Done
S3-07	May 3, 2026	May 8, 2026	Done

Table 43: Sprint 3 actual tracking

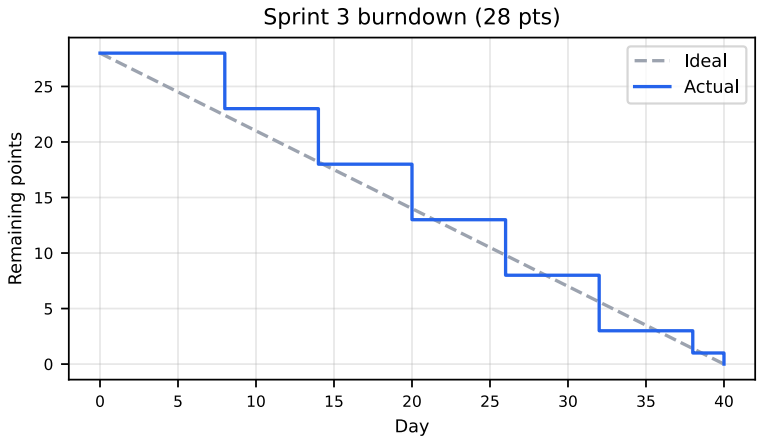

Figure 5: Sprint 3 burndown: ideal vs. actual

2.2.1.5 Sprint 4

Task	Start	End	Status
S4-01	May 9, 2026	May 12, 2026	Done
S4-02	May 11, 2026	May 14, 2026	Done
S4-03	May 13, 2026	May 16, 2026	Done
S4-04	May 15, 2026	May 19, 2026	Done
S4-05	May 18, 2026	May 21, 2026	Done
S4-06	May 20, 2026	May 23, 2026	Done
S4-07	May 22, 2026	May 25, 2026	Done
S4-08	May 24, 2026	May 26, 2026	Done

Table 44: Sprint 4 actual tracking

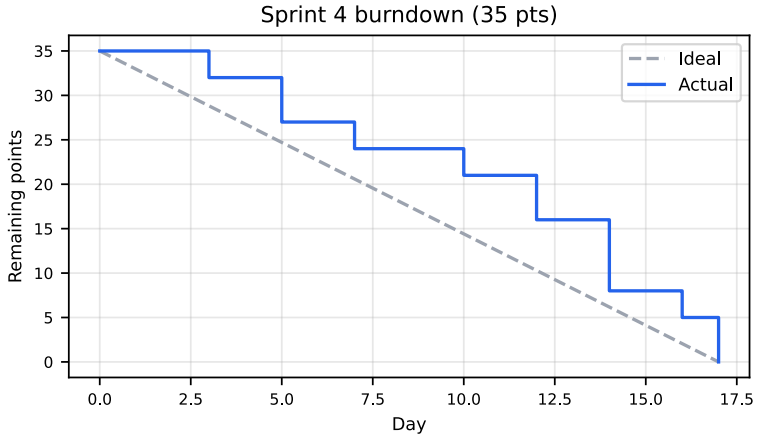

Figure 6: Sprint 4 burndown: ideal vs. actual

2.2.1.6 Sprint 5

Task	Start	End	Status
S5-01	May 27, 2026	May 29, 2026	Done

Task	Start	End	Status
S5-02	May 28, 2026	May 30, 2026	Done
S5-03	May 30, 2026	Jun 1, 2026	Done
S5-04	May 31, 2026	Jun 3, 2026	Done
S5-05	Jun 2, 2026	Jun 4, 2026	Done
S5-06	Jun 3, 2026	Jun 6, 2026	Done
S5-07	Jun 5, 2026	Jun 7, 2026	Done
S5-08	Jun 6, 2026	Jun 9, 2026	Done
S5-09	Jun 8, 2026	Jun 9, 2026	Done

Table 45: Sprint 5 actual tracking

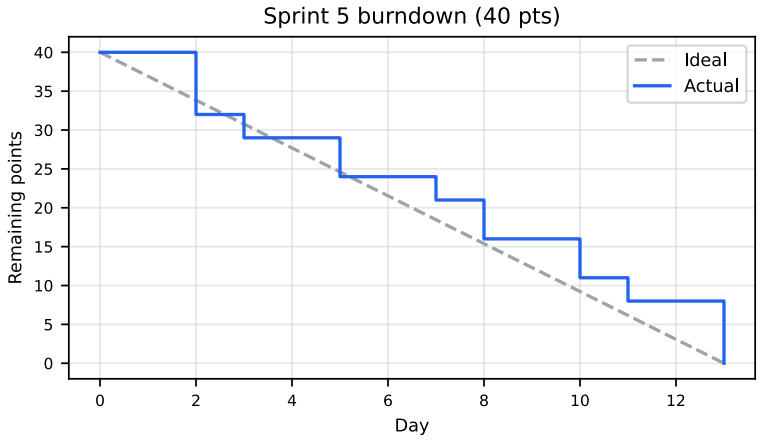

Figure 7: Sprint 5 burndown: ideal vs. actual

2.2.1.7 BURNDOWN

The chart below combines all six sprints into a single project timeline. Each sprint's date range is shaded with a distinct light background colour, matching the order in which the sprints appear in Table 5.

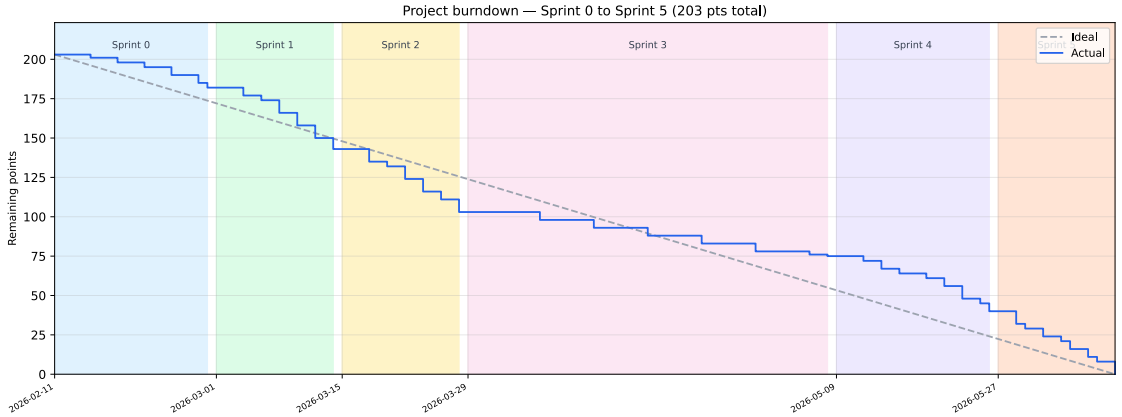

Figure 8: Project burndown (Sprint 0 to Sprint 5): ideal vs. actual remaining points, shaded by sprint

2.2.2 Project Issue Log

Incidents encountered during development with a non-trivial resolution. None of these blocked delivery for more than a few hours; all are also referenced from the relevant risk in Section 9.1 where applicable.

Date	Issue	Impact	Resolution
Sprint 1	`entrypoint.sh` had CRLF line endings on Windows, breaking container startup	Blocked all `docker compose up` runs on the development machine	Converted to LF (`sed -i 's/\r//' entrypoint.sh`)
Sprint 1	`httpx` was missing from `requirements.txt`	Frontend container failed to start (import error)	Added `httpx` to `requirements.txt`
Sprint 1	`docker-compose.yml` needed an explicit `POSTGRES_PORT: 5432` override, since `.env` holds the host-side port (5435)	API container could not connect to the database	Added a per-service port override in `docker-compose.yml`
Sprint 2	A Playwright E2E locator matched elements in both the create and edit matrix panels (e.g. a `.badge` text match)	Flaky / blocked E2E test runs	Scoped locators to the specific panel container (e.g. `#mm_edit_stage_tags .badge`)
Sprint 2	Integration tests were not isolated from the development database	Test runs polluted local development data	Added per-session `matrix_db_test` creation/teardown in `conftest.py`

Date	Issue	Impact	Resolution
Documentation phase	US-11 (edit a custom matrix) was specified and the backend fully implemented it, but the frontend only exposed `common_name`/`country_code` for editing (species, kingdom, matrix cell values, and stage names were not editable)	Acceptance criteria for a Must-priority user story were not actually met by the shipped feature	Extracted a shared `matrix_grid.py` cell-grid editor, extended the edit form to full parity with the create form, added end-to-end test coverage
Documentation phase	Saving an edited matrix produced no visible confirmation, even though the save succeeded server-side (caused by `on_modified()` re-rendering the sidebar and recreating the edit panel's container before the scheduled confirmation message could attach to it)	Edit flow appeared broken to the user despite working correctly	Switched the success message to a toast notification (`ui.notification_show`), matching the pattern already used by the delete flow, which is unaffected by container recreation

Table 46: Project issue log

2.2.3 Risks

Of the thirteen risks identified in Section 2.1.5, two materialised during execution: **T-06** (Windows/Linux environment inconsistency, the `entrypoint.sh` CRLF issue above) and **PM-01** (schedule slippage, the requirements/technology research phase and the frontend reactive-state implementation each ran roughly a week over estimate). Both were absorbed using the schedule buffer without requiring scope reduction; neither reached its contingency trigger. The remaining red-zone risks (O-01, O-02, PM-02) did not materialise. Full risk sheets, including the contingency plans that would have been triggered, are in Section 9.1.

2.3 Project Closure

2.3.1 Final Planning

All five development sprints plus the inception sprint were completed; no planned scope was cut. The documentation phase, originally scheduled as part of Sprint 5, ran longer than planned and continued past the sprint boundary, which is the PM-01 risk (PM-01) above. Section 2.2.1 shows the final planned-vs-actual comparison per sprint.

2.3.2 Final Risk Report

Two risks materialised (T-06, PM-01, both described above); both were resolved without invoking their formal contingency plans. No red-zone risk required schedule reserve beyond ordinary buffer days. See Section 9.1 for the full risk register and the status of every individually tracked risk.

2.3.3 Final Cost Budget

The project was completed within the initial budget estimates. No scope additions, emergency tooling purchases, or unplanned infrastructure costs arose during the four-month development period. The two risks that materialised (T-06 and PM-01, see Section 2.1.5) were absorbed within the contingency reserve without requiring additional expenditure. The final cost budget therefore matches Table 38 in its entirety.

2.3.4 Lessons Learned

- **Environment parity matters even for a solo developer**: the CRLF issue (T-06) was the single largest source of early friction, entirely avoidable with an `.editorconfig` or a pre-commit hook (a concrete improvement identified too late to apply retroactively without risk to a stable pipeline).
- **Writing acceptance criteria does not guarantee they are met**: US-11 frontend gap went unnoticed for months because no automated test exercised the edit flow end-to-end, even though the backend was fully tested. The fix this pass came with full TDD discipline (failing unit and E2E tests written first) precisely because that gap was costly to discover late.
- **Buffer days are worth protecting**: both materialised risks (T-06, PM-01) were absorbed without schedule renegotiation only because buffer days existed and were not pre-allocated to other work.

Chapter 3

STAKEHOLDER REQUIREMENTS

3.1 SYSTEM SCOPE

The system is a **web-based population dynamics simulation platform**, accessible from any modern browser, aimed at biology researchers and students who work with stage-structured demographic matrix models. It encompasses:

- A REST API backend (FastAPI) that manages user accounts, population matrices, simulation runs, and asynchronous analytics jobs.
- A web frontend (Python Shiny) providing an interactive interface for browsing the matrix catalogue, running simulations, and managing custom matrices.
- A PostgreSQL database pre-seeded with the COMPADRE Plant Matrix Database and COMADRE animal database (approximately 18,000 matrices), alongside user-created custom matrices.
- A CI/CD pipeline (GitHub Actions, Section 5.4) for automated testing and security scanning.

Out of scope for this project:

- Native mobile applications.
- Real-time collaboration (concurrent editing of the same matrix by multiple users), identified as future work (Section 8.2).
- Automatic re-synchronisation with COMPADRE (seeding is a one-time operation performed at container build/start time).

3.2 STAKEHOLDER REQUIREMENTS

3.2.1 FUNCTIONAL REQUIREMENTS

SR-F-01. Authentication: The system shall provide mechanisms for users to register, access, and leave the system securely.

SR-F-01.1. User Registration: The system shall allow new users to create an account.

SR-F-01.1.1. Registration Fields: The system shall require a username, an email address, and a password to create an account.

SR-F-01.1.2. Username Uniqueness: The system shall reject a registration attempt if the provided username is already associated with an existing account.

SR-F-01.1.3. Email Uniqueness: The system shall reject a registration attempt if the provided email address is already associated with an existing account.

SR-F-01.1.4. Password Policy: The system shall require a password of at least 8 characters to create an account.

SR-F-01.1.5. Secure Password Storage: The system shall store user passwords in a secure, non-recoverable form. Plain-text passwords shall never be persisted.

SR-F-01.2. User Login: The system shall allow registered users to authenticate and obtain access to protected resources.

SR-F-01.2.1. Login Credentials: The system shall allow users to log in using either their username or their email address, combined with their password.

SR-F-01.2.2. Credential Validation: The system shall reject login attempts where the provided credentials do not match any registered account.

SR-F-01.2.3. Session Credential: Upon successful login, the system shall issue a session credential to the client for use in subsequent authenticated requests.

SR-F-01.2.4. Failed Login Feedback: The system shall return a generic error message on failed login attempts without disclosing which credential was incorrect.

SR-F-01.3. Session Management: The system shall maintain authenticated user sessions across page interactions.

SR-F-01.3.1. Session Persistence: The system shall preserve the user's authenticated session across page reloads without requiring re-authentication.

SR-F-01.3.2. Session Expiry: The system shall automatically expire sessions after a defined period of inactivity.

SR-F-01.3.3. Invalid Session Rejection: The system shall deny access to protected resources when the session credential is absent, invalid, or expired.

SR-F-01.4. User Logout: The system shall allow authenticated users to terminate their session.

SR-F-01.4.1. Session Termination: Upon logout, the system shall revoke the user's active session so that it cannot be reused to access protected resources.

SR-F-02. Population Matrix Catalog: The system shall provide a publicly accessible catalog of population projection matrices sourced from the COMPADRE Plant Matrix Database.

SR-F-02.1. Matrix Browsing: The system shall display a list of available population projection matrices to any visitor without requiring authentication.

SR-F-02.2. Matrix Search and Filtering: The system shall allow visitors to search and filter the matrix catalog by relevant attributes such as species name or taxonomic group.

SR-F-02.3. Matrix Detail View: The system shall display the full details of a selected matrix, including its metadata and numerical values.

SR-F-02.3.1. Matrix Life Cycle Diagram: The system shall display a life cycle diagram alongside the matrix detail view, representing life-history stages as nodes and transition values as directed, labelled edges.

SR-F-03. Matrix Management: The system shall allow authenticated users to create, import, export, and manage their own population projection matrices.

SR-F-03.1. Matrix Creation: The system shall allow an authenticated user to define a new population projection matrix by entering its numerical values and descriptive metadata.

SR-F-03.2. Matrix Import: The system shall allow an authenticated user to load a population projection matrix from a local file.

SR-F-03.3. Matrix Export: The system shall allow an authenticated user to download any matrix accessible to them as a local file.

SR-F-03.4. Matrix Persistence: The system shall allow an authenticated user to save a matrix to their account so that it is stored in the application and accessible across sessions.

SR-F-03.5. Matrix Editing: The system shall allow an authenticated user to modify the values and metadata of matrices they own.

SR-F-03.6. Matrix Visibility Control: The system shall allow an authenticated user to set the visibility of any matrix they own.

SR-F-03.6.1. Public Visibility: A matrix set to public shall be visible and accessible to all users of the application, including unauthenticated visitors.

SR-F-03.6.2. Private Visibility: A matrix set to private shall be visible and accessible only to its owner.

SR-F-03.6.3. Shared Visibility: A matrix set to shared shall be visible and accessible only to its owner and to the specific users with whom it has been shared.

SR-F-03.7. Matrix Sharing: The system shall allow an authenticated user to share any matrix they own with one or more specific registered users.

SR-F-04. Population Simulation: The system shall allow authenticated users to create, configure, run, manage, and export population growth simulations.

SR-F-04.1. Simulation Creation: The system shall allow an authenticated user to create a new simulation by selecting one or more projection matrices, specifying an initial population vector, and choosing the number of time steps to run.

SR-F-04.1.1. Deterministic Simulation: The system shall allow the user to run a deterministic simulation using a single projection matrix, computing the population vector at each time step by multiplying the current vector by the selected matrix.

SR-F-04.1.2. Stochastic Simulation: The system shall allow the user to run a stochastic simulation by selecting two or more projection matrices of matching dimensions and species; at each time step the system shall select one of the matrices at random to advance the population vector.

SR-F-04.2. Matrix Selection for Simulation: The system shall allow the user to select multiple matrices for a stochastic simulation and shall only permit matrices of the same dimensions to be combined in a single simulation run.

SR-F-04.3. Results Visualization: The system shall display the results of a completed simulation as a chart showing the population size per stage across all simulated time steps.

SR-F-04.4. Demographic Analysis: The system shall compute and display key demographic metrics derived from the projection matrix used in a deterministic simulation, or from a mean matrix when working in a stochastic context.

SR-F-04.4.1. Dominant Growth Rate: The system shall compute and display the dominant eigenvalue (λ) of the projection matrix, representing the asymptotic per-capita population growth rate.

SR-F-04.4.2. Stable Stage Distribution: The system shall compute and display the stable stage distribution, derived from the right eigenvector associated with the dominant eigenvalue, representing the proportional composition of each life-history stage at demographic equilibrium.

SR-F-04.4.3. Reproductive Values: The system shall compute and display the reproductive value for each stage, derived from the left eigenvector associated with the dominant eigenvalue, representing the relative contribution of each stage to future population growth.

SR-F-04.4.4. Sensitivity Analysis: The system shall compute and display the sensitivity matrix, quantifying how an absolute change in each matrix element affects the dominant eigenvalue.

SR-F-04.4.5. Elasticity Analysis: The system shall compute and display the elasticity matrix, quantifying the proportional contribution of each matrix element to the dominant eigenvalue.

SR-F-04.5. Stochastic Analysis: The system shall compute and display additional demographic statistics when the user runs a stochastic simulation.

SR-F-04.5.1. Stochastic Population Summary: The system shall display summary statistics of the stage population sizes at the final time step across all replicate simulation runs, including the median and confidence intervals for each stage.

SR-F-04.5.2. Stochastic Growth Rate: The system shall estimate and display the stochastic population growth rate (log λ_s), reporting the theoretical approximation (Tuljapurkar's method), the simulated rate, and confidence intervals for the simulated rate.

SR-F-04.5.3. Quasi-Extinction Probability: The system shall compute and display a quasi-extinction probability curve showing the cumulative probability that the total population size falls below a user-defined threshold at or before each simulated time step, across all replicate runs.

SR-F-04.6. Simulation Persistence: The system shall allow an authenticated user to save a simulation to their account so that its parameters and results are stored in the application.

SR-F-04.7. Simulation Modification: The system shall allow an authenticated user to modify the parameters of a saved simulation and re-run it.

SR-F-04.8. Simulation Export: The system shall allow an authenticated user to download the parameters and results of any of their simulations as a local file.

SR-F-04.9. Simulation Import: The system shall allow an authenticated user to load a previously exported simulation from a local file and restore its parameters and results.

SR-F-04.10. Simulation History: The system shall maintain a history of the authenticated user's saved simulations and allow them to review and manage past runs.

SR-F-04.10.1. History List: The system shall present the authenticated user with a list of their saved simulations.

SR-F-04.10.2. History Detail: The system shall allow the user to view the full parameters and results of any saved simulation.

SR-F-04.10.3. Simulation Deletion: The system shall allow the authenticated user to permanently delete any of their own saved simulations.

3.2.2 Non-Functional Requirements

SR-NF-01. Performance: The system shall respond to any user interaction within 3 seconds under normal load conditions (100 concurrent users).

SR-NF-02. Availability: The system shall be available at least 99.5% of the time, measured on a monthly basis, excluding scheduled maintenance windows.

SR-NF-03. Security: The system shall restrict access to authenticated-only functionalities to registered and logged-in users.

SR-NF-04. Maintainability: The system shall be designed in a modular way so that any individual component can be updated or replaced with minimal impact on the rest of the system.

SR-NF-05. Usability: A new user with basic computer skills shall be able to complete the main tasks without prior training in less than 5 minutes.

SR-NF-06. Portability: The system shall operate correctly on the following web browsers: Chrome, Firefox, Opera, Safari. No client-side installation shall be required beyond a standard web browser.

SR-NF-07. Legal & Regulatory Compliance: The system shall comply with applicable regulations, including GDPR / LOPD-GDD for personal data protection, the LPI and the LSSICE.

3.2.3 System Constraints

System constraints are restrictions imposed on the solution from outside the development team; they are not design choices but fixed conditions that any acceptable solution must satisfy.

SR-C-01. Programming Language: All application components (backend, frontend, and data-access layer) shall be implemented in Python. This constraint was mandated by the Biology Faculty tutor to ensure long-term maintainability by the research group.

SR-C-02. Frontend Framework: The user interface shall be built using the Python Shiny framework. The choice of framework was explicitly specified by the Biology Faculty tutor and is not open to substitution.

SR-C-03. Matrix Data Source: The system shall pre-seed its population matrix catalogue exclusively from the COMPADRE Plant Matrix Database and the COMADRE Animal Matrix Database. No alternative or supplementary matrix source is within scope.

SR-C-04. Delivery Format: The system shall be delivered as a web application accessible from any standard modern browser. Native desktop and native mobile applications are explicitly out of scope.

SR-C-05. Legal and Regulatory Compliance: The system shall comply with all applicable regulations: the General Data Protection Regulation (GDPR) and its Spanish transposition (LOPD-GDD) for personal data protection, the Intellectual Property Law (LPI), and the Law on Information Society Services and Electronic Commerce (LSSICE).

3.3 Alternatives

This section is divided into three subsections: "Solution Alternatives", "Technology Alternatives", and "Deployment Alternatives". The first describes the different solutions presented for this project (web app, desktop app (with Python or with a `.exe` file), a python script or Jupyter notebook...).

The second describes the different technical alternatives considered to build the project, the selected programming language (Python) for the frontend (used Python Shiny as a constraint from the stakeholder), backend (FastAPI), the database selection (relational Postgres database), the migration technology, and the ORM used.

The third compares the cloud deployment platforms evaluated for hosting the application, and justifies the selection of Railway over Microsoft Azure.

3.3.1 Solution Alternatives

Before committing to a specific delivery format, four plausible approaches were evaluated: distributing a plain Python script or Jupyter notebook, packaging the application as a platform-native compiled executable, running the tool as a local desktop application with a graphical interface, and hosting it as a web application accessible through a browser. Each alternative was assessed against the project's primary goals: universal accessibility, integration with the COMPADRE database, multi-user collaboration, and zero installation friction.

3.3.1.1 Plain Python Script / Jupyter Notebook

The simplest possible delivery format would be a Python script (`.py`) or an interactive Jupyter notebook (`.ipynb`) that users download and run locally. The simulation mathematics - matrix multiplication with NumPy [5] - are a natural fit for this environment, and the barrier to *development* is minimal.

However, this approach places the entire setup burden on the end user. Every person wanting to run a simulation must first install Python at the correct version, create a virtual environment, resolve dependency conflicts between NumPy, SciPy [6], and any visualisation library, and keep everything up to date. In an academic context where target users are biology students and researchers - not software engineers - this friction is a significant deterrent. There is also no graphical interface: the user interacts through a terminal or a notebook cell, which is unsuitable for an audience unfamiliar with command-line tools.

Beyond usability, the model offers no persistence or sharing. Each user maintains their own disconnected copy of any saved matrices or simulation results. Integrating the COM-

PADRE Plant Matrix Database [7] (over 6,000 plant matrices; the companion COMADRE animal database contributes a further 12,000) would require either bundling a large static file with every distribution or asking users to fetch and manage it themselves - both unacceptable for a tool meant to serve a broad, non-technical audience. Dependency conflicts (Python version mismatches, `pip` environment pollution) are a recurring source of failure in academic computing environments, making this option fragile in practice.

3.3.1.2 Standalone Desktop Application (.exe)

Packaging the application as a compiled, platform-native executable using tools such as PyInstaller [8] or cx_Freeze removes the requirement for users to have Python installed. The runtime is bundled into the binary, and the user's experience resembles installing any conventional desktop software.

The appeal here is real: a single double-click gets the tool running, with no environment setup and no command line. However, the practical drawbacks are substantial. Because the Python interpreter is bundled, the resulting executable is large - typically 150-300 MB for a moderately complex application. More critically, compiled executables are platform-specific: separate build artefacts must be produced and maintained for Windows, macOS, and Linux, each requiring its own signing and distribution pipeline.

Updates present a further problem. There is no mechanism to push a fix or a new COMPADRE snapshot to installed copies; every user must manually download and reinstall the application. University and corporate IT environments frequently block or quarantine unsigned executables, making deployment to the most common user environment - a university workstation - unreliable. Finally, every installed instance is isolated: there is no shared database, no user accounts, and no way to share a custom matrix between two colleagues running the tool on different machines.

3.3.1.3 Local Desktop Application (Python)

A richer alternative to a raw script is a locally-hosted graphical application built with a Python GUI framework such as Python Shiny [9] (running on localhost), tkinter, or PyQt6. This preserves the use of the Python ecosystem while providing a reactive, point-and-click interface close in feel to the final product.

Python Shiny in particular is attractive because it shares the same programming model as the chosen solution and could, in principle, reuse large portions of the application code. The tool would work offline once installed, and the simulation logic would remain entirely on the user's machine.

In practice, however, this approach still requires the user to install Python, Shiny, NumPy, matplotlib, and all transitive dependencies - precisely the environment setup problem identified above. Python Shiny's local mode launches a local HTTP server and opens a browser tab, but it is not a true desktop application: it depends on available ports, can be disrupted by firewalls, and produces an experience indistinguishable from a web app - without any of the distribution benefits of one. Each user's data remains local, collaborative features are impossible, and the live COMPADRE database still requires a network

connection regardless of the "offline" framing. In effect, this alternative inherits all the drawbacks of the script approach while adding the complexity of a GUI framework.

3.3.1.4 Web Application

The chosen solution is a three-tier web application: a Python Shiny frontend served at port 8080, a FastAPI REST API at port 8000, and a PostgreSQL database

- all orchestrated with Docker Compose [10] and accessible from any modern browser.

This architecture directly satisfies every constraint the other alternatives failed to meet. The user requires nothing beyond Chrome or Firefox; there is no installation, no version management, and no environment to maintain. The COMPADRE database is seeded once on the server and is immediately available to all users, always up to date. Authentication and access controls enable multiple users to own their own matrices and simulations, share matrices with colleagues by username, and persist simulation workspaces across sessions and devices. Deploying a new version of the application - whether a bug fix, a new COMPADRE snapshot, or a new feature - propagates to every user the moment the container restarts, with no client action required.

The API-first design (`/v1/` REST endpoints) means the simulation logic is decoupled from any particular interface, leaving the door open for future clients - a mobile application, a third-party integration, or a command-line tool - without duplicating any business logic. Security practices that are impractical in a distributed desktop application (centralised JWT [11] authentication, automated SAST with Bandit and CodeQL, dependency vulnerability scanning with pip-audit and Trivy) become straightforward in a server-hosted architecture.

The main trade-off is infrastructure dependency: the application requires an internet connection and a running server. For a public educational tool targeting university students and researchers worldwide, this is an acceptable and expected constraint - far less disruptive than requiring every user to manage a local Python environment.

3.3.1.5 Comparison and Justification

Criterion	Script	.exe	Local app	Web app
No installation required	×	✓	×	✓
Works on all platforms	~	×	~	✓
Graphical interface	×	✓	✓	✓
Live COMPADRE database	×	×	×	✓
Persistent user storage	×	×	×	✓
Multi-user collaboration	×	×	×	✓
Centralised updates	×	×	×	✓
Works without Python on client	×	✓	×	✓

Criterion	Script	.exe	Local app	Web app
99.5% availability guarantee	×	×	×	✓
Security pipeline (SAST/CVE scan)	×	×	×	✓

Table 47: Comparison of solution alternatives across key project criteria (✓ = fully satisfied, ~ = partially satisfied, × = not satisfied).

The table makes the outcome clear: the web application is the only alternative that satisfies all ten criteria simultaneously. The non-functional requirement SR-NF-Portability (SR-NF-06) explicitly mandates "Chrome, Firefox, no client installation", which eliminates the script and local desktop options outright. The non-functional requirement SR-NF-Availability (SR-NF-02) demands 99.5% monthly uptime, which requires infrastructure-level guarantees rather than per-user processes. The integration of the COMPADRE database as a shared, always-current resource is only practical when the data lives on a server accessible to all users concurrently. Collaboration features - matrix sharing, simulation persistence across devices, and future team workspaces - are architecturally impossible in any isolated, single-user deployment model.

The additional complexity of the three-tier architecture - compared to a script or a compiled binary - is justified by the breadth of capabilities it unlocks and is itself a demonstration of the full range of software engineering competencies expected of a Final Degree Project: API design, relational database modelling, containerisation, continuous integration, and automated security scanning.

3.3.2 Technology Alternatives

Every technology decision in this project was made deliberately, with concrete alternatives evaluated and rejected for specific reasons. This section documents that process for the six most consequential choices: programming language, frontend framework, backend framework, database, ORM, and migration tooling.

3.3.2.1 Programming Language - Python

The project requires three distinct capabilities from a single codebase: numerical matrix operations at the core of the simulation engine, a REST API serving JSON over HTTP, and a reactive graphical interface. The choice of programming language had to satisfy all three without forcing the use of multiple languages with separate runtimes, toolchains, and dependency trees.

R [12] was the first alternative considered given the project's roots in population ecology. R has a strong statistical computing ecosystem and its own Shiny framework for reactive web applications. However, R's production web server story is fragile - `plumber` APIs are rarely deployed outside of data science teams, and combining R with a relational database, a REST API, and a persistent user system requires bridging to tools that are not idiomatic in the language. The result would be an unusual and hard-to-maintain stack.

Node.js / TypeScript [13] is the dominant choice for REST APIs and single-page applications and would have produced an excellent backend. However, it has no equivalent to NumPy for matrix algebra. Population projection matrices require efficient n-dimensional array operations backed by optimised BLAS routines; implementing these from scratch in JavaScript is neither practical nor appropriate. A hybrid approach - TypeScript API, Python microservice for simulation - would reintroduce cross-language complexity and split the codebase.

Java / Kotlin provides robust concurrency and mature API frameworks (Spring Boot, Quarkus), but brings heavyweight tooling, verbose boilerplate, a JVM runtime requirement, and no first-class scientific computing ecosystem. The NumPy problem remains unsolved.

Python [14] resolves all three requirements simultaneously. NumPy [5] provides BLAS-backed matrix multiplication natively; FastAPI and Python Shiny handle the API and UI within the same runtime; and the entire stack can be managed with a single `requirements.txt`. Python 3.13 was chosen specifically for its interpreter performance improvements and because it matches the version available in the GitHub Actions CI runner.

3.3.2.2 Frontend Framework - Python Shiny

The use of Python for the frontend was a stakeholder constraint set by the Biology Faculty tutor. Within that constraint, three Python-native alternatives to Python Shiny were evaluated.

Streamlit [15] has an exceptionally low learning curve and is the most popular choice for rapid data prototyping. Its rendering model, however, re-runs the entire script on every interaction - a limitation that makes fine-grained state management for multi-step workflows (parameter configuration, simulation run, result inspection) require fragile session-state workarounds. Its layout system defaults to a single column, making a multi-tab application with a sidebar and a chart panel awkward to build.

Dash [16] (Plotly) is purpose-built for data dashboards and integrates naturally with Plotly charts. As the number of reactive components grows, its callback graph becomes difficult to reason about; every input-to-output relationship must be declared explicitly, and circular dependencies or missing callbacks produce runtime errors that are hard to trace. Its layout is defined in verbose Python dict-style HTML, which produces more code than UI.

FastAPI with Jinja2 templates would give full control over the HTML and CSS, but any interactive behaviour beyond a form submission requires JavaScript - directly contradicting the stakeholder's constraint and splitting the frontend into two languages.

The reactive programming model of Python Shiny [9] - where output functions automatically re-execute when their declared inputs change - maps cleanly onto the application's interaction pattern: a set of simulation parameters driving a live population chart. The framework keeps all UI logic in Python, integrates naturally with NumPy arrays and

matplotlib figures, and produces a responsive single-page experience without requiring JavaScript knowledge.

3.3.2.3 Backend Framework - FastAPI

With Python chosen as the language, three web frameworks were evaluated for the REST API: FastAPI [17], Flask [18], and Django REST Framework [19].

Flask is the most minimal of the three. It provides routing and a request/response cycle and nothing else, which means that everything the API needs - input validation, schema serialisation, OpenAPI documentation, and async support - must be assembled from separate third-party packages (marshmallow or pydantic-flask, flask-smorest, and eventually a WSGI → ASGI adapter). For an API with well-defined contracts and a strict layered architecture, this assembly cost is unnecessary overhead that Flask's flexibility does not justify.

Django REST Framework is comprehensive and production-proven. Its built-in ORM, authentication system, admin panel, pagination, and browsable API cover a wide range of use cases. However, Django's ORM is tightly coupled to Django [20] itself: using SQLAlchemy alongside Django creates a dual-ORM situation where migrations, session management, and model definitions are controlled by two separate systems. SQLAlchemy is a hard requirement here because Alembic - the chosen migration tool - depends on it directly, and the repository pattern in the service layer relies on SQLAlchemy sessions. Beyond the ORM conflict, Django's template engine, admin interface, and full-stack conventions are unused weight for a headless API whose only output is JSON.

FastAPI resolves both problems. It generates a live Swagger UI at `/docs` automatically from the route and schema definitions, with no additional packages. Input validation and response serialisation are declared as Pydantic v2 [21] model classes, which FastAPI validates and serialises natively and efficiently (Pydantic v2's Rust-backed core is substantially faster than its predecessor). The framework is async-first, built on Starlette [22], leaving the door open for non-blocking I/O without a rewrite. Its dependency injection system (`Depends`) maps directly onto the controller → service → repository architecture: a DB session, a service instance, and the authenticated user are all resolved and injected per request with a few lines of code in `api/deps.py`. Finally, FastAPI imposes no ORM preference, integrating with SQLAlchemy without conflict.

Framework	Auto OpenAPI docs	Native Pydantic	Async support	ORM flexibility	Setup overhead
FastAPI	✓	✓	✓	✓	Low
Flask	×	×	✓	✓	High
Django REST Framework	✓	×	✓	×	High

Table 48: Backend framework comparison (✓ = supported out of the box, × = not supported or requires extra packages).

3.3.2.4 Database - PostgreSQL

The data model has an unusual characteristic: several columns store structured, variable-length nested data (the population matrix components `matrix_a`, `matrix_u`, `matrix_f`; the simulation trajectory `result_history`; the list of stage labels `stage_names`; and the stochastic matrix index array `matrix_ids`). At the same time, the model relies on relational integrity - foreign keys from `simulation_runs` to `population_matrices`, unique constraints on `users.username` and `users.email`, and transactional guarantees when a simulation record is created alongside its metadata. This combination is the key constraint that drove the database decision.

MySQL / MariaDB [23] is a mature, widely-deployed relational database with comparable performance characteristics to PostgreSQL. However, it does not provide a native JSONB type. Storing variable-length nested structures would require TEXT columns with manual serialisation, losing the ability to index or query into nested fields, or a hybrid approach that offloads document-shaped data to a separate store - reintroducing the complexity that a single unified database is meant to avoid.

SQLite [24] is the natural choice for local development: zero configuration, a single file, and full SQLAlchemy compatibility. It is, however, unsuitable for the production use case. SQLite's write concurrency model uses file-level locking, which serialises all write operations and degrades severely under simultaneous requests from multiple users. It also has no native JSONB type and no production-grade connection pooling.

MongoDB [25] stores documents natively, which would handle the variable-length matrix columns elegantly without any schema overhead. The trade-off is the loss of the relational guarantees that the rest of the data model depends on: there are no enforced foreign keys, no multi-document transactions in the free tier, and no straightforward equivalent to the unique constraints on user credentials. SQLAlchemy's MongoDB support is provided through a separate ODM rather than the standard ORM interface, complicating the repository pattern and the Alembic migration workflow.

PostgreSQL [26] provides both capabilities in a single engine. Its native JSONB type stores nested structures as binary-indexed JSON, allowing efficient querying into nested fields while preserving full relational integrity around them. Version 16 introduces further performance improvements to parallel query execution and JSONB handling that are directly relevant to the query patterns of this application - particularly list queries that filter across both relational columns and JSONB metadata fields.

Database	Native JSONB	Concurrent writes	Relational integrity	SQLAlchemy support	Production-ready
PostgreSQL	✓	✓	✓	✓	✓
MySQL	×	✓	✓	✓	✓
SQLite	×	×	✓	✓	×
MongoDB	✓	✓	×	~	✓

Table 49: Database comparison (~ = partial support via a separate ODM; ✓ = fully satisfied, × = not satisfied).

3.3.2.5 ORM - SQLAlchemy

Three approaches to database access were evaluated: Django ORM, Tortoise ORM, and raw SQL via `psycopg2`.

Django ORM [20] was excluded together with Django REST Framework. Using Django's ORM outside of a Django application requires pulling in the entire framework to initialise the ORM's application registry, which creates lifecycle and configuration conflicts with FastAPI's request handling.

Tortoise ORM [27] is an async-native ORM designed specifically for use with FastAPI and ASGI frameworks. Its API is clean and Pythonic. However, it is a younger project with a smaller ecosystem, and - critically - Alembic, the de-facto standard for SQLAlchemy migrations, does not support Tortoise ORM. Adopting Tortoise ORM would require either a different migration tool (none of which offer `--autogenerate` from model diffs as cleanly as Alembic) or managing migrations manually.

Raw SQL via psycopg2 [28] offers maximum control and the lowest query overhead. The cost is that query logic becomes tightly coupled to the database schema; any schema change requires manually updating every affected query string. More importantly for the test strategy, raw SQL queries cannot be replaced with `unittest.mock.MagicMock` objects - the entire test suite would require a live database connection, making fast, isolated unit tests impossible.

SQLAlchemy [29] uses a declarative mapping style that keeps model definitions readable and co-located with Python type annotations. Its session model integrates with FastAPI's `Depends` injection with a single generator function, scoping each session to the lifetime of a single HTTP request. ORM objects returned by repository methods can be replaced wholesale with `MagicMock` instances in unit tests, enabling the service layer to be tested in complete isolation from the database. This is the foundation of the two-speed test strategy described in the design chapter.

3.3.2.6 Database Migrations - Alembic

Three alternatives to Alembic were considered for managing schema evolution.

Django migrations are excluded together with Django ORM - they are not usable outside the Django framework.

Flyway [30] and **Liquibase** [31] are JVM-based migration tools with strong track records in enterprise Java projects. Both require a separate Java runtime installed alongside the Python application - a significant operational dependency for a Python-only stack. Neither tool has the ability to introspect SQLAlchemy model definitions and auto-generate migration scripts from model diffs, which means every migration must be written by hand in SQL.

Manual SQL scripts give the developer complete control over every DDL statement and avoid any migration framework entirely. The cost is the absence of versioning, rollback paths, drift detection, and the guarantee that the running code and the database schema are in sync. In a containerised deployment where the schema must be updated before

the application starts, manual scripts require a custom orchestration layer that Alembic provides out of the box.

Alembic [32] is the direct companion library to SQLAlchemy and requires no integration work. It generates migration scripts automatically by comparing the current SQLAlchemy model definitions against the database schema (`alembic revision --autogenerate`), versions them sequentially in `alembic/versions/`, and applies them idempotently at container startup via `entrypoint.sh` (`alembic upgrade head`). Two migrations exist in the current codebase: `0001_initial_schema`, which creates the `users` and `population_matrices` tables, and `0002_simulation_runs`, which adds the `simulation_runs` table with its stochastic columns. Both run automatically on every container start, ensuring the database schema is always in sync with the deployed code without any manual intervention.

3.3.3 Deployment Alternatives

Two cloud platforms were evaluated for hosting the three-service stack (PostgreSQL, FastAPI, Python Shiny): **Microsoft Azure** and **Railway**. Both can host containerised web applications and managed databases, but they differ substantially in setup complexity, developer experience, and cost at academic scale.

3.3.3.1 Microsoft Azure

Azure is Microsoft's enterprise cloud platform. Deploying this project on Azure would require provisioning several independent resources: an **Azure App Service Plan** for the compute layer, two **Web App** instances (one for the API, one for the frontend), an **Azure Database for PostgreSQL – Flexible Server** for the database, and **Azure Key Vault** for secret management. Each resource must be configured separately, linked by connection strings, and protected by network security rules. Continuous deployment from GitHub requires either **Azure DevOps Pipelines** or a manually crafted GitHub Actions workflow targeting each service individually. Staging and production environments are not first-class concepts in App Service (they are implemented as **deployment slots**, available only on Standard tier or above).

The main strengths of Azure are its enterprise maturity, globally distributed regions, strong SLAs, and its integration with the wider Microsoft ecosystem (Active Directory, Teams, Office 365). These advantages matter for large-scale institutional deployments, but they also require institutional credentials and introduce operational overhead that is disproportionate for an academic TFG project.

3.3.3.2 Railway

Railway is a modern Platform-as-a-Service designed around the developer-experience principle of minimal configuration. Deployment consists of connecting a GitHub repository to a Railway project; Railway detects the Dockerfiles automatically and builds each service on every push to the tracked branch. A managed PostgreSQL instance is added as a plugin with a single click, and its connection string is injected into the other services via Railway's built-in variable interpolation. Multiple isolated environments (e.g. `production`

tracking `main`, `staging` tracking `dev`) are a first-class feature of every project, with no tier restrictions.

Railway's Hobby plan provides sufficient compute and database storage for the traffic expected during development, demonstration, and initial classroom use, without requiring an institutional subscription or per-resource billing setup.

3.3.3.3 Comparison and Justification

Dimension	Azure	Railway
Setup complexity	High — resource groups, App Service plans, managed DB, Key Vault, networking rules	Low — connect GitHub repo, add PostgreSQL plugin, set environment variables
GitHub integration	Requires Azure DevOps or a custom GitHub Actions workflow per service	Native — redeploy on push, branch-per-environment built in
Environments	Deployment slots (Standard tier+); not built in on free/basic plans	First-class multi-environment support on all plans
Cost at academic scale	Free tier limited; production pricing per vCPU, RAM, and DB storage	Hobby plan sufficient; simple credit-based billing
Vendor maturity	Enterprise-grade, globally distributed, strong SLAs	PaaS focused on developer experience; smaller ecosystem
Academic fit	Overkill; requires institutional Azure subscription	Self-contained; free tier adequate; minimal operational overhead

Table 50: Deployment platform comparison: Azure vs Railway

Railway is selected as the deployment platform. Its GitHub-native continuous deployment, built-in multi-environment support, and zero-configuration PostgreSQL provisioning match the scale and operational constraints of this project without requiring an institutional cloud subscription or dedicated DevOps infrastructure. The live deployment is accessible at https://popgrowthsim.marioorviz.dev and is described in detail in Section 7.2.3.

Chapter 4

System Requirements

4.1 Functional Requirements

4.1.1 System Functions

4.1.1.1 Use Case Diagram

4.1.1.1.1 Overview

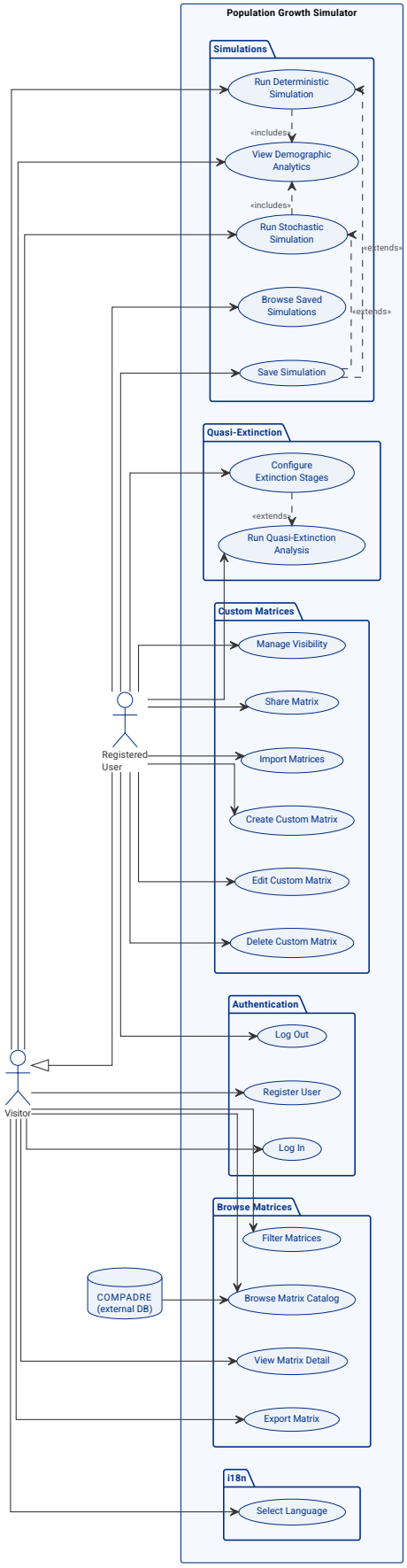

Figure 9: Use Case Diagram - Overview (all actors and subsystems)

4.1.1.1.2 Authentication

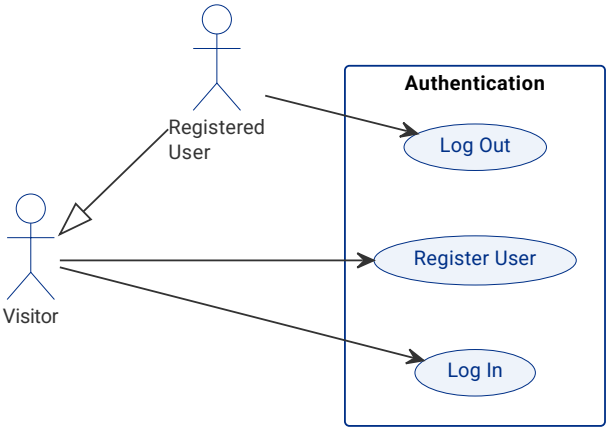

Figure 10: Use Case Diagram - Authentication

4.1.1.1.3 Browse Matrices

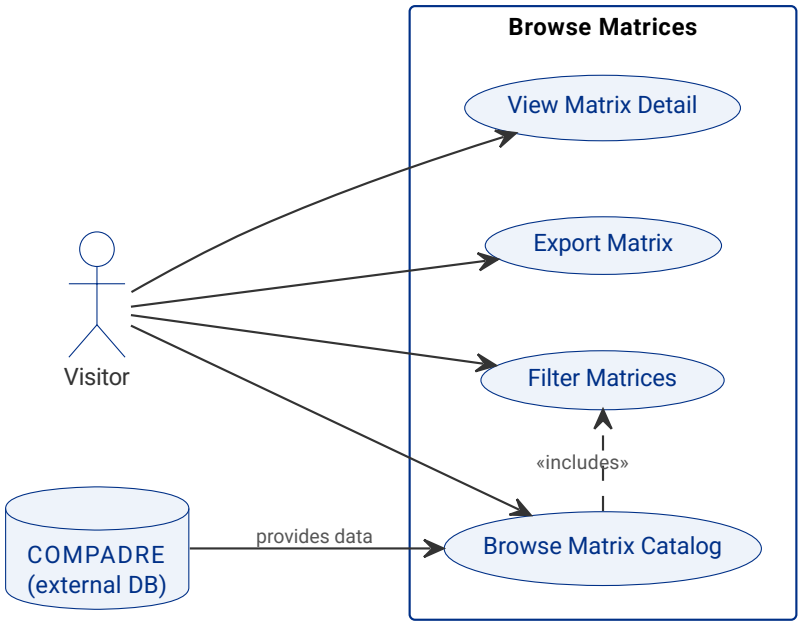

Figure 11: Use Case Diagram - Browse Matrices

4.1.1.1.4 Custom Matrices

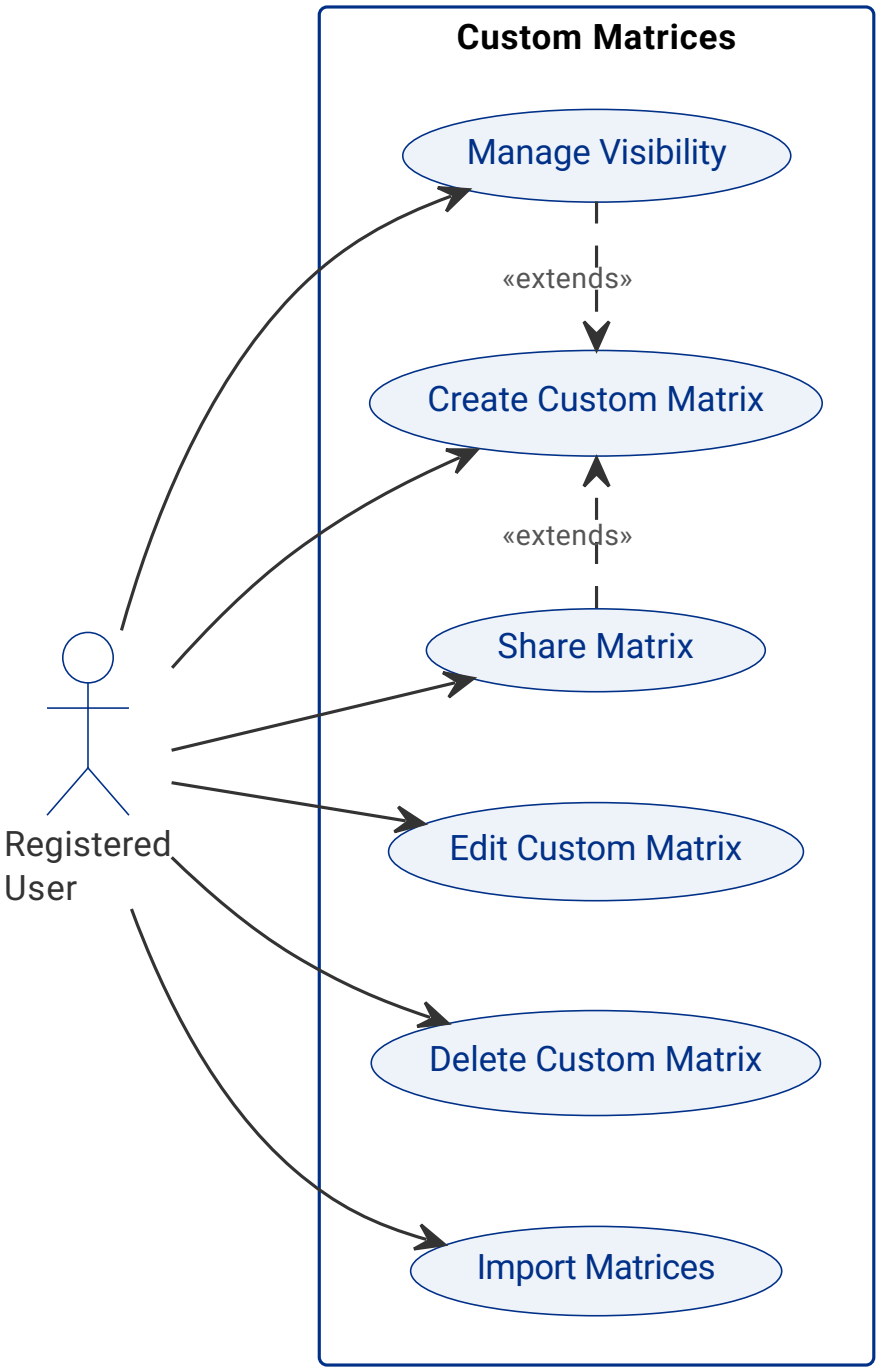

Figure 12: Use Case Diagram - Custom Matrices

4.1.1.1.5 Simulations and Analytics

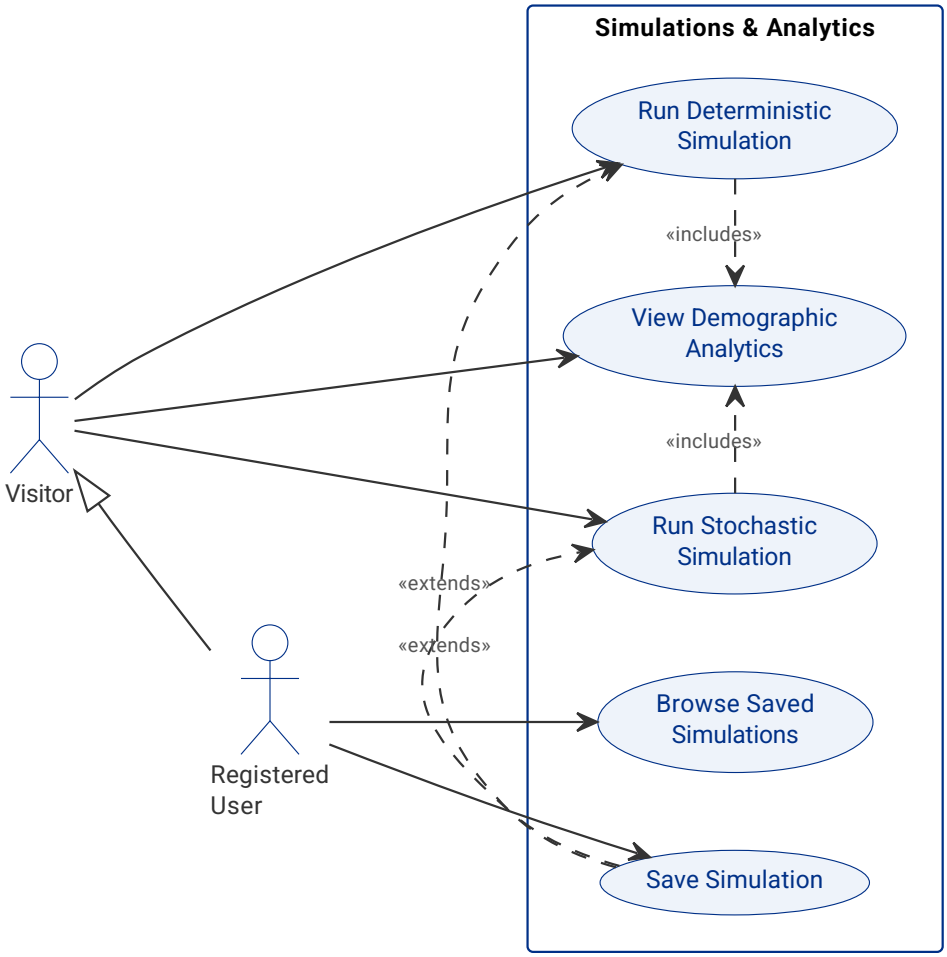

Figure 13: Use Case Diagram - Simulations and Analytics

4.1.1.1.6 Quasi-Extinction Analysis

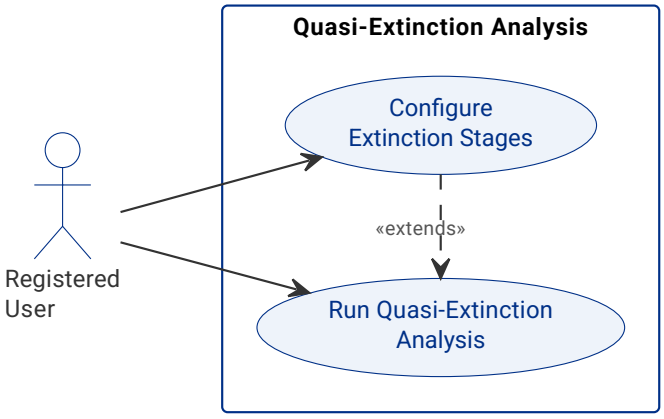

Figure 14: Use Case Diagram - Quasi-Extinction Analysis

4.1.1.2 Use Case Descriptions

UC-01 - Register User	
Description	A visitor creates a new account by providing a unique username, email address, and password, gaining access to the features reserved for registered users.
Actors	Visitor (primary).
Trigger	The visitor clicks the "Register" option in the authentication panel.
Preconditions	The visitor is not authenticated. The registration form is accessible.
Postconditions	A new user account is created; the user can immediately log in with the new credentials.
Normal flow	1. Visitor opens the registration form. 2. Visitor enters a username, email address, and password (minimum 8 characters). 3. Visitor submits the form. 4. System validates that the username and email are unique and that the password meets the length requirement. 5. System creates the user account. 6. System confirms successful registration and presents the login form.
Exceptions	(4') Username or email is already registered → system returns a descriptive error identifying the conflicting field. (4") Password is shorter than 8 characters → system returns a validation error.

UC-02 - Log In	
Description	A registered user authenticates with their credentials and receives a session token that unlocks restricted features.
Actors	Visitor (primary).
Trigger	The visitor clicks the "Log In" option in the authentication panel.
Preconditions	The visitor has a registered account.
Postconditions	The user is authenticated; a 7-day JWT token is issued; restricted features (simulations, custom matrices, quasi-extinction) become accessible.
Normal flow	1. Visitor opens the login form. 2. Visitor enters their username or email address and password. 3. Visitor submits the form. 4. System validates credentials against the stored hash. 5. System issues a JWT token valid for 7 days. 6. System grants the user access to registered-user features.
Exceptions	(4') Credentials are invalid → system returns a generic "Invalid credentials" error (no enumeration of which field is wrong).

UC-03 - Log Out	
Description	An authenticated user terminates their current session, revoking access to restricted features.
Actors	Registered User (primary).
Trigger	The user clicks "Log Out" in the account panel.
Preconditions	The user is authenticated with a valid JWT token.
Postconditions	The session token is cleared client-side; the user is returned to the public view as a visitor.
Normal flow	1. User clicks "Log Out". 2. System clears the session token from the client. 3. System redirects the user to the public Browse Matrices view.

UC-04 - Browse Matrix Catalog	
Description	Any visitor can browse the publicly accessible catalog of population matrices, comprising COMPADRE seeded data and user-created public matrices, displayed in a paginated list with key metadata per entry.
Actors	Visitor (primary); COMPADRE (secondary - provides the seeded data).
Trigger	The visitor opens the “Browse Matrices” tab.
Preconditions	The matrix catalog has been seeded. The API is available.
Postconditions	A paginated matrix list is displayed; each entry shows species name, common name, kingdom, source type, and matrix dimension.
Normal flow	1. Visitor navigates to the “Browse Matrices” tab. 2. System queries the catalog API with default pagination parameters. 3. System displays the paginated list (species name, common name, kingdom, source, dimension). 4. Visitor scrolls or pages through the results.
Alternative flows	(2') Visitor applies filters → UC-05 is triggered and the list refreshes with filtered results.

UC-05 - Filter Matrices	
Description	The visitor narrows the matrix catalog by applying a free-text search and optional dropdown filters (kingdom, source type) to find matrices of interest.
Actors	Visitor (primary).
Trigger	The visitor types in the search box or selects a filter value in the Browse Matrices tab.
Preconditions	The Browse Matrix Catalog view is open (UC-04 is active).
Postconditions	The catalog list is refreshed to show only matrices that match all applied filter criteria.
Normal flow	1. Visitor types a species name or keyword in the search box, or selects a kingdom and/or source type from the filter controls. 2. System sends a query to the API with the applied filter parameters. 3. System updates the matrix list to show only matching results. 4. System displays the number of results found.
Alternative flows	(3') No matrices match the criteria → system displays a “No results found” message and a suggestion to clear filters.

UC-06 - View Matrix Detail	
Description	A visitor selects a matrix from the catalog to inspect its full metadata, projection matrices (A, U, F), stage names, and a rendered life-cycle network diagram.
Actors	Visitor (primary).
Trigger	The visitor clicks on a matrix entry in the catalog list.
Preconditions	The selected matrix exists and is publicly accessible (or shared with the authenticated user).
Postconditions	A detail panel is shown with the full matrix record; the visitor can proceed to export or use the matrix in a simulation.
Normal flow	1. Visitor clicks a matrix entry in the catalog. 2. System fetches the full matrix record from the API. 3. System displays the A matrix as a labelled grid, stage names, and species metadata (kingdom, country, source). 4. System renders the life-cycle network diagram. 5. System shows “Export” and “Simulate with this matrix” controls.

UC-07 - Export Matrix	
Description	A visitor downloads a population matrix from the detail view as a structured JSON or CSV file for use in external tools.
Actors	Visitor (primary).
Trigger	The visitor clicks “Export as JSON” or “Export as CSV” in the matrix detail panel.
Preconditions	A matrix detail view is open.
Postconditions	A file is downloaded to the visitor’s device in the chosen format.
Normal flow	1. Visitor selects “Export as JSON” or “Export as CSV” in the matrix detail panel. 2. System calls the export endpoint with the appropriate format parameter. 3. System returns the file in the requested format. 4. The browser downloads the file to the visitor’s device.

UC-08 - Create Custom Matrix	
Description	An authenticated user defines a new population projection matrix by providing its name, dimensions, stage names, and matrix values. The matrix is stored privately by default.
Actors	Registered User (primary).
Trigger	The user clicks “New Matrix” in the My Matrices tab.
Preconditions	The user is authenticated.
Postconditions	A new custom matrix record is persisted with visibility “private”; it appears in the user’s matrix list and is available for simulations.
Normal flow	1. User navigates to the “My Matrices” tab and clicks “New Matrix”. 2. System presents the matrix creation form. 3. User enters the matrix name, number of stages, stage names, and the projection matrix A values. 4. User optionally fills in the U (survival) and F (fertility) sub-matrices. 5. User submits the form. 6. System validates that all required fields are complete and that matrix values are non-negative. 7. System persists the matrix with visibility set to “private” and the current user as owner. 8. System displays the new matrix in the user’s list.
Alternative flows	(7') User sets visibility to “shared” or “public” before submitting → UC-11 is applied immediately on creation.
Exceptions	(6') A required field is empty or a matrix value is negative → system highlights the invalid fields and blocks submission.

UC-09 - Edit Custom Matrix	
Description	An authenticated user modifies the name, stage names, or matrix values of a custom matrix they own. Previously saved simulations are unaffected because they store a matrix snapshot.
Actors	Registered User (primary).
Trigger	The user clicks “Edit” on a matrix they own in the My Matrices tab.
Preconditions	The user is authenticated; the selected matrix is owned by the user (source_type = “custom”).
Postconditions	The matrix record is updated; all future simulations using it will use the new values; existing simulations retain their snapshot.
Normal flow	1. User clicks “Edit” on a custom matrix. 2. System loads the current matrix values into the edit form. 3. User modifies name, stage names, and/or matrix values. 4. User submits the changes. 5. System validates the updated values. 6. System persists the changes and returns the updated matrix. 7. System confirms the update and refreshes the detail view.
Exceptions	(5') Updated value is negative or a required field is empty → validation error per field. (5'') User attempts to edit a COMPADRE matrix → system returns 403 Forbidden.

UC-10 - Delete Custom Matrix	
Description	An authenticated user permanently removes one of their own custom matrices. Existing simulations that referenced the matrix are unaffected due to snapshot storage.
Actors	Registered User (primary).
Trigger	The user clicks “Delete” on a matrix they own.
Preconditions	The user is authenticated; the selected matrix is owned by the user.
Postconditions	The matrix record is deleted; the matrix no longer appears in any catalog or list; existing simulation snapshots are preserved.
Normal flow	1. User clicks “Delete” on a custom matrix. 2. System presents a confirmation dialog. 3. User confirms the deletion. 4. System deletes the matrix record. 5. System removes the matrix from the user’s list and refreshes the view.
Alternative flows	(3') User cancels the confirmation dialog → no changes are made.
Exceptions	(4') User attempts to delete a COMPADRE matrix → system returns 403 Forbidden.

UC-11 - Manage Matrix Visibility	
Description	An authenticated user changes the visibility level of one of their own custom matrices, controlling whether it is private, shared with specific users, or publicly listed in the catalog.
Actors	Registered User (primary).
Trigger	The user changes the visibility selector on a matrix they own.
Preconditions	The user is authenticated; the matrix is owned by the user.
Postconditions	The matrix visibility is updated; public matrices appear in the public catalog; shared matrices are accessible only to explicitly listed users.
Normal flow	1. User opens a custom matrix in “My Matrices”. 2. User selects the target visibility level (private / shared / public) from the visibility control. 3. System updates the matrix visibility field. 4. System confirms the change.

UC-12 - Share Matrix with User	
Description	An authenticated user grants read access to one of their own custom matrices to another specific registered user, identified by username.
Actors	Registered User (primary).
Trigger	The user clicks “Share” on a custom matrix they own and enters a recipient username.
Preconditions	The user is authenticated; the matrix is owned by the user; matrix visibility is set to “shared”.
Postconditions	A MatrixShare record is created; the recipient can access the matrix in their browse view.
Normal flow	1. User opens a custom matrix and clicks “Share”. 2. System presents the share form. 3. User enters the recipient’s username. 4. User submits the share request. 5. System looks up the recipient by username. 6. System creates a MatrixShare record linking the matrix and the recipient. 7. System confirms the share with the recipient’s username.
Exceptions	(5') Recipient username does not exist ⟶ system returns an error message. (6') The matrix is already shared with this user ⟶ system shows an informational message.

UC-13 - Import Matrices	
Description	An authenticated user bulk-imports one or more custom matrices from a JSON file or ZIP archive. The system processes each entry and reports the outcome per matrix.
Actors	Registered User (primary).
Trigger	The user clicks “Import” in the My Matrices tab and uploads a file.
Preconditions	The user is authenticated; the file is a valid JSON array or a ZIP of JSON files.
Postconditions	Valid matrices are persisted under the user’s account; a BatchImportResult is returned showing successful imports and any per-entry errors.
Normal flow	1. User clicks “Import” in the My Matrices tab. 2. System presents the file upload control. 3. User selects a JSON or ZIP file and submits. 4. System validates the file format and parses matrix definitions. 5. System creates a matrix record for each valid definition, setting the current user as owner. 6. System returns a BatchImportResult listing the number of imported matrices and any skipped entries with reasons. 7. System refreshes the user’s matrix list.
Exceptions	(4') The file format is invalid or cannot be parsed → system returns a descriptive error; no matrices are imported. (5') An individual matrix definition is malformed → that entry is skipped and reported in the result; valid entries are still imported.

UC-14 - Run Deterministic Simulation	
Description	The user selects a single population matrix and an initial population vector, then runs a deterministic simulation to project stage-structured population dynamics and obtain demographic analytics.
Actors	Visitor (primary - ephemeral run); Registered User (primary - may also save the result).
Trigger	User completes the simulation parameter form and clicks “Run”.
Preconditions	At least one matrix is accessible (public, shared, or owned by the user). The API is available.
Postconditions	A line plot of population size per stage over time is displayed; demographic analytics are computed and shown alongside the plot.
Normal flow	1. User selects a matrix from the catalog or from their own matrices. 2. System retrieves the matrix and displays the stage names. 3. User selects “Deterministic” simulation mode. 4. User enters the initial population vector (one non-negative value per stage). 5. User enters the number of time steps (1-1000). 6. User clicks “Run”. 7. System validates input dimensions and value ranges. 8. System computes the trajectory $\boldsymbol{v}(t+1) = \boldsymbol{A} \cdot \boldsymbol{v}(t)$ for each step. 9. System displays a line plot of population size per stage over time. 10. System computes and displays: dominant eigenvalue λ_1, stable stage distribution, reproductive values, sensitivity matrix, and elasticity matrix.
Alternative flows	(10') User clicks “Save” after reviewing the result → UC-16 is triggered to persist the run.
Exceptions	(4') Any component of the initial vector is negative → validation error; submission is blocked. (5') Number of time steps is outside [1, 1000] → validation error; submission is blocked.

UC-15 - Run Stochastic Simulation	
Description	The user selects two or more population matrices of the same dimension and runs a stochastic simulation consisting of N independent runs, each committing to one randomly-chosen matrix for the full time horizon, producing ensemble mean, variance, and min/max trajectories.
Actors	Visitor (primary - ephemeral run); Registered User (primary - may also save the result).
Trigger	User completes the stochastic simulation parameter form and clicks “Run”.
Preconditions	At least two matrices of identical dimension are accessible. The API is available.
Postconditions	The ensemble mean trajectory is plotted (one line per stage) with a shaded min/max band; stochastic analytics are displayed.
Normal flow	1. User selects two or more matrices. 2. System validates that all selected matrices share the same dimension. 3. User selects “Stochastic” simulation mode. 4. User enters the initial population vector. 5. User enters the number of time steps (1-1 000). 6. User enters the number of runs (10-1 000; default 100). 7. User optionally sets a random seed for reproducibility. 8. User clicks “Run”. 9. System executes N independent runs; each run commits to one matrix chosen uniformly at random (once per run) and projects $\boldsymbol{v}(t+1) = \boldsymbol{A}_i \cdot \boldsymbol{v}(t)$ for all T steps. Mean, variance, minimum, and maximum trajectories are computed across all runs. 10. System displays the stochastic trajectory plot: mean trajectory (one line per stage) with a shaded min/max band representing the spread across runs. 11. System computes and displays: stochastic long-run growth rate λ_s, mean projection matrix, and elasticities of the mean.
Alternative flows	(11') User clicks “Save” → UC-16 is triggered; random seed, number of runs, committed matrix index per run, and per-run statistics (variance, min/max histories) are stored for reproducibility.
Exceptions	(2') Selected matrices have mismatched dimensions → system shows an error immediately and prevents the run

UC-16 - Save Simulation	
Description	An authenticated user persists the result of a completed simulation run to their account, storing a matrix snapshot so that the record is immune to future edits or deletions of the source matrices.
Actors	Registered User (primary).
Trigger	User clicks “Save” on a completed simulation result.
Preconditions	User is authenticated; a simulation result is currently displayed (UC-14 or UC-15 has completed).
Postconditions	The simulation run is stored with its parameters, matrix snapshot, result history, and computed analytics; it appears in the user’s saved simulations list.
Normal flow	1. User clicks “Save” in the simulation result view. 2. System prompts the user for an optional run name. 3. User enters a name or leaves it blank and confirms. 4. System stores the run, capturing the matrix values at the time of saving (snapshot). 5. System confirms the save and adds the run to the user’s saved simulations list.

UC-17 - Browse Saved Simulations	
Description	An authenticated user views the list of their previously saved simulation runs and reopens any run to restore its parameters, trajectory plot, and analytics.
Actors	Registered User (primary).
Trigger	User opens the “Library” panel in the Simulate tab.
Preconditions	User is authenticated.
Postconditions	The list of saved runs is displayed; the user can inspect or delete any run.
Normal flow	1. User opens the “Library” view in the Simulate tab. 2. System fetches the user’s saved runs from the API, ordered by creation date descending. 3. System displays the list with run name, creation date, simulation mode (deterministic / stochastic), and matrix names. 4. User clicks a run to reload it. 5. System restores the simulation view: trajectory plot, analytics, and original parameters.
Alternative flows	(4') User clicks “Delete” on a run → system asks for confirmation, then removes the record and refreshes the list.

UC-18 - View Demographic Analytics	
Description	After any simulation is executed, the system automatically computes and displays ecological demographic analytics alongside the population trajectory plot, without requiring a separate navigation step.
Actors	Visitor (primary - analytics are shown on ephemeral runs); Registered User (primary - also on saved runs).
Trigger	A simulation run completes (this use case is included in UC-14 and UC-15).
Preconditions	A simulation result is available.
Postconditions	Demographic analytics are displayed in the simulation result panel.
Normal flow	1. Simulation computation completes (UC-14 or UC-15). 2. System computes analytics server-side from the run parameters and matrix. 3. For a deterministic run: system displays λ_1 (dominant eigenvalue), stable stage distribution, reproductive values, sensitivity matrix, and elasticity matrix. 4. For a stochastic run: system displays λ_s (stochastic growth rate), mean projection matrix, and elasticities of the mean. 5. All metrics are shown in the same view as the trajectory plot.

UC-19 - Run Quasi-Extinction Analysis	
Description	An authenticated user submits an asynchronous Monte Carlo quasi-extinction job to estimate the cumulative probability that a stochastic population falls below a given threshold within a defined time horizon. The job runs in the background and its result is retrieved by polling.
Actors	Registered User (primary).
Trigger	User configures the analysis parameters and clicks “Run Analysis” in the Quasi-Extinction tab.
Preconditions	User is authenticated; at least one matrix is selected; the API is available.
Postconditions	A background job is created; on completion, the system displays a cumulative quasi-extinction probability curve over time.
Normal flow	1. User opens the Quasi-Extinction tab. 2. User selects one or more population matrices. 3. User enters the initial population vector, global extinction threshold, and time horizon. 4. User optionally configures stage-level settings (UC-20). 5. User clicks “Run Analysis”. 6. System submits the job and returns HTTP 202; job status is set to “pending”. 7. System displays a progress indicator. 8. Background task runs the Monte Carlo simulation and updates job status to “running”, then “completed”. 9. System stores the quasi-extinction probability result in the job record. 10. System renders the cumulative quasi-extinction probability curve (probability vs. time step).
Alternative flows	(7') User navigates away from the tab - the job continues running in the background; the progress indicator is shown again when the user returns.
Exceptions	(3') Extinction threshold is ≤ 0 or time horizon is ≤ 0 ⟶ validation error; submission is blocked. (8') Job fails server-side ⟶ system sets job status to “failed” and displays an error message with a retry option.

UC-20 - Configure Extinction Stages	
Description	Before running a quasi-extinction analysis, the user customises which population stages are included in the extinction threshold comparison and optionally sets per-stage threshold values.
Actors	Registered User (primary).
Trigger	User opens the stage configuration panel in the Quasi-Extinction tab.
Preconditions	User is authenticated; at least one matrix is selected in the quasi-extinction form; UC-19 has not yet been submitted.
Postconditions	Stage inclusion flags and per-stage thresholds are recorded in the job parameter object to be submitted with UC-19.
Normal flow	1. User opens the stage configuration panel. 2. System displays all stages derived from the selected matrices, each with an enable/disable toggle and a threshold field. 3. User enables or disables individual stages and enters per-stage threshold values. 4. User confirms the configuration. 5. System records the stage settings in the pending job parameters.
Alternative flows	(4') User clicks "Reset to defaults" → all stages are enabled and the global threshold is restored.

UC-21 - Select Language	
Description	Any user selects the interface language from the six supported options (English, Spanish, Asturian, Galician, Basque, Catalan). The preference is applied immediately for the current session.
Actors	Visitor (primary).
Trigger	User clicks the language selector control.
Preconditions	None.
Postconditions	The interface is rendered in the selected language for the current session.
Normal flow	1. User clicks the language selector. 2. System displays the list of available languages: English (EN), Spanish (ES), Asturian (AS), Galician (GL), Basque (EU), Catalan (CA). 3. User selects a language. 4. System re-renders the interface text in the chosen language.

4.1.1.3 Story Map

Authentication	Browse Matrices	Custom Matrices	COMPADRE	Simulations	Analytics	Quasi-Extinction	Testing & DevOps	i18n
Register	View catalog	Create	Seed catalog	Determ. sim.	Growth rate λ_1	Start job	CI pipeline	Language sel.
Log in	Filter	Edit	Filter metadata	Stoch. sim.	Stable distrib.	Stage config.	Sec. scanning	
Log out	View detail	Delete	Sim. source	Save run	Sensitivities	View results	Docker	
	Export	Visibility		Browse runs			DB migrations	
		Share		Export/import				
		Import						

Table 51: User Story Map

4.1.1.4 User Stories

US-01 **Must** 5 pt

*As a **visitor**, I want to **register a new account with a username, email address, and password**, so that **I can access features reserved for registered users, such as saving simulations and managing custom matrices**.*

Acceptance Criteria

- The system accepts a unique username, a unique and valid-format email address, and a password of at least 8 characters with numbers and letters.
- Attempting to register with an already-used email or username returns a descriptive error message.
- After successful registration, the user can immediately log in with the new credentials.

US-02 **Must** 5 pt

*As a **registered user**, I want to **log in with my email address and password**, so that **access my simulations, custom matrices, and other personalised content**.*

Acceptance Criteria

- The system accepts a valid email and password combination and issues a session token.
- Invalid credentials return a generic error that does not reveal which field is wrong.
- A successful login establishes a session that persists across page reloads.

US-03 **Must** 2 pt

*As a **registered user**, I want to **log out of my current session**, so that **I can protect my account when using a shared or public device**.*

Acceptance Criteria

- Clicking “Log out” immediately invalidates the session token and clears it from the browser.
- After logout, attempting to access any authenticated endpoint redirects to the login prompt.

US-04 **Must** 3 pt

*As a **visitor**, I want to **browse a paginated catalog of population matrices**, so that **I can discover available matrices for my research without needing an account**.*

Acceptance Criteria

• The catalog is visible without authentication.
• Each entry shows the species name, source (COMPADRE, COMADRE or custom), and matrix dimension.
• The list is paginated so that the page loads in under two seconds even with thousands of entries.

US-05 **Must** 5 pt

*As a **visitor**, I want to **search and filter the matrix catalog by species name, taxonomic kingdom, and country of origin**, so that **I'm able to quickly find matrices relevant to my study species without manually scrolling through thousands of entries**.*

Acceptance Criteria

• Text search matches against species name and common name (case-insensitive, partial match).
• All filters act as an AND filter.
• A clear "no results" message appears when the query returns an empty set.

US-06 **Must** 3 pt

*As a **visitor**, I want to **view the full details of a population matrix**, so that **I can inspect the matrix entries and metadata before deciding to use it in a simulation**.*

Acceptance Criteria

• The detail view shows the projection matrix A, survival matrix U, and fertility matrix F as readable grids.
• Stage names are displayed as row and column labels on each matrix grid.
• Ecological metadata (species, kingdom, country, study duration) is shown alongside the matrices.
• A cyclic graph is shown as a graphical representation of the matrix

US-07 **Should** 3 pt

*As a **registered user**, I want to **export a population matrix to a JSON or CSV file**, so that **I can use the matrix data in external tools or archive it for my own records**.*

Acceptance Criteria

- JSON export includes the full matrix data (A, U, F), stage names, and all metadata fields.
- CSV export produces a tidy grid of the A-matrix values with stage names as row and column headers.
- The exported file downloads automatically in the browser without requiring a separate step.

US-08 **Must** 8 pt

*As a **developer**, I want to **have the COMPADRE plant matrix database and COMADRE animal matrix database seeded into the application automatically when the server starts for the first time**, so that **users have immediate access to thousands of real-world population matrices without any manual data entry**.*

Acceptance Criteria

- On first container startup the seed script runs automatically and populates the database with COMPADRE matrices.
- Subsequent startups detect that seeding was already performed and skip the process.
- Seeded matrices are read-only: they are marked with `source_type = "compadre"` and cannot be edited or deleted by any user.

US-09 **Must** 5 pt

*As a **researcher**, I want to **filter the matrix catalog by taxonomic and ecological metadata such as kingdom, country, and species name**, so that **I can efficiently narrow down the matrices relevant to a comparative demographic analysis without manual inspection**.*

Acceptance Criteria

• Filtering by kingdom (e.g., Plantae, Animalia) shows only matrices from that taxonomic group.
• Filtering by country shows only matrices collected in that geographic region.
• Multiple filters applied at the same time narrow the results as AND conditions.

US-10 **Must** 8 pt

*As a **registered user**, I want to **create my own population matrix by specifying stage names and entering the cell values**, so that **I can model populations not covered by COMPADRE or test hypothetical demographic scenarios**.*

Acceptance Criteria

• I can choose the matrix dimension n (n ≥ 2) and enter each cell of the A (projection) matrix.
• I can optionally provide the U (survival) and F (fertility) sub-matrices.
• I can name each life-history stage and add optional metadata (species, country, etc.).
• The matrix is saved to my account and immediately available for simulation.

US-11 **Must** 5 pt

*As a **registered user**, I want to **edit the cell values and metadata (species, common name, kingdom, country, stage names) of a custom matrix I own**, so that **I can correct mistakes or update the demographic model as new field data becomes available**.*

Acceptance Criteria

• Only the owner of a custom matrix may edit it; other users receive a permission error.
• Editing a matrix does not retroactively alter any saved simulation runs that referenced it, because matrix values are captured in an immutable snapshot at run time.
• Changes are persisted immediately and apply to all future simulations using that matrix.

US-12 **Must** 3 pt

*As a **registered user**, I want to **delete a custom matrix that I no longer need**, so that **I can keep my personal matrix library organised and free of outdated models**.*

Acceptance Criteria

• Only the owner can delete a custom matrix.
• Deletion is permanent and requires an explicit confirmation step to prevent accidental loss.
• Deleting a matrix does not delete simulation runs that referenced it, because snapshots preserve the matrix values independently.

US-13 **Should** 5 pt

*As a **registered user**, I want to **set the visibility of my custom matrix to private, shared, or public**, so that **I can control who can see and use my matrix, keeping experimental work private or making finished models available to the community**.*

Acceptance Criteria

• **Private**: only the owner can see and use the matrix.
• **Shared**: only users the owner has explicitly added to the share list can access it.
• **Public**: any visitor can view and use the matrix, regardless of authentication.
• The owner can change the visibility setting at any time without affecting existing simulation snapshots.

US-14 Could 5 pt

*As a **registered user**, I want to **share my custom matrix with specific other registered users by entering their username**, so that **I can collaborate on demographic modelling with colleagues without making the matrix fully public**.*

Acceptance Criteria

- The owner can add a user to the share list by entering their exact username.
- Shared users can view and use (but not edit or delete) the matrix.
- The owner can revoke access for any shared user at any time.

US-15 Should 5 pt

*As a **registered user**, I want to **import one or more population matrices from a JSON or ZIP file**, so that **I can restore previously exported matrices or bulk-load a collection of matrices prepared offline without re-entering data manually**.*

Acceptance Criteria

- A single JSON file imports exactly one matrix.
- A ZIP archive can contain multiple JSON files and imports them all in a single operation.
- After the import, a summary report shows how many matrices were successfully imported and lists any failures with a reason.

US-16 **Must** 8 pt

*As a **registered user**, I want to **run a deterministic simulation by selecting a single population matrix and specifying an initial population vector**, so that **I can project population dynamics over time under a constant-environment assumption and observe the long-run trajectory**.*

Acceptance Criteria

- I can select any matrix I have access to (COMPADRE or custom) as the simulation input.
- I enter the initial population vector (one non-negative value per stage) and the number of time steps (1-1000).
- The system computes the trajectory using the recurrence $\boldsymbol{v}(t+1) = \boldsymbol{A} \cdot \boldsymbol{v}(t)$.
- A line plot showing population size per stage over time is displayed upon completion.

US-17 **Must** 8 pt

*As a **registered user**, I want to **run a stochastic simulation by selecting two or more population matrices of the same dimension**, so that **I can model population dynamics under environmental variability, where each of N independent runs commits to one randomly-chosen environment (matrix) for its full duration, enabling ensemble-based population forecasting**.*

Acceptance Criteria

- I can select two or more matrices of the same dimension; the system validates that all dimensions match.
- The system executes N independent runs (10-50 000, default 100); each run commits to one randomly-chosen matrix and applies $\boldsymbol{v}(t+1) = \boldsymbol{A}_i \cdot \boldsymbol{v}(t)$ for all T steps with the same $\boldsymbol{A}_i$ throughout.
- I can provide a random seed to make the simulation reproducible.
- The result shows the mean population trajectory per stage with a shaded min/max band representing ensemble spread; I can also set the number of runs (10-50 000, default 100).

US-18 **Must** 5 pt

*As a **registered user**, I want to **save a completed simulation run to my account with a descriptive name**, so that **I can retrieve and compare past results at any time without re-running the computation**.*

Acceptance Criteria

• I can name the simulation before or after running it.
• The saved run stores an immutable snapshot of the matrix values used, so future edits to the source matrix do not affect the stored result.
• The saved run appears in my simulations list immediately after saving.

US-19 **Must** 5 pt

*As a **registered user**, I want to **browse my list of saved simulation runs and reload any of them**, so that **I can compare different scenarios or revisit a past analysis without redoing the computation**.*

Acceptance Criteria

• The simulations list shows run name, creation date, type (deterministic or stochastic), and the names of the matrices used.
• Opening a saved run restores the population dynamics plot and ecological analytics exactly as they were at the time of saving.
• I can permanently delete a run I no longer need from the list.

US-20 **Must** 5 pt

*As a **registered user**, I want to **view the ecological analytics computed from my simulation alongside the population trajectory**, so that **I can understand the long-run behaviour of the population (growth rate, stage composition, sensitivity to vital rates) beyond raw trajectory data**.*

Acceptance Criteria

- For deterministic simulations: the dominant eigenvalue λ_1, stable stage distribution, reproductive values, sensitivity matrix, and elasticity matrix are displayed.
- For stochastic simulations: the stochastic long-run growth rate λ_s, the mean projection matrix, and its elasticities are displayed.
- Analytics are shown in the same view as the simulation plot without requiring a separate navigation step.

US-21 **Must** 8 pt

*As a **registered user**, I want to **start a quasi-extinction analysis on a stochastic population scenario**, so that **I can estimate the probability that the population falls below a critical abundance threshold within a given time horizon**.*

Acceptance Criteria

- I can submit a quasi-extinction job by selecting the matrices, initial vector, extinction threshold, and time horizon.
- The job runs asynchronously; I receive an immediate acknowledgement (HTTP 202) and can continue using the application.
- I can poll the job status and view a progress indicator while it is running.
- On completion, a probability-over-time curve is rendered showing the cumulative quasi-extinction probability.

US-22 **Should** 5 pt

*As a **registered user**, I want to **configure per-stage exclusions and minimum thresholds before running a quasi-extinction analysis**, so that **I can focus the extinction criterion on ecologically relevant life stages (e.g., reproductive adults only) and ignore juvenile stages that naturally fluctuate near zero**.*

Acceptance Criteria

- I can assign a minimum abundance threshold to each life-history stage independently.
- I can exclude specific stages from the extinction check entirely.
- The configuration is validated before job submission: at least one stage must be included in the check.

US-23 **Must** 5 pt

*As a **developer**, I want to **an automated CI pipeline to run the full test suite and security scans on every commit and pull request**, so that **I can detect regressions immediately and prevent code with known vulnerabilities or failing tests from being merged into the main branch**.*

Acceptance Criteria

- Unit tests (no database required) run first on every push; a failure blocks the pipeline immediately.
- Integration tests run against a live PostgreSQL service container after migrations are applied.
- Static security analysis (Bandit) and dependency vulnerability scanning (pip-audit) execute as part of the same pipeline run.
- A high-severity security finding or any failing test blocks the pull-request merge.

US-24 **Should** 3 pt

*As a **user**, I want to **select my preferred display language from the application header**, so that **I can use the interface in my native language and have that preference preserved across sessions**.*

Acceptance Criteria

- A language selector is visible in the application header on every tab, regardless of authentication state.
- Selecting a language updates all labels, buttons, and messages immediately without a page reload.
- The selected language is persisted in the browser so it is restored on the next visit.
- Supported languages: English, Spanish (Español), Asturian (Asturianu), Galician (Galego), Basque (Euskara), and Catalan (Català).

4.1.2 Domain Data Model

The application's domain is structured around five entities: **User** (registered account), **PopulationMatrix** (an ecological stage-structured matrix with its species metadata), **MatrixShare** (an access-control record that grants a second user read access to a private matrix), **SimulationRun** (the stored result of a deterministic or stochastic population projection), and **SimulationJob** (a long-running asynchronous analysis such as a quasi-extinction probability run). Figure 15 shows all five domain entities and their primary associations.

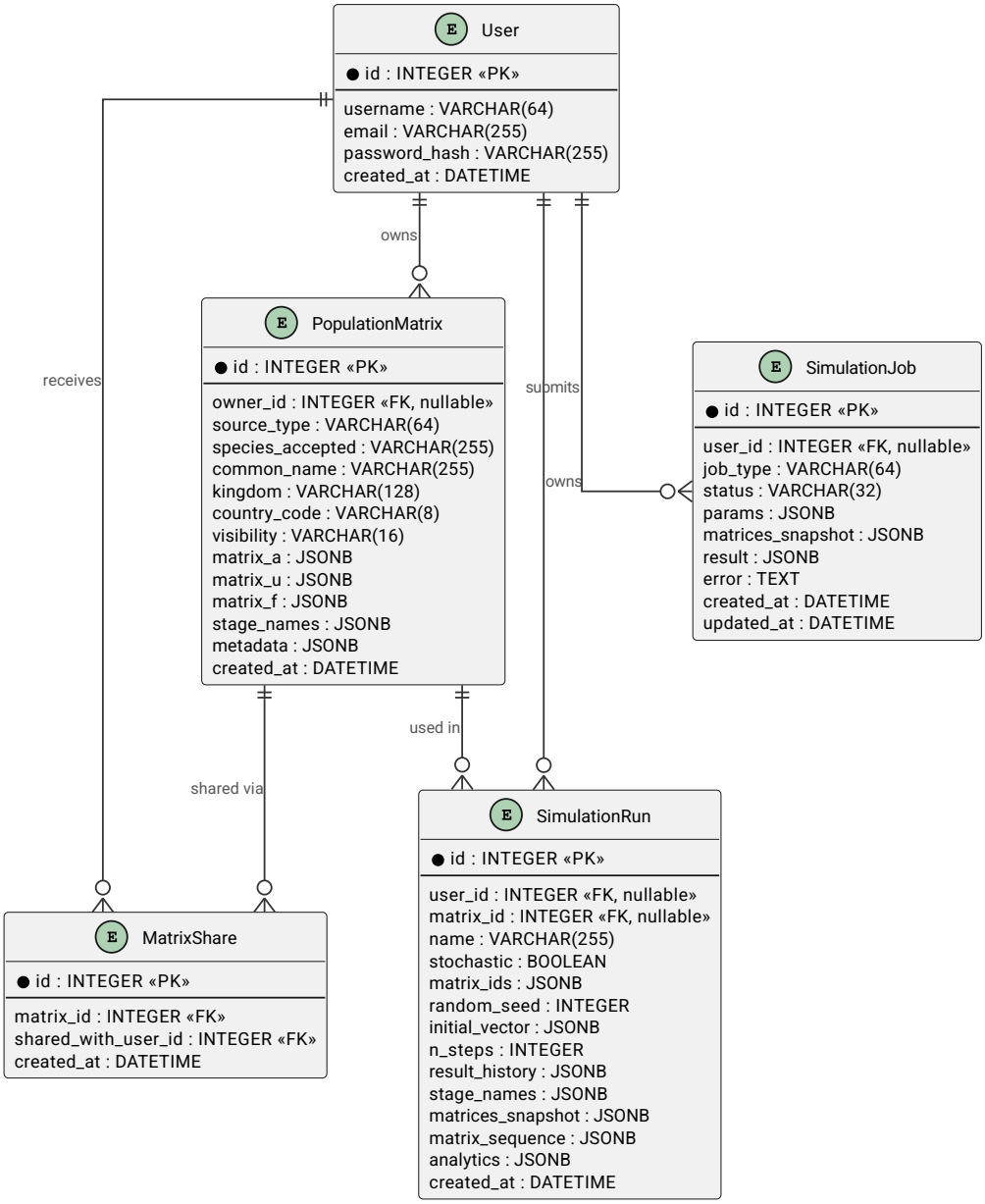

Figure 15: Entity-relationship diagram - all domain entities and their primary associations

4.1.2.1 Entity Catalogue

User

A registered account with a unique username and email address. Users may own custom matrices, save simulation runs, and submit background analysis jobs.

Attribute	Type	Description
`id`	Integer	Surrogate primary key.
`username`	String	Unique display name (3-64 characters, no whitespace). Used for login and sharing.
`email`	String	Unique email address. Used for account creation; not exposed in public API responses.
`created_at`	DateTime	UTC timestamp of account creation.

Table 52: User entity attributes

PopulationMatrix

The central domain object. Represents a stage-structured projection matrix (Leslie or Lefkovitch type) for a specific organism, together with its taxonomic and ecological metadata. Matrices seeded from the COMPADRE/COMADRE databases are read-only; matrices created by users are fully editable and support access-control through a three-tier visibility model.

Attribute	Type	Description
`id`	Integer	Surrogate primary key.
`source_type`	Enum	`"compadre"` - seeded from COMPADRE/COMADRE, read-only. `"custom"` - user-created, editable.
`owner`	→ User	The User who created this matrix. Null for COMPADRE-seeded matrices.
`species_accepted`	String	Accepted binomial or common species name.
`common_name`	String	Vernacular species name (optional).
`kingdom`	String	Taxonomic kingdom (e.g., Plantae, Animalia).
`country_code`	String	ISO 3166-1 alpha-2 country code of the study site.
`matrix_a`	Matrix	Main projection matrix A (square 2-D array of Floats). Required.
`matrix_u`	Matrix	Survival sub-matrix U (optional, same dimension as A).
`matrix_f`	Matrix	Fecundity sub-matrix F (optional, same dimension as A).
`stage_names`	List[String]	Labels for each life-history stage (length = matrix dimension).
`visibility`	Enum	`"private"` - owner only. `"shared"` - owner + explicitly granted users. `"public"` - all users including unauthenticated.
`created_at`	DateTime	UTC timestamp of record creation.

Table 53: PopulationMatrix entity attributes

MatrixShare

A junction entity recording that a specific User has been granted read access to a specific custom matrix whose visibility is `"shared"`. The pair (matrix, grantee) is unique; duplicate grants are rejected.

Attribute	Type	Description
`id`	Integer	Surrogate primary key.
`matrix`	→ PopulationMatrix	The matrix being shared. Cascade-deleted when the matrix is removed.
`shared_with_user`	→ User	The user receiving access. Cascade-deleted when the user account is removed.
`created_at`	DateTime	UTC timestamp when the share was granted.

Table 54: MatrixShare entity attributes (unique constraint on matrix + shared_with_user)

SimulationRun

A persisted population-projection run. Stores the full trajectory (population vector at every time step), a snapshot of the input matrices, and the computed ecological analytics. Snapshotting the matrix data at run time makes stored simulations immune to subsequent edits or deletions of the source matrices.

Attribute	Type	Description
`id`	Integer	Surrogate primary key.
`user`	→ User	Owner of this run. Nullable (unauthenticated ephemeral runs are not stored).

Attribute	Type	Description
`name`	String	Optional user-facing label for the run.
`stochastic`	Boolean	`false` - deterministic (single matrix). `true` - stochastic (multiple matrices; each run commits to one randomly-chosen matrix for all steps).
`matrix_id`	→ PopulationMatrix	Source matrix for deterministic runs. Null for stochastic runs.
`matrix_ids`	List[Integer]	Source matrix IDs for stochastic runs (≥2). Null for deterministic runs.
`initial_vector`	List[Float]	Starting population vector $\boldsymbol{v}(0)$ (one value per life-history stage).
`n_steps`	Integer	Number of projection time steps (1-1 000).
`random_seed`	Integer	PRNG seed for stochastic runs; enables exact reproduction.
`result_history`	List[List[Float]]	Population vector $\boldsymbol{v}(t)$ at each time step $t = 0...n$. For stochastic runs this is the mean population vector across all N runs at each step.
`matrices_snapshot`	List[Matrix]	Copy of `matrix_a` arrays captured at run time; immutable thereafter.
`matrix_sequence`	List[Integer]	Index into `matrix_ids` committed to for each run; one entry per run (length = `n_runs`, stochastic only).
`n_runs`	Integer	Number of independent runs executed (10-1 000, default 100). Null for deterministic runs.
`result_variance`	List[List[Float]]	Variance of the population vector per stage at each time step, computed across all N runs. Null for deterministic runs.
`result_min_history`	List[List[Float]]	Per-stage minimum population value at each step across all runs. Null for deterministic runs.
`result_max_history`	List[List[Float]]	Per-stage maximum population value at each step across all runs. Null for deterministic runs.
`analytics`	Dictionary	Computed ecological metrics: λ_1, stable-stage distribution, reproductive value, sensitivities and elasticities (deterministic); or λ_s, mean-matrix elasticities (stochastic).
`created_at`	DateTime	UTC timestamp of run creation.

Table 55: SimulationRun entity attributes

SimulationJob

A persistent record for a long-running background analysis (currently: quasi-extinction probability estimation via Monte Carlo). The job progresses through a defined status lifecycle. Both the input parameters and the matrix data are captured at submission time so that the background task is independent of any subsequent changes to the source matrices.

Attribute	Type	Description
`id`	Integer	Surrogate primary key.
`user`	→ User	Submitting user. Set to null if the account is later deleted.
`job_type`	String	Identifies the analysis algorithm, e.g., `"quasi_extinction"`.
`status`	Enum	`"pending"` → `"running"` → `"completed"` or `"failed"`. Set by the background worker.
`params`	Dictionary	Full input snapshot captured at submission: matrix IDs, initial vector, threshold, time horizon, stage configuration.
`matrices_snapshot`	List[Matrix]	Copy of `matrix_a` arrays at submission time; the job uses these even if source matrices change.
`result`	Dictionary	Computed output on successful completion (e.g., cumulative quasi-extinction probability curve).
`error`	Text	Human-readable error message on failure.
`created_at`	DateTime	UTC timestamp when the job was submitted.

Attribute	Type	Description
`updated_at`	DateTime	UTC timestamp of the most recent status change; useful for polling.

Table 56: SimulationJob entity attributes

4.1.2.2 Key Domain Rules

Matrix origin and mutability. A matrix with `source_type = "compadre"` originates from the COMPADRE or COMADRE databases and is seeded automatically on application start-up. These matrices are read-only system data: no authenticated user may edit or delete them. Matrices with `source_type = "custom"` are created by registered users through the API and may be freely edited, shared, or deleted by their owner.

Simulation snapshotting. When a simulation run or background job is created, the system copies the `matrix_a` arrays of all input matrices into a `matrices_snapshot` field on the new record. From that point forward, stored simulations and jobs are self-contained: editing or deleting the source matrices has no effect on previously computed results, preserving reproducibility of the scientific record.

Deterministic vs. stochastic mode. A `SimulationRun` is deterministic when a single `matrix_id` is provided: the same projection matrix $\boldsymbol{A}$ is used at every step via $\boldsymbol{v}(t+1) = \boldsymbol{A} \cdot \boldsymbol{v}(t)$. It is stochastic when two or more matrices are provided via `matrix_ids`: at each step a matrix is chosen uniformly at random (seeded by `random_seed`) and the `matrix_sequence` field records which index was selected, making the run exactly reproducible from the stored seed alone.

Async job lifecycle. A `SimulationJob` is created synchronously with status `"pending"`; the HTTP response is returned immediately (HTTP 202 Accepted). A background worker then claims the job, sets status to `"running"`, and on completion writes either the `result` payload (status `"completed"`) or an `error` message (status `"failed"`). The `updated_at` timestamp changes on each transition and is the intended polling signal for the client.

4.1.3 User Interface

The Population Growth Simulator is delivered as a **single-page application** (SPA) running at `http://localhost:8080`. The interface exposes four top-level sections - **Browse Matrices**, **Simulate**, **Quasi-Extinction**, and **My Matrices** - accessible via a persistent navigation header with a dark-green background. Session state (authentication token and username) is persisted in the browser's `localStorage`, so a page reload does not require re-login.

Authentication is implemented through **modal dialogs** that overlay the current view. There are no dedicated login or registration pages; the modals open on top of whatever tab the user is on, allowing their context to be preserved after signing in.

The interface supports six languages selectable from a header dropdown: English, Spanish (Español), Asturian (Asturianu), Galician (Galego), Basque (Euskara), and Catalan (Català). The selected language is applied immediately and persisted across sessions.

4.1.3.1 Application Navigation

Table 57 summarises the navigability trace of the application. Unless noted otherwise, transitions between main sections are performed by clicking the corresponding header tab.

Screen / View	Entry point	Exits to	Auth
Browse Matrices	App start (default tab)	Matrix Detail, Log-In modal, Sign-Up modal	No
Matrix Detail	Click any row in Browse results	Browse Matrices (back arrow)	No
Log-In modal	"Log In" button in any tab header	Dismisses in-place; user becomes authenticated	-
Sign-Up modal	"Sign Up" button in any tab header	Dismisses showing success message; then Log-In	-
Simulate - Run view	"Simulate" tab or "New simulation" button	Simulate - Library view (toggle)	No[1]
Simulate - Library view	"Library" toggle or click a saved run	Simulate - Run view (re-run)	Yes
My Matrices	"My Matrices" header tab	Create/edit matrix form, import dialog	Yes
Quasi-Extinction	"Quasi-Extinction" header tab	New analysis form, past analyses	Yes

Table 57: Application navigability trace

4.1.3.2 User-Story Traceability

Table 58 maps each user-facing user story to the UI screen that realises it. Stories US-08 (COMPADRE seeding) and US-23 (CI pipeline) are back-end and infrastructure concerns with no direct UI representation and are therefore omitted.

User Story	UI Screen
US-01 - Register	Sign-Up modal
US-02 - Log in	Log-In modal
US-03 - Log out	"Sign Out" link in account area (visible in all tabs after login)

[1] Saving a run requires login; running ephemerally does not.

User Story	UI Screen
US-04 - Browse catalog	Browse Matrices tab - results list
US-05 - Search and filter	Browse Matrices tab - filter bar (Species, Kingdom, Source)
US-06 - View details	Browse Matrices → Matrix detail panel
US-07 - Export matrix	Browse Matrices → Matrix detail panel - Export JSON / Export CSV
US-09 - Filter metadata	Browse Matrices tab - Kingdom and Source dropdowns
US-10 - Create matrix	My Matrices tab - create matrix form
US-11 - Edit matrix	My Matrices tab - edit form
US-12 - Delete matrix	My Matrices tab - delete action with confirmation
US-13 - Visibility	My Matrices tab - visibility selector (private / shared / public)
US-14 - Share matrix	My Matrices tab - share with users by username
US-15 - Import matrices	My Matrices tab - import from JSON / ZIP file
US-16 - Deterministic sim.	Simulate tab - step 2: Deterministic mode selected
US-17 - Stochastic sim.	Simulate tab - step 2: Stochastic mode; step 3: add matrices
US-18 - Save simulation	Simulate tab - "Save as new" button (step 4)
US-19 - Browse simulations	Simulate tab - Library view (sidebar list)
US-20 - Analytics	Simulate tab - results panel (dynamics chart + analytics)
US-21 - QE job	Quasi-Extinction tab - new analysis form
US-22 - Stage config.	Quasi-Extinction tab - stage threshold configuration modal
US-24 - Language	Language selector (header dropdown, visible on every tab)

Table 58: User-story to UI screen traceability

4.1.3.3 Authentication

Authentication is accessible from any tab via the **"Log In"** and **"Sign Up"** buttons in the header. The sign-up form (US-01, Figure 16) collects username, email address, and a password of at least eight characters. Duplicate usernames or emails produce an inline error (Figure 17) without exposing which value conflicts. On success the modal shows a confirmation message (Figure 19) and prompts the user to log in.

The log-in form (US-02, Figure 20) accepts username and password. Failed attempts display a generic red-text error that does not reveal which field is wrong (Figure 21). After a successful login (Figure 22) the header replaces the auth buttons with the signed-in username and a **"Sign Out"** link (US-03).

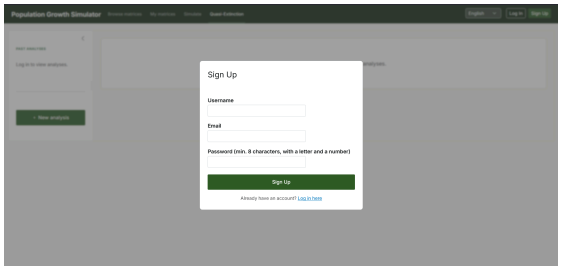

Figure 16: Sign-Up modal - empty form (US-01)

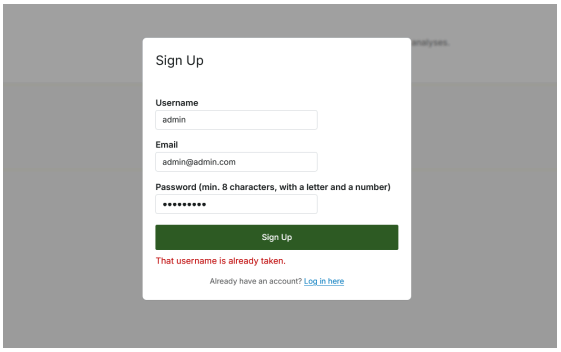

Figure 17: Sign-Up - duplicate-credential error (US-01)

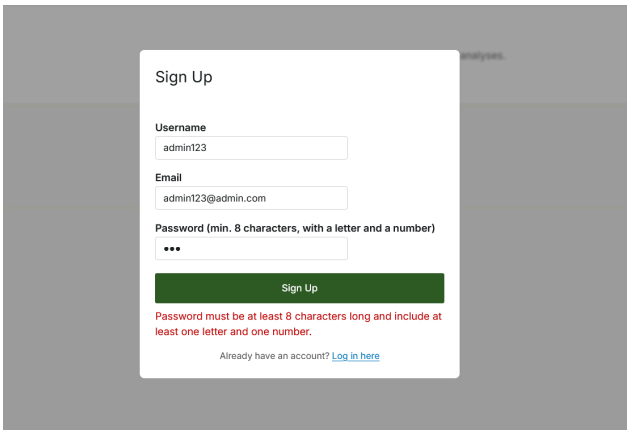

Figure 18: Sign-Up - invalid password error: must be ≥ 8 characters and include at least one letter and one number (US-01)

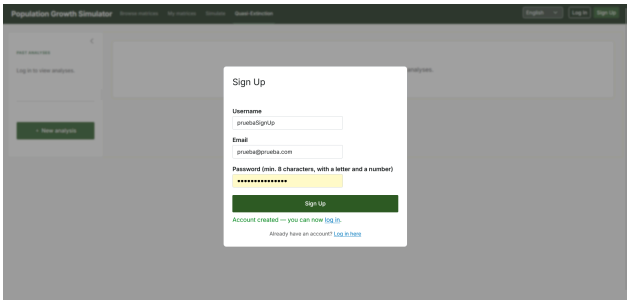

Figure 19: Sign-Up - account created successfully (US-01)

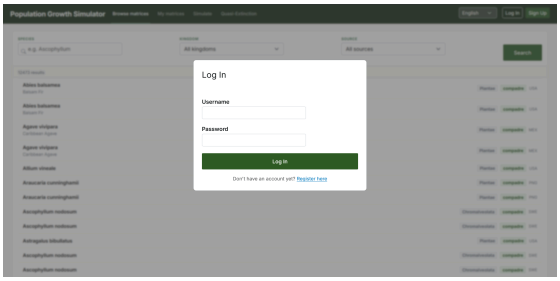

Figure 20: Log-In modal - initial state (US-02)

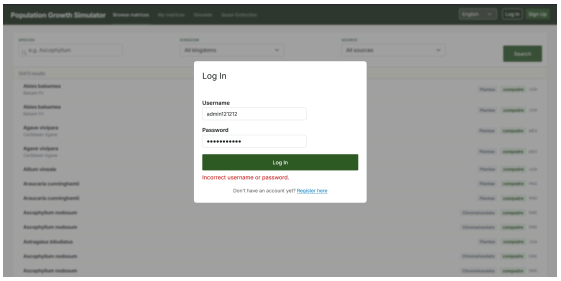

Figure 21: Log-In - invalid credentials error (US-02)

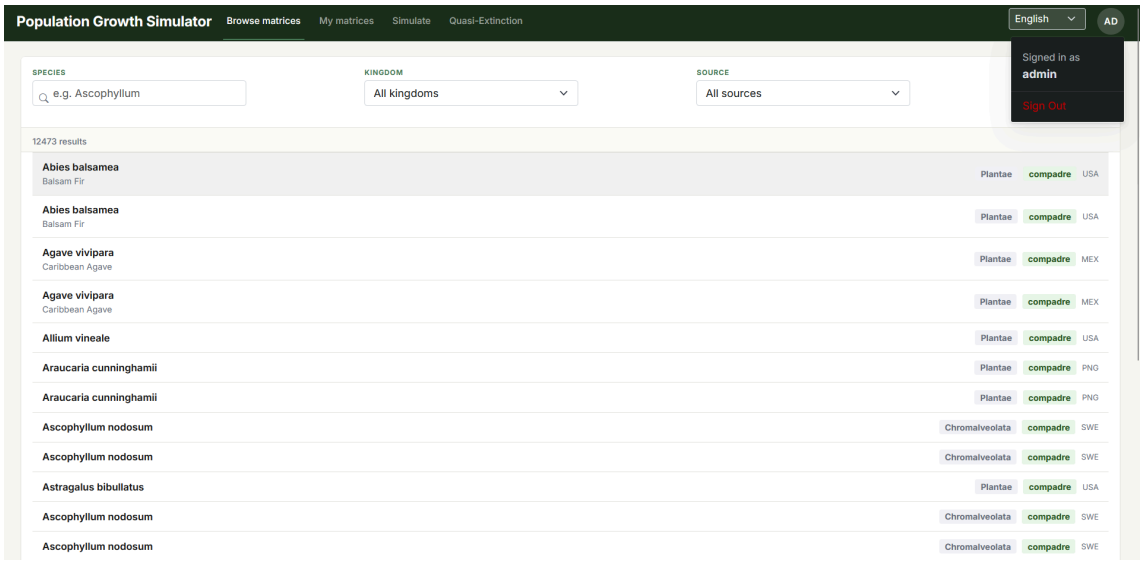

Figure 22: Browse Matrices after successful login - username visible in header, Sign Out link available (US-02, US-03)

4.1.3.4 Browse Matrices

The Browse Matrices tab (Figure 23) is the application's default landing view and requires no authentication (US-04). It displays a paginated list of population matrices from the COMPADRE plant database, the COMADRE animal database, and any public custom matrices created by registered users - 15 783 entries in the default view. Each row shows species name, taxonomic kingdom, data source, and country of origin.

The filter bar (US-05, US-09) provides a free-text species search and two dropdowns: **Kingdom** and **Source**. When multiple filters are active they combine as AND conditions. Results update on each filter change.

Clicking any row navigates to the **Matrix detail panel** (US-06, Figure 24), which shows: the projection matrix **A**, survival matrix **U**, and fecundity matrix **F** as grids with stage names as headers; a **life-cycle network diagram** that renders transitions as an interactive directed graph with physics simulation; and full species metadata. Two export buttons allow downloading the matrix as JSON or CSV (US-07).

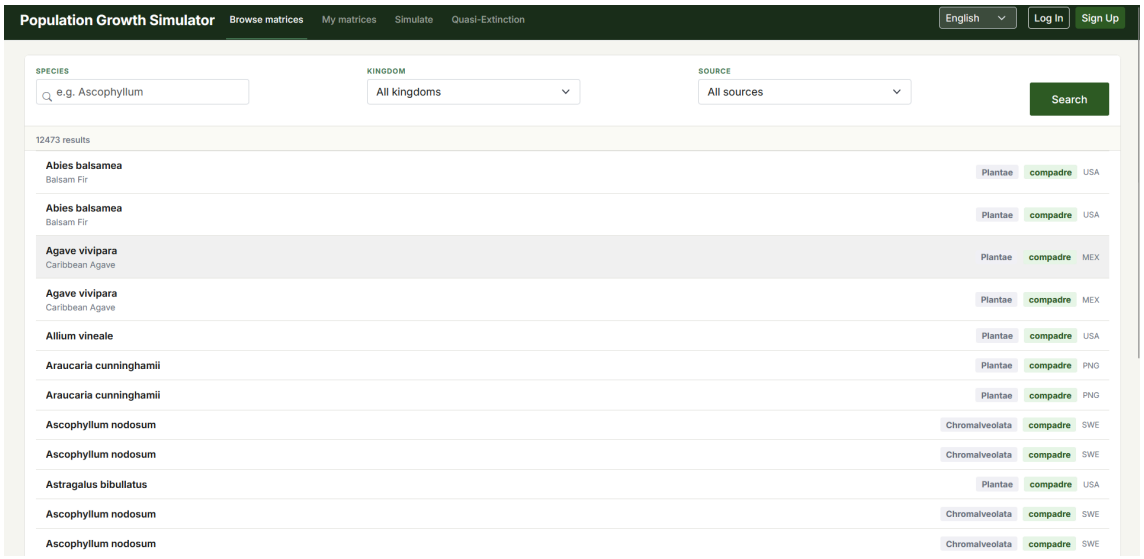

Figure 23: Browse Matrices tab - unauthenticated view with 15 783 results and filter controls (US-04, US-05, US-09)

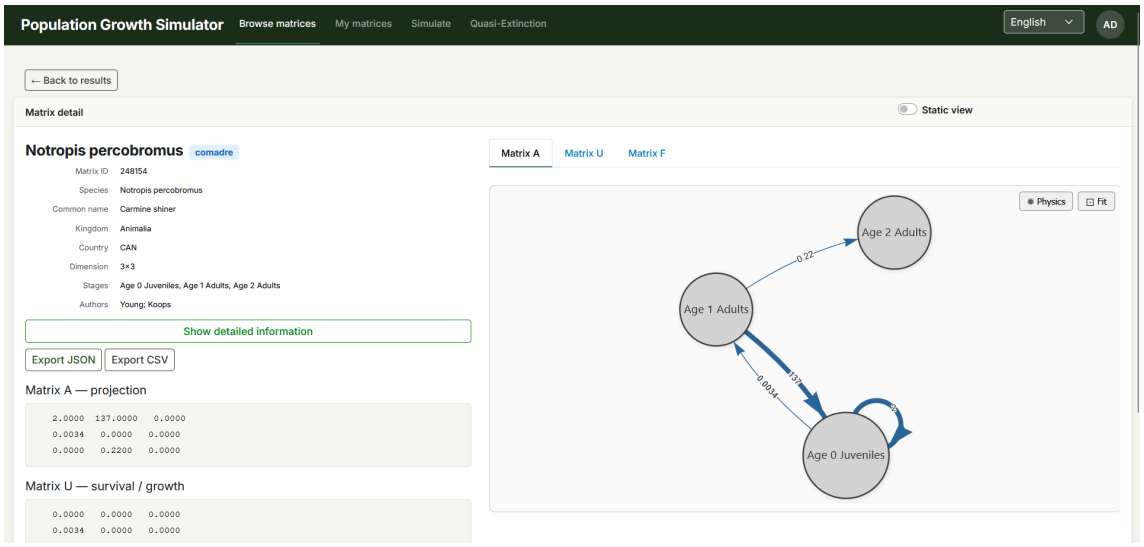

Figure 24: Matrix detail panel - A/U/F matrix tabs, life-cycle network diagram, and export buttons (US-06, US-07)

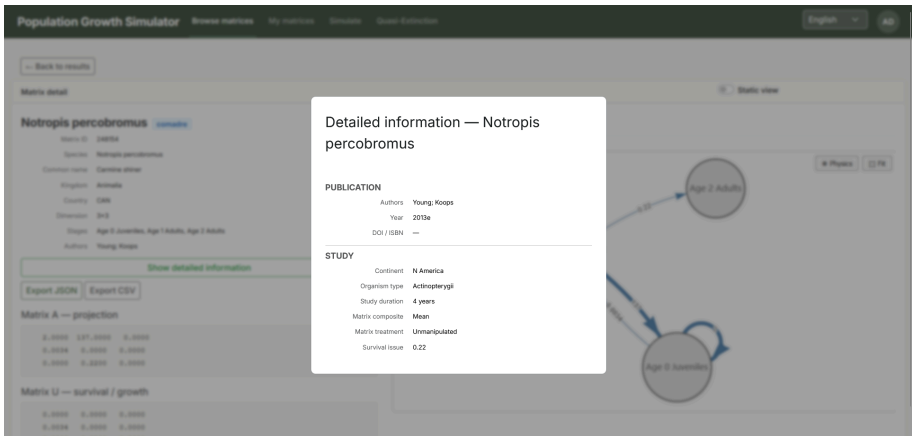

Figure 25: Matrix detail - additional metadata panel showing publication source, study duration, and organism parameters (US-06)

4.1.3.5 SIMULATE

The Simulate tab is the core analytical workspace (Figure 26 through Figure 32). Unauthenticated users may run ephemeral (unsaved) simulations; saving results requires login. The tab offers two views switched by a **Run / Library** toggle.

The **Run view** is built around a four-step sidebar:

1. **Matrix** - species search field and scrollable results list to select the simulation input.
2. **Mode** - radio button for **Deterministic** (one matrix, US-16) or **Stochastic** (multiple matrices, US-17).
3. **In Simulation** - list of currently selected matrices; additional matrices can be appended for stochastic runs.
4. **Parameters** - initial population vector (one value per life-history stage), number of time steps (1-50 000), optional random seed for reproducibility, and an optional run name (US-18).

Submitting the form computes the trajectory and renders a population-dynamics line chart (one line per stage) alongside the ecological analytics panel (US-20).

The **Library view** lists all saved simulation runs in the sidebar (US-19). Opening a run restores the population-dynamics chart, analytical results, and a numerical data table. Saved runs can be deleted from the Library via a confirmation dialog.

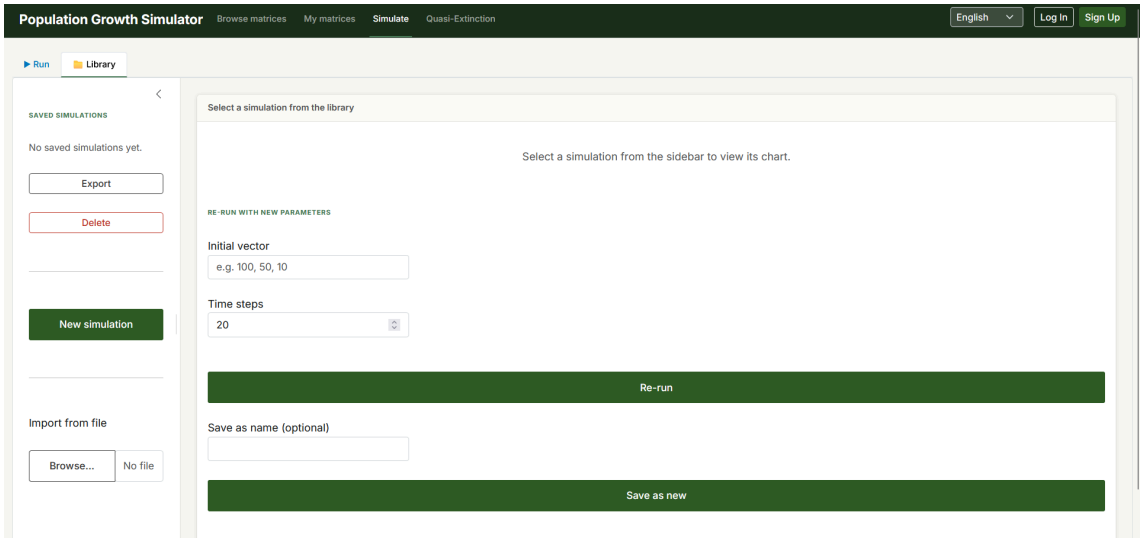

Figure 26: Simulate tab - unauthenticated state showing Run / Library toggle and Import from file option

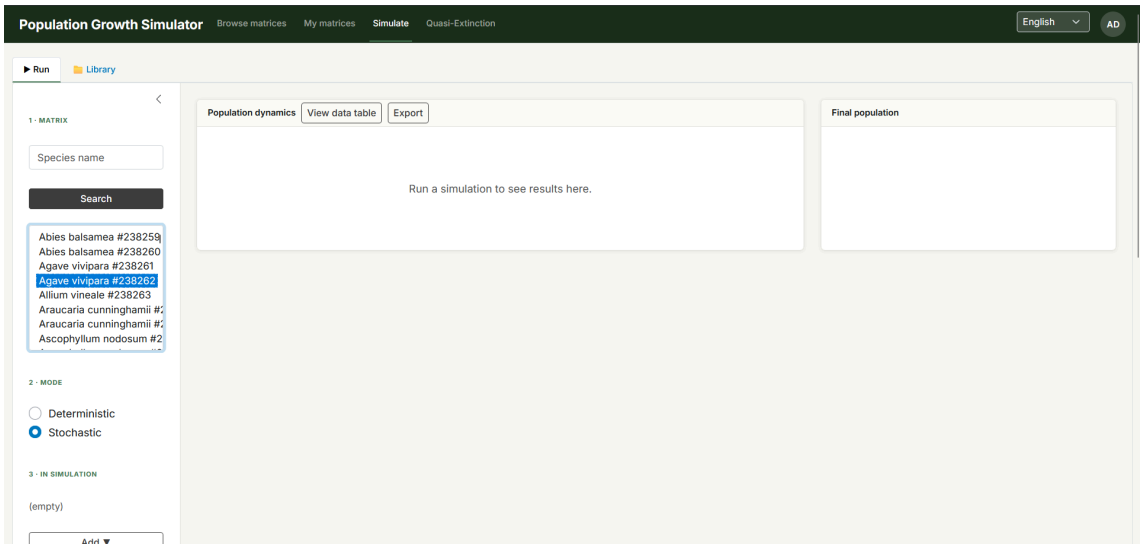

Figure 27: Simulate - four-step sidebar: matrix search (step 1), mode selection (step 2), matrix list (step 3), and parameters including initial vector, time steps, seed, and save controls (step 4) (US-16, US-17, US-18)

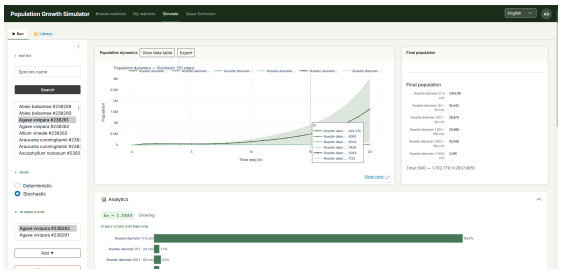

Figure 28: Simulation results - population dynamics chart and final population breakdown by stage (US-20)

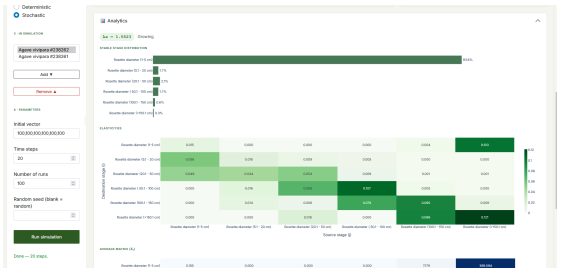

Figure 29: Results - analytics panel: growth rate λ, stable stage distribution, and elasticity matrix (US-20)

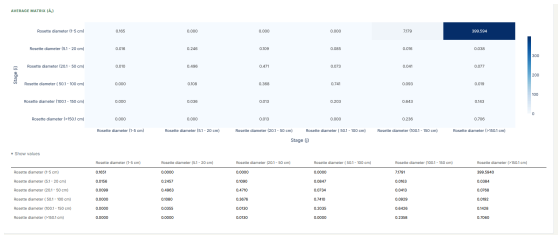

Figure 30: Results - elasticities heatmap and average projection matrix A (US-20)

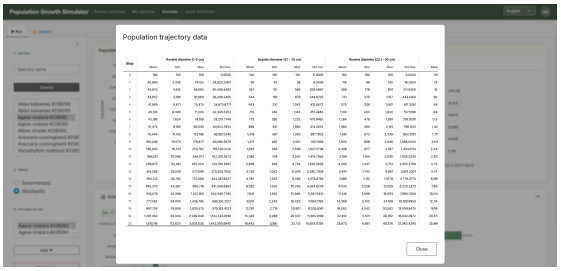

Figure 31: Population trajectory data table - numerical export of per-stage values across all time steps (US-20)

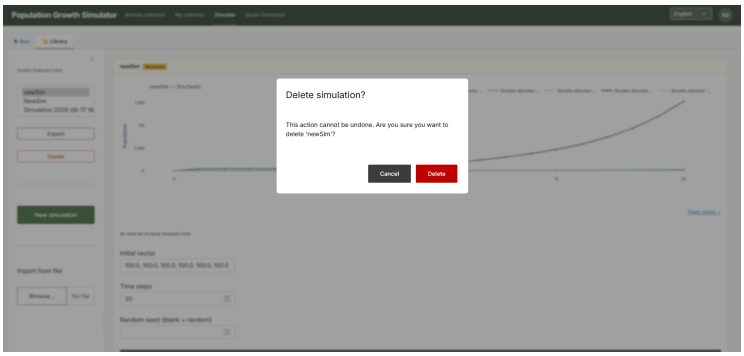

Figure 32: Library view - delete confirmation dialog for a saved simulation run (US-19)

4.1.3.6 My Matrices

The My Matrices tab (Figure 33) provides lifecycle management of custom population matrices and is restricted to authenticated users. Visitors see a login prompt. Once signed in, users can: create a new matrix by specifying dimension, stage names, and cell values for A, U, and F (US-10); edit any matrix they own (US-11); delete matrices with a confirmation step to prevent accidental loss (US-12); set visibility to private, shared, or public (US-13); share a matrix with specific users by entering their username (US-14); and import matrices from a JSON file or a ZIP archive containing multiple JSON files (US-15).

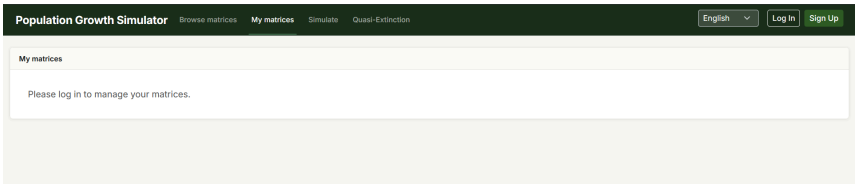

Figure 33: My Matrices tab - unauthenticated state requiring login (US-10 through US-15)

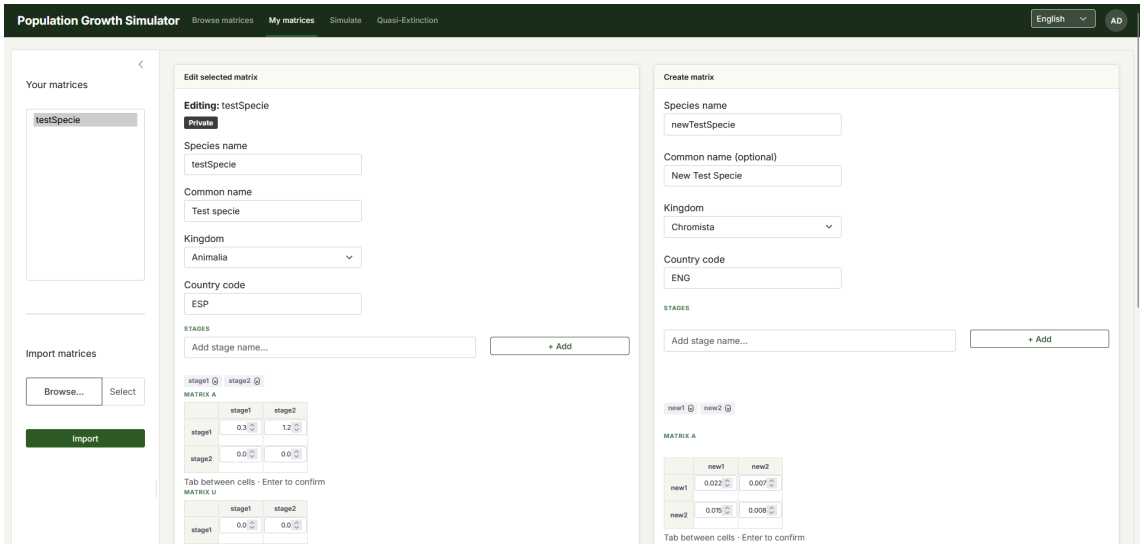

Figure 34: My Matrices tab - authenticated view with matrix list and management controls (US-10 through US-15)

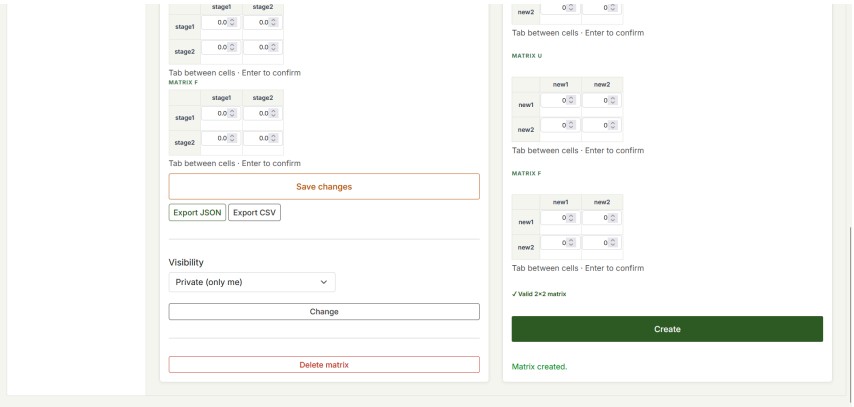

Figure 35: My Matrices - stage-name configuration step during matrix creation or editing (US-10, US-11)

4.1.3.7 Quasi-Extinction Analysis

The Quasi-Extinction tab (Figure 36) provides asynchronous Monte-Carlo probability analysis and requires authentication. The left sidebar lists past analyses and offers a **New analysis** button. The main panel presents the configuration form: matrix selection (same multi-matrix interface as stochastic simulation), initial population vector, extinction threshold, and time horizon. An optional stage-configuration modal (US-22) allows setting per-stage minimum thresholds and excluding specific stages from the extinction check - useful when juvenile stages naturally fluctuate near zero. Once submitted (US-21), the job runs in the background and the user can navigate away; polling updates the status indicator. On completion, the panel renders the mean population trajectory, the cumulative quasi-extinction probability curve, extinction timing and causation charts, a stochastic growth-rate distribution, and a numerical data table. Completed analyses can be deleted from the sidebar via a confirmation dialog.

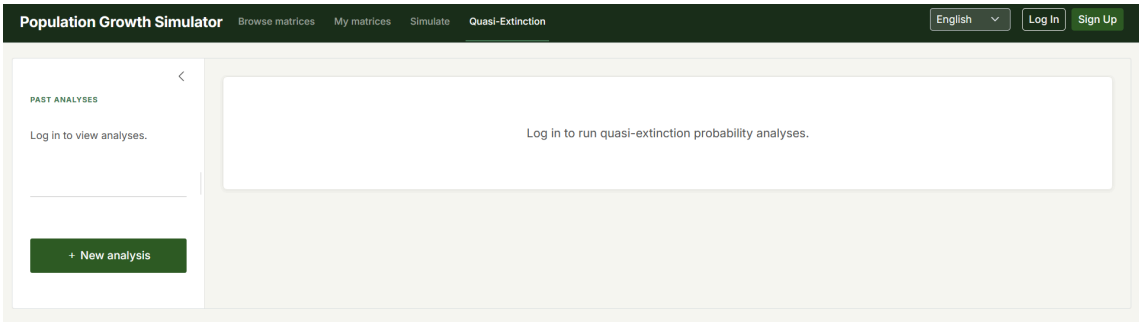

Figure 36: Quasi-Extinction tab - unauthenticated state showing sidebar and main-panel login prompt (US-21, US-22)

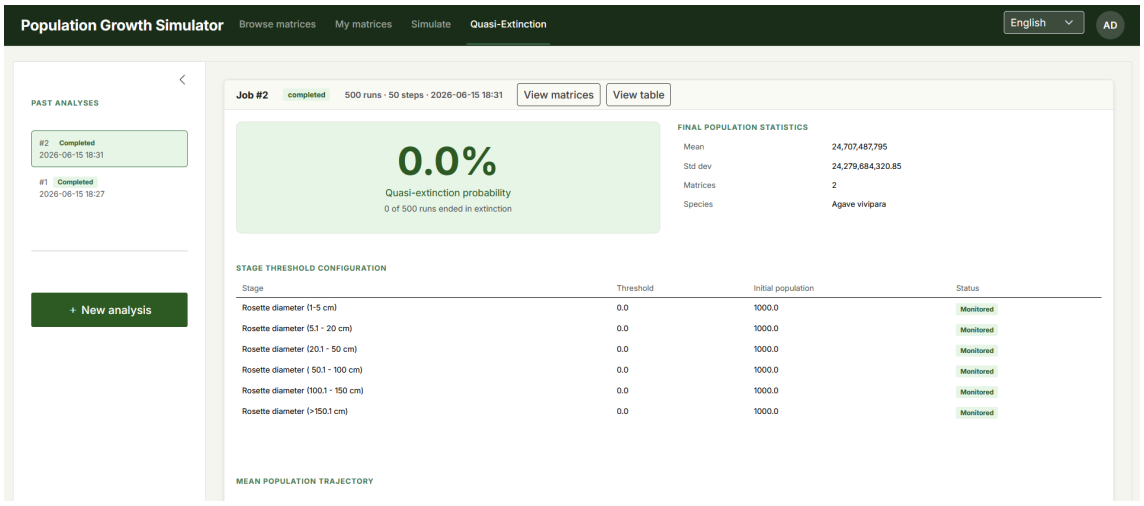

Figure 37: Quasi-Extinction tab - authenticated state: sidebar with past analyses and "New analysis" button (US-21)

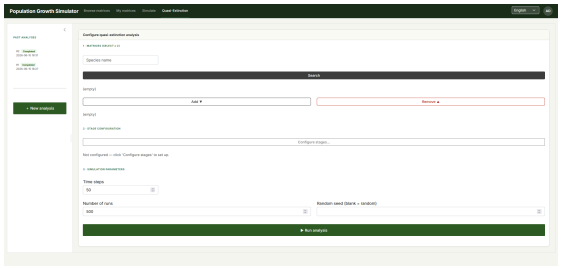

Figure 38: New analysis form - initial configuration (US-21)

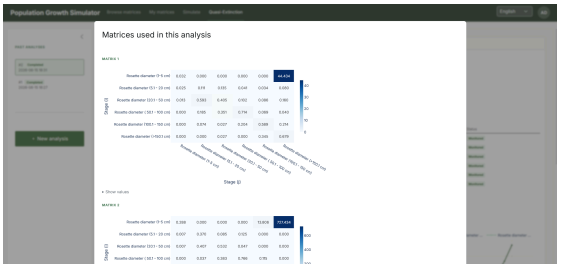

Figure 39: New analysis - matrices selected for the run (US-21)

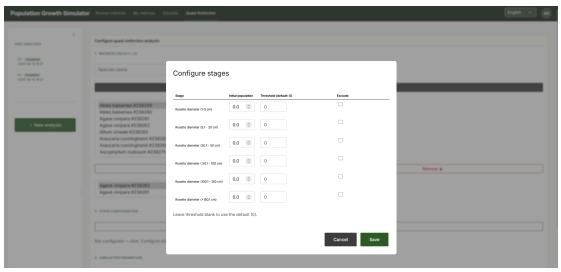

Figure 40: Stage-threshold configuration modal (US-22)

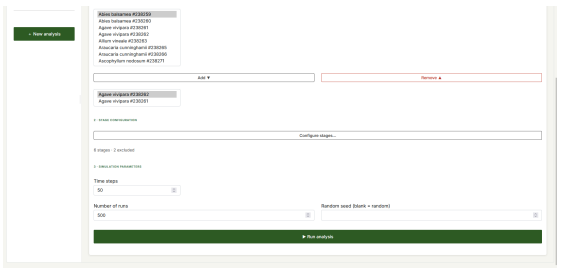

Figure 41: Analysis fully configured - ready to submit (US-21, US-22)

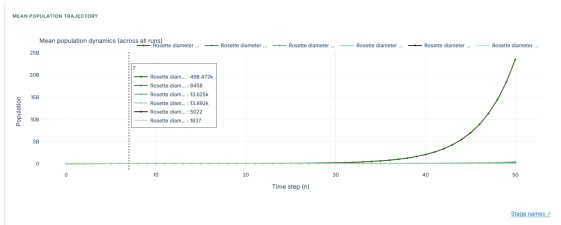

Figure 42: Quasi-extinction job in progress (US-21)

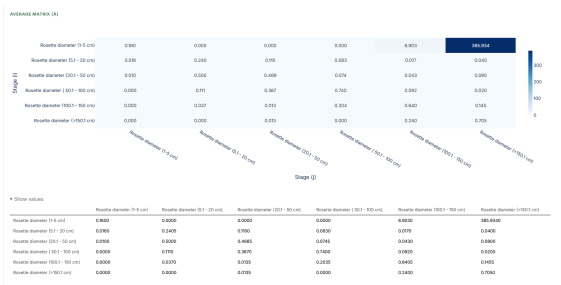

Figure 43: Quasi-extinction results - probability curve over time (US-21)

Figure 44: Mean population trajectory - stage-name legend modal overlaid on the dynamics chart (US-21)

Figure 45: Comprehensive results panel: extinction timing distribution, causation by life stage, and stochastic growth rate distribution (US-21)

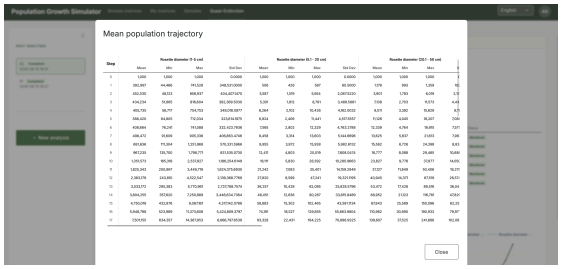

Figure 46: Mean population trajectory data table - numerical values per stage and time step (US-21)

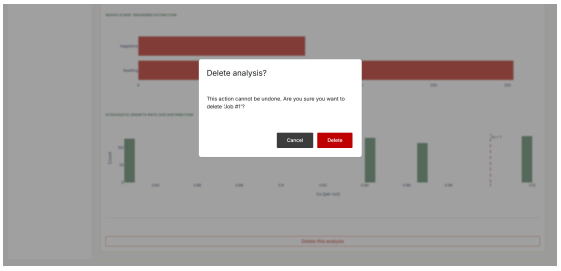

Figure 47: Delete analysis confirmation dialog (US-21)

4.1.3.8 Language Selection

The language selector (Figure 48, US-24) is available in the application header on every tab. Clicking it opens a dropdown with six options. The choice takes effect immediately across all labels, buttons, and messages, and is persisted so that the selected language is restored on the next visit. The six supported locales are: English, Spanish (Español), Asturian (Asturianu), Galician (Galego), Basque (Euskara), and Catalan (Català), reflecting the linguistic diversity of the Spanish academic community that forms a key stakeholder group.

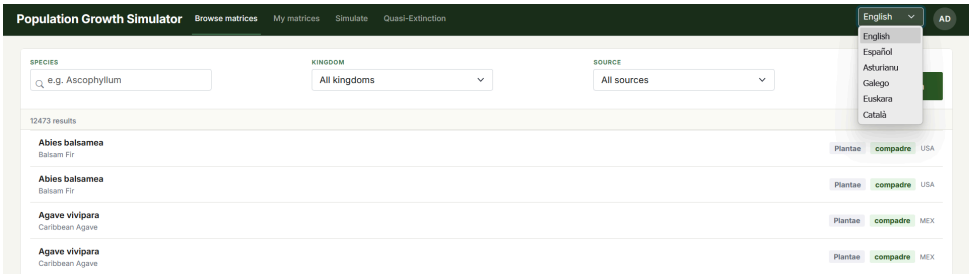

Figure 48: Language selector dropdown - six supported locales available from any tab (US-24)

4.1.4 State Diagrams

In this section, I present some state diagrams that illustrate the behavior of some components of the application.

4.1.4.1 User Session

Figure 49: State Diagram - User Session

4.1.4.2 Simulation Job

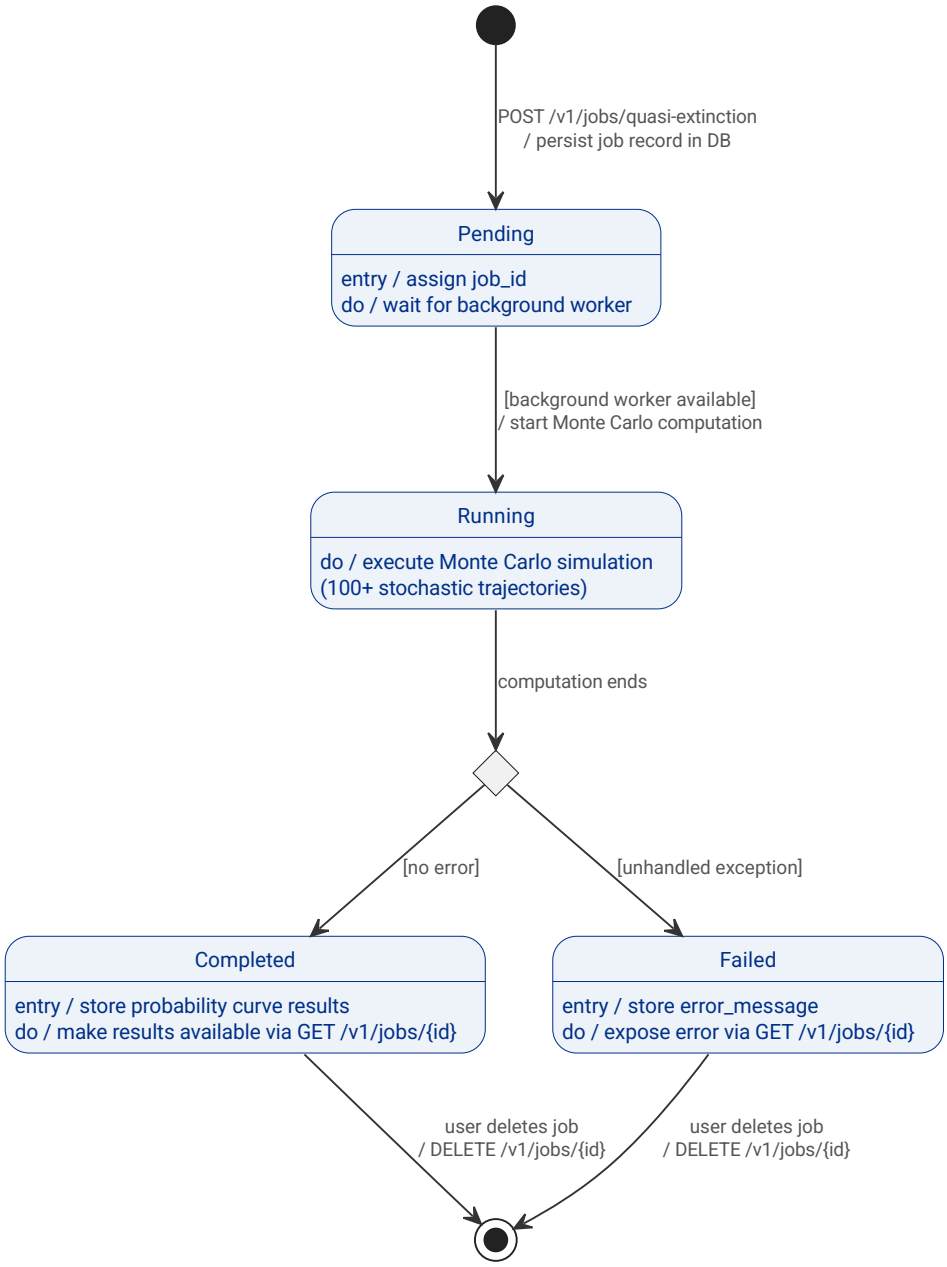

Figure 50: State Diagram - Simulation Job (quasi-extinction async task)

4.1.4.3 Population Matrix

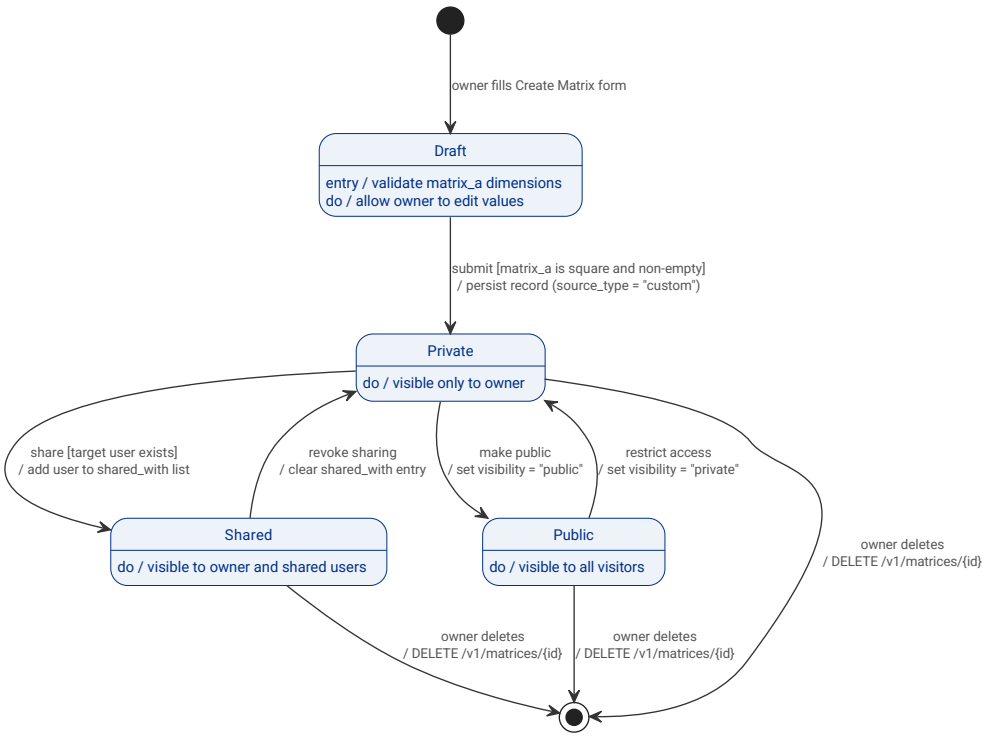

Figure 51: State Diagram - Population Matrix visibility and lifecycle

4.2 Non-Functional Requirements

4.2.1 Security

The system implements a custom JWT-based authentication mechanism rather than delegating to a third-party identity provider (e.g., Google or Microsoft). This was a deliberate academic trade-off: third-party OAuth reduces the attack surface in production deployments, but custom JWT authentication was chosen to demonstrate the authentication mechanics as part of the learning objectives of this project. The implementation follows established security practices to mitigate the associated risks.

NFR-SEC-01. Bcrypt Password Hashing: Passwords shall be stored as bcrypt hashes with auto-generated salts; plain-text passwords shall never be persisted. *Verification: code review (`api/services/auth_service.py`); Bandit SAST scan.*

NFR-SEC-02. JWT Token Expiry: JWT tokens shall use the HS256 algorithm and expire after 7 days. *Verification: code review (`api/deps.py`); integration test.*

NFR-SEC-03. Externalised Secrets: The JWT signing secret and database credentials shall be supplied via environment variables (`.env` file); no secrets shall be hardcoded in source code. *Verification: code review; Bandit SAST scan (no hardcoded secrets).*

NFR-SEC-04. Input Validation: All API inputs shall be validated by Pydantic schemas before processing; invalid input shall return HTTP 422 with a descriptive error body. *Verification: `api/schemas.py`; integration tests.*

NFR-SEC-05. COMPADRE Immutability: Matrices with `source_type = "compadre"` shall be permanently read-only; no authenticated user may edit or delete them. *Verification: service-layer enforcement in `api/services/matrix_service.py`; integration tests.*

NFR-SEC-06. Resource Ownership: Custom matrices and simulation runs shall be accessible only to the user who created them. *Verification: ownership checks in `matrix_service.py` and `simulation_service.py`; integration tests.*

NFR-SEC-07. SAST Scan: The codebase shall pass a Bandit SAST scan (medium+ severity threshold) on every push to `main`. *Verification: GitHub Actions `security.yml`.*

NFR-SEC-08. Dependency CVE Scan: All Python dependencies shall pass a pip-audit CVE scan on every push to `main`. *Verification: GitHub Actions `security.yml`.*

4.2.2 Performance

The application targets interactive response times for a single-user to small-group academic deployment. Computationally intensive operations (quasi-extinction Monte Carlo analysis) are offloaded to asynchronous background jobs to keep the API responsive.

NFR-PER-01. List Endpoint Latency: List and filter endpoints (`GET /v1/matrices`, `GET /v1/simulations`) shall return within 500 ms under normal load. *Verification: manual testing; integration tests.*

NFR-PER-02. Deterministic Simulation Latency: A deterministic simulation with n_steps ≤ 1000 and matrix dimension ≤ 10 shall complete within 200 ms. *Verification: integration tests.*

NFR-PER-03. Stochastic Simulation Latency: A stochastic simulation with n_steps ≤ 1000 and 2-5 matrices shall complete within 500 ms. *Verification: integration tests.*

NFR-PER-04. COMPADRE Catalogue Filtering: The COMPADRE catalogue (6 000 matrices) shall support filtering by species, kingdom, country, and source type with acceptable latency. *Verification: database indexes on filtered columns; integration tests.*

NFR-PER-05. Simulation Step Cap: `n_steps` shall be capped at 1 000 to bound worst-case memory and CPU usage per request. *Verification: Pydantic validator in `api/schemas.py`.*

4.2.3 Availability and Reliability

The system targets continuous availability throughout the evaluation period. Reliability is enforced through automatic service restart, database persistence across container restarts, idempotent migrations, and simulation result snapshots that are immune to future data changes.

NFR-AVL-01. Uptime Target: The system shall provide ≥ 99.5% uptime during the evaluation period. *Verification: Docker restart policy; health check monitoring.*

NFR-AVL-02. Health Endpoint: `GET /health` shall return HTTP 200 when all services are operational. *Verification: integration test; Docker health check.*

NFR-AVL-03. Data Persistence: Database state shall persist across container restarts via a named Docker volume. *Verification: Docker Compose volume configuration; manual test.*

NFR-AVL-04. Idempotent Migrations: Database schema migrations shall run automatically and idempotently on container startup via Alembic `upgrade head`. *Verification: `entrypoint.sh`; CI pipeline.*

NFR-AVL-05. Simulation Snapshots: Stored simulation results shall include a snapshot of matrix values at run time, unaffected by subsequent matrix edits or deletions. *Verification: `matrices_snapshot` JSONB column in `SimulationRun`; unit tests.*

4.2.4 Usability

The application targets biology researchers, students, and educators with no assumption of technical expertise beyond basic web browsing. No client-side installation is required beyond a modern web browser.

NFR-USA-01. Browser-Based Access: The application shall be accessible via a modern web browser with no client-side installation required. *Verification: end-to-end tests (Playwright); user manual.*

NFR-USA-02. Cross-Browser Compatibility: The interface shall function correctly on Chrome ≥ 120, Firefox ≥ 121, and Edge ≥ 120. *Verification: manual cross-browser testing.*

NFR-USA-03. First-Use Task Completion: A first-time user shall be able to run a simulation within 5 minutes following the user manual. *Verification: usability evaluation.*

NFR-USA-04. Password Field Copy-Paste: Password fields shall not disable copy-paste, to allow the use of password managers. *Verification: manual test.*

4.2.5 Maintainability and Testability

The backend enforces a strict three-tier architecture to keep HTTP handling, business logic, and data access independently testable. Services are the sole locus of business rules and can be exercised with mocked repositories without a live database.

NFR-MNT-01. Three-Tier Architecture: The backend shall follow a strict controller → service → repository layering; no layer may bypass another. *Verification: code review; architecture documentation (Chapter 5).*

NFR-MNT-02. Isolated Unit Tests: Services shall be unit-testable in isolation using mocked repositories, without requiring a live database. *Verification: `tests/unit/` test suite; CI pipeline.*

NFR-MNT-03. Test Coverage: API layer test coverage shall reach ≥ 80%. *Verification: Codecov report; CI badge.*

NFR-MNT-04. Versioned Migrations: Every database schema change shall be implemented as a versioned Alembic migration in `alembic/versions/`. *Verification: code review.*

4.2.6 Deployability and DevOps

The entire stack is containerised and deployable with a single command. All configuration is externalised via environment variables. Automated pipelines enforce correctness and security on every push to the main branch.

NFR-DEV-01. Single-Command Deployment: The full system (API, frontend, database) shall deploy with a single `docker compose up` command. *Verification: user manual; manual test.*

NFR-DEV-02. Environment-Based Configuration: All configurable parameters (database credentials, JWT secret, ports) shall be supplied via a `.env` file; no values shall be hardcoded in source. *Verification: code review; Bandit SAST scan.*

NFR-DEV-03. Correctness Pipeline: A CI pipeline shall run unit and integration tests automatically on every push or pull request to `main`. *Verification: GitHub Actions `ci.yml`.*

NFR-DEV-04. Security Pipeline: A security pipeline shall run SAST (Bandit), SCA (pip-audit), and container scan (Trivy) on every push to `main` and on a weekly schedule. *Verification: GitHub Actions `security.yml`.*

NFR-DEV-05. Automated Dependency Updates: Dependency updates shall be managed automatically via Dependabot for pip, GitHub Actions, and Docker base images. *Verification: `.github/dependabot.yml`.*

4.3 Test Plan

The testing strategy is structured as a three-tier pyramid: unit tests at the base, integration tests in the middle, and end-to-end tests at the top. Each tier targets a different test object, operates at a different level of abstraction, and is fully automated. The strict three-layer backend architecture (controller → service → repository) was a deliberate design decision that enables each tier to be exercised in isolation, satisfying NFR-MNT-02.

4.3.1 Unit Tests

Scope: Service layer and Pydantic input schemas - the only locations where business rules, validation logic, and domain algorithms reside.

Tool: pytest with `unittest.mock`. Repository dependencies are replaced with mock objects, so no database or network access is required.

Rationale: Because services contain all business logic, testing them in isolation provides the fastest and most targeted feedback. This tier is particularly valuable for the numerical modules (simulation algorithms, analytics computation, quasi-extinction Monte Carlo) where a logic error would propagate silently through the HTTP layer and produce wrong results without triggering an obvious failure.

4.3.2 Integration Tests

Scope: REST API endpoints - each endpoint is exercised from the HTTP request through the service and repository layers to a real PostgreSQL database.

Tool: pytest with FastAPI's `TestClient` and a dedicated test database instance. Tables are truncated between tests to guarantee isolation.

Rationale: Unit tests with mocked repositories cannot reveal ORM mapping errors, broken SQL queries, wrong HTTP status codes, or missing authentication and authorisation guards. Integration tests verify that all three layers compose correctly and that the system behaves as specified when real data is persisted and read back. This tier also verifies that state-dependent flows - visibility transitions, ownership rules, async job lifecycle - produce the correct observable outcomes across requests.

4.3.3 End-to-End Tests

Scope: The browser-facing user interface, covering the complete flow from user interaction to rendered result.

Tool: pytest with Playwright driving the Python Shiny frontend.

The test suite supports two execution modes. In **mock mode** (used in CI), a lightweight mock server stands in for the real API; no database is required and the tests run reliably in any environment. In **real mode**, Playwright drives the full running stack, providing final validation before deployment.

Rationale: Integration tests exercise the API contract but cannot detect UI rendering problems, broken navigation, or frontend reactive binding failures. End-to-end tests validate the acceptance criteria of user stories as an actual user would experience them, and are the only tier that covers the gap between a correct API and a correctly functioning interface.

4.3.4 Test Coverage and Automation

All three tiers are executed automatically in the CI pipeline on every push or pull request to the main branch (NFR-MNT-03, NFR-DEV-03). Unit tests run first because they require no infrastructure. Integration tests follow using a PostgreSQL service container. End-to-end tests run in mock mode for reliability.

The following table summarises the mapping between testing tiers, test objects, and the requirements they verify:

Tier	Test object	Requirements verified
Unit	Service layer, schemas	NFR-MNT-02, NFR-SEC-04
Integration	REST API endpoints, DB layer	Functional requirements; NFR-SEC-04, NFR-SEC-05, NFR-SEC-06, NFR-PER-01, NFR-PER-02, NFR-PER-03, NFR-AVL-02
End-to-end	Browser UI, user flows	US-01 - US-22, NFR-USA-01

Table 59: Test tier mapping

4.3.5 Out of Scope

Load testing and formal usability testing fall outside the automated test suite. Performance non-functional requirements (NFR-PER-01, NFR-PER-02, NFR-PER-03) are verified through integration test timings and targeted manual testing. The usability requirement NFR-USA-03 (first-use task completion within five minutes) is validated through a structured evaluation session with representative users; the results are presented in the evaluation chapter.

Chapter 5

DESIGN

This chapter documents the design decisions and structures that shape the Population Growth Simulator. It covers the system architecture, internal component design, development workflow, CI/CD pipeline, observability approach, and test design. The aim is to explain not only *what* was built but *why* each structural choice was made.

5.1 ARCHITECTURAL DESIGN

The system follows a classic three-tier architecture: a presentation layer, an application layer, and a data layer. Each tier is deployed as an independent Docker container and communicates only through well-defined interfaces, allowing each tier to be scaled, replaced, or tested in isolation.

5.1.1 HIGH-LEVEL OVERVIEW

Figure 52 shows the system boundary and its two external actors. The *User* interacts with the system through a web browser, either browsing public matrix data or running and saving simulations after authentication. The *COMPADRE Plant Matrix Database* [7] is an external data source whose matrix data is seeded into the system at startup; it has no runtime relationship with the running application.

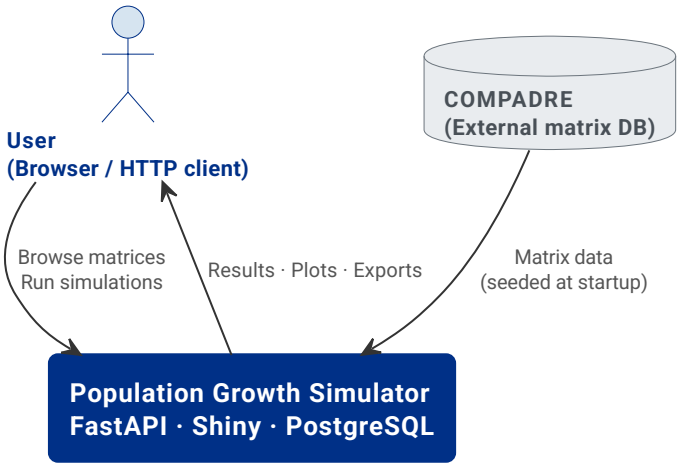

Figure 52: System context diagram. The Population Growth Simulator sits between the user's browser and the COMPADRE data source.

Figure 53 shows the three tiers and, within the application tier, the internal layering of the API service. The presentation tier (Shiny frontend, port 8080) communicates with

the application tier exclusively through HTTP REST calls. The application tier (FastAPI, port 8000) is itself structured into controllers, services, and repositories. The data tier (PostgreSQL, port 5432 inside Docker) is accessed only by the repository layer.

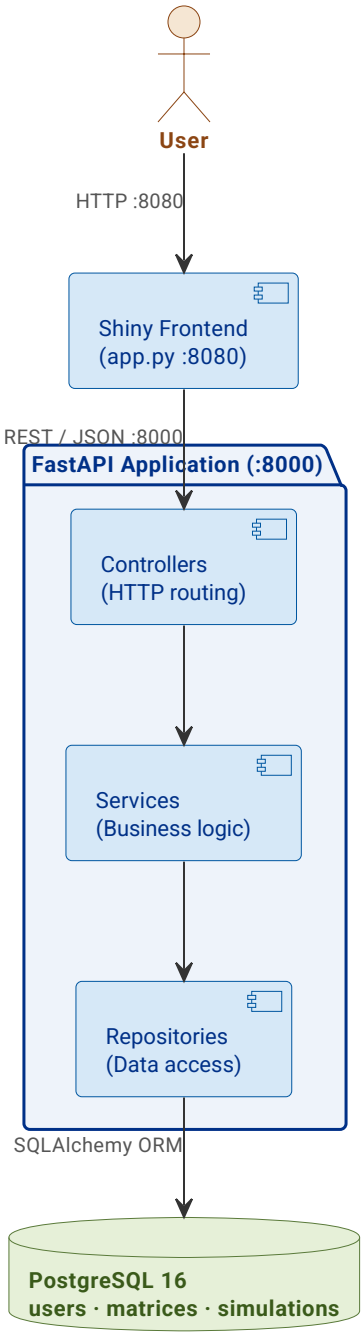

Figure 53: Building-blocks view. The three tiers and the internal controller → service → repository layering of the API.

5.1.2 Service Architecture

Within the API tier, a second layer of separation is enforced: no layer may skip another, and each has a single, well-defined responsibility.

Controllers (`api/controllers/`) handle all HTTP concerns. They parse incoming request bodies into Pydantic schema objects, delegate to the appropriate service method, and

return the resulting record with the correct HTTP status code. Controllers contain no business logic and perform no database access.

Services (`api/services/`) contain all business logic: ownership checks, immutability rules for COMPADRE matrices, cross-matrix dimension validation, and the simulation algorithm itself. Services receive Pydantic schemas as input and return domain records as output. All persistence is delegated to repositories.

Repositories (`api/repositories/`) are the only layer permitted to interact with SQLAlchemy sessions. They expose a narrow, entity-centric interface (`get_by_id`, `create`, `update`, `list`) and return raw ORM objects to the service layer. No business rule is encoded here.

Schemas and Records (`api/schemas.py`, `api/records.py`) form two distinct families of Pydantic models that cross the API boundary. Schemas (input DTOs) validate data arriving over HTTP - all field constraints and cross-field invariants live exclusively in schemas. Records (output models) represent business entities as they flow out of the service layer into HTTP responses, constructed from ORM objects via `model_validate` with `from_attributes=True`. This separation prevents accidental exposure of internal fields and makes the contract of each layer explicit.

Dependency injection (`api/deps.py`) wires the full request dependency tree using FastAPI's `Depends` mechanism: a SQLAlchemy `Session` scoped to each request, pre-built service instances receiving their repository dependencies, and the `get_current_user` guard that validates JWT tokens and resolves the authenticated user before any controller logic runs.

5.1.3 Deployment and Infrastructure

Figure 54 shows the Docker Compose deployment. The three containers share a Docker-managed bridge network; only the host-side ports listed in Table 60 are exposed externally.

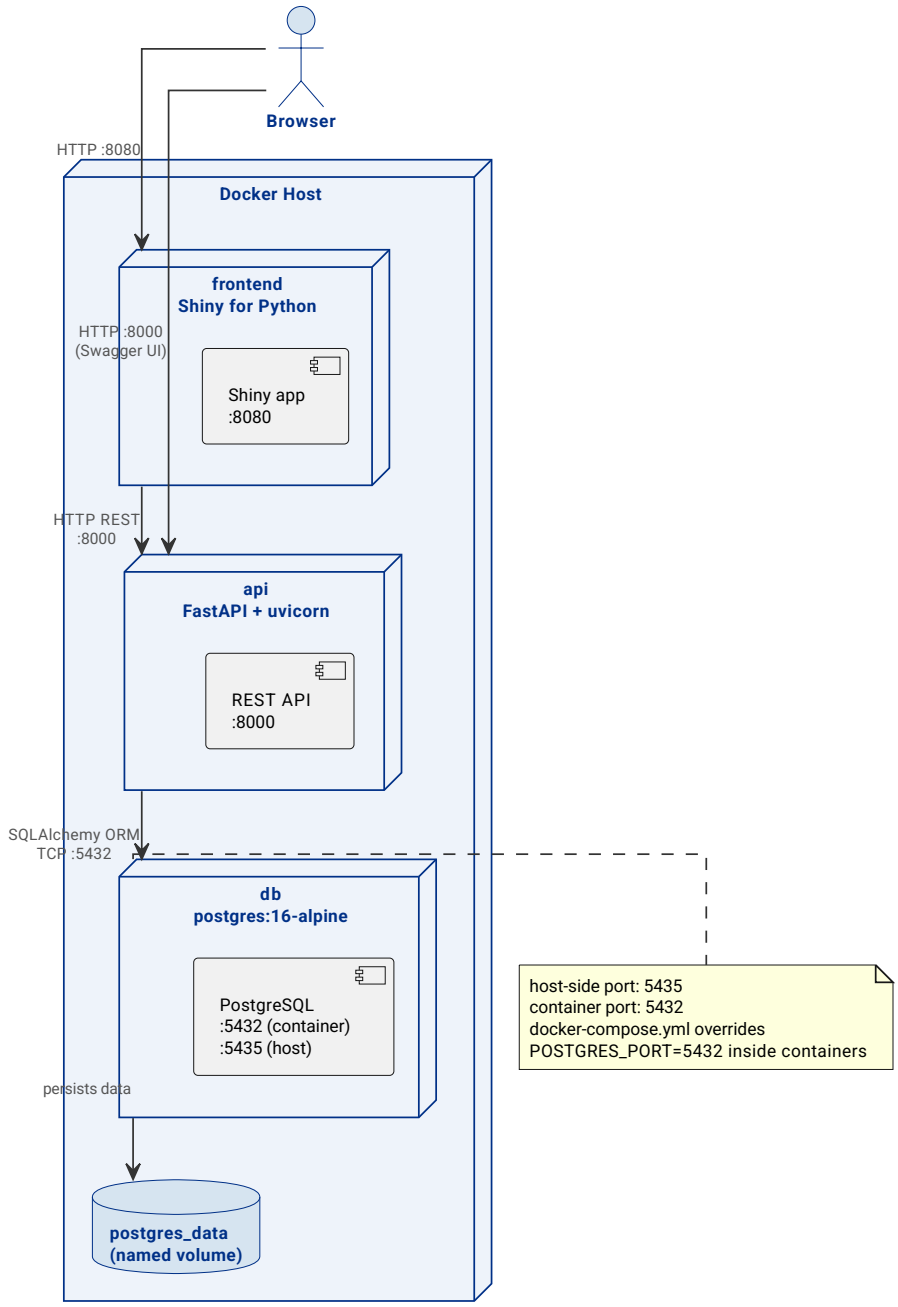

Figure 54: Deployment diagram. Three Docker containers communicate over an internal bridge network; only host-side ports are externally accessible.

Service	Image	Exposed port (host)
`db`	`postgres:16-alpine`	`5435 → 5432 (container)`
`api`	built from `Dockerfile`	`8000`
`frontend`	built from `frontend/Dockerfile`	`8080`

Table 60: Docker Compose services and exposed ports.

The `db` container uses a named volume (`postgres_data`) to persist data across container restarts. The `api` and `frontend` services declare a `depends_on` condition tied to the `db`

health check, so they only start after PostgreSQL is ready to accept connections. On startup, the `api` container runs `entrypoint.sh`, which performs three idempotent steps before launching Uvicorn: apply pending Alembic migrations (`alembic upgrade head`), seed COMPADRE data (`python db/seed_compadre.py`), and start the server. Restarting the container does not duplicate data or re-apply already-applied migrations.

A deliberate port override exists: `.env` sets `POSTGRES_PORT=5435` (the host-side port), while `docker-compose.yml` overrides `POSTGRES_PORT=5432` for the `api` and `frontend` services so that they connect to the correct container-internal port without requiring a separate environment file for Docker.

5.2 Detailed Design

5.2.1 Main System Flows

The following eight sequence diagrams document all major user journeys through the system. Each diagram captures the actors involved, the message exchange between tiers, and the key business rules enforced at each step.

User registration (Figure 55). The service layer checks that neither the chosen username nor the email is already taken before hashing the password with bcrypt and persisting the new user record.

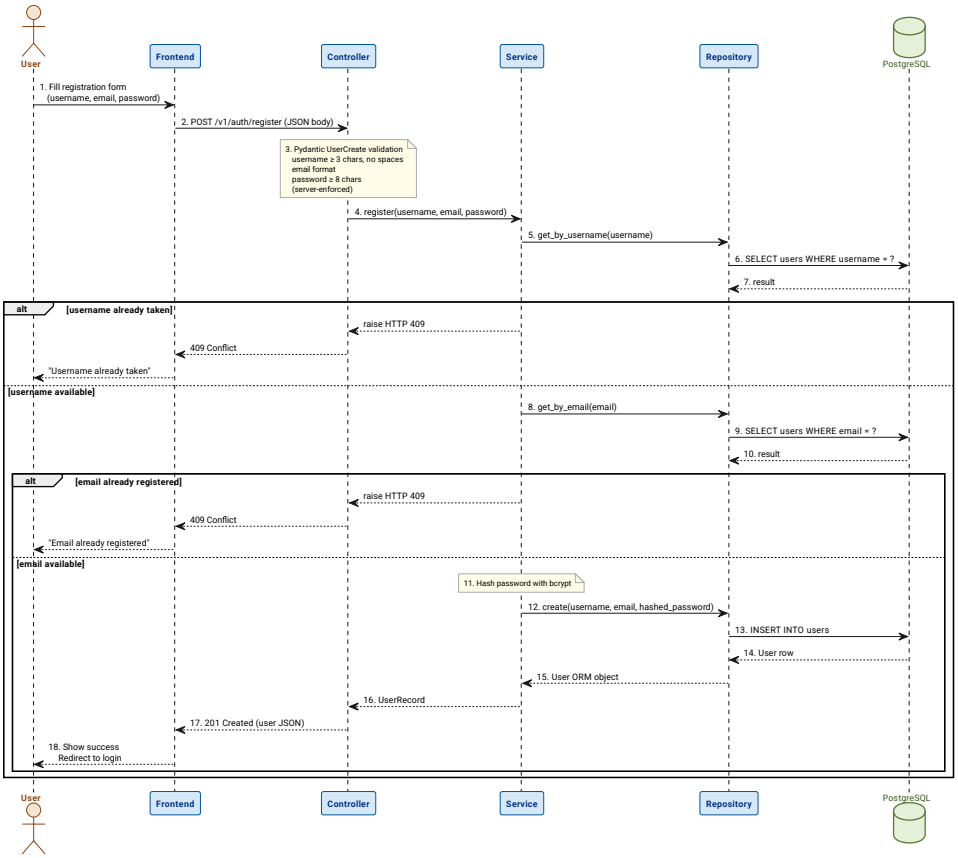

Figure 55: Sequence diagram: user registration. Uniqueness is checked before the password is hashed.

User login (Figure 56). The client submits credentials through the OAuth2 password form. The service verifies the bcrypt digest and, on success, issues a signed JWT (HS256, seven-day expiry) that the client must include as a Bearer token in subsequent authenticated requests.

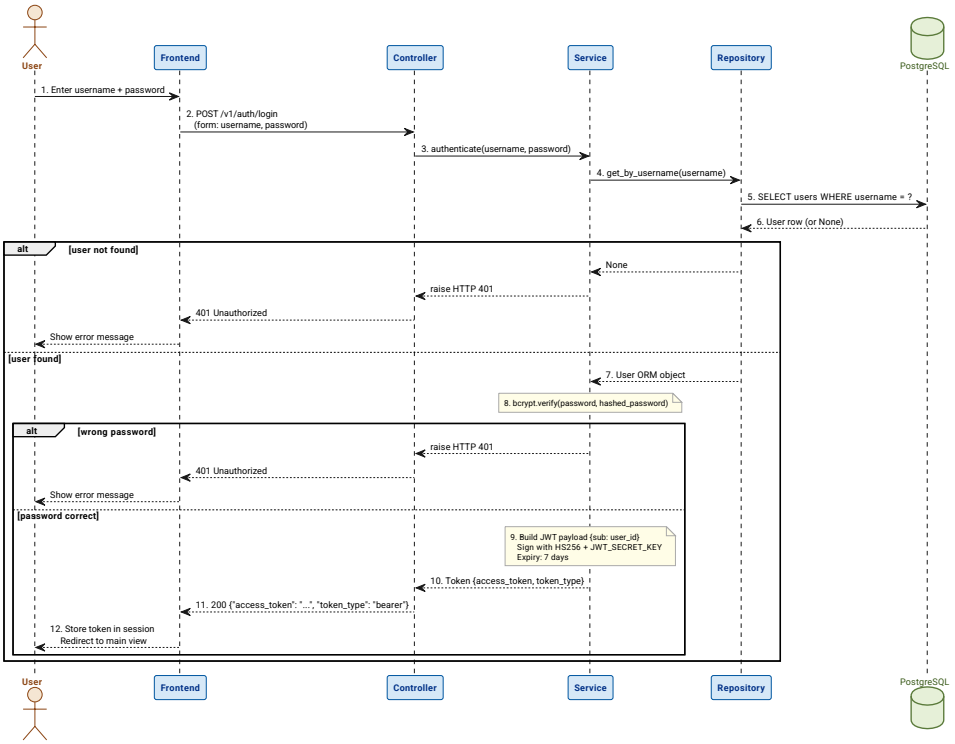

Figure 56: Sequence diagram: user login. Successful authentication produces a signed JWT.

Search and browse matrices (Figure 57). This endpoint is public - no authentication is required. The repository applies optional filters (species, kingdom, source type) and returns summary projections rather than full matrix data to keep responses compact.

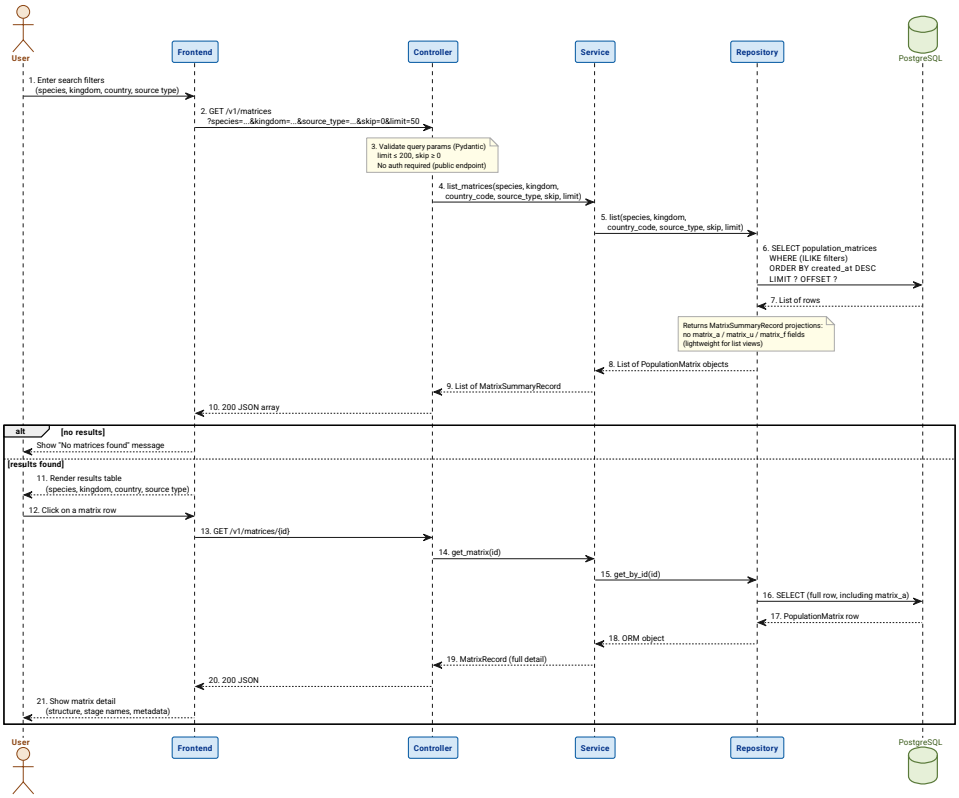

Figure 57: Sequence diagram: browse and filter matrices. The endpoint is public and returns summary projections.

Create a custom matrix (Figure 58). Authentication is required. The controller validates the request body through a Pydantic schema; the service sets the `owner_id` to the authenticated user and persists the matrix with `source_type = "custom"`.

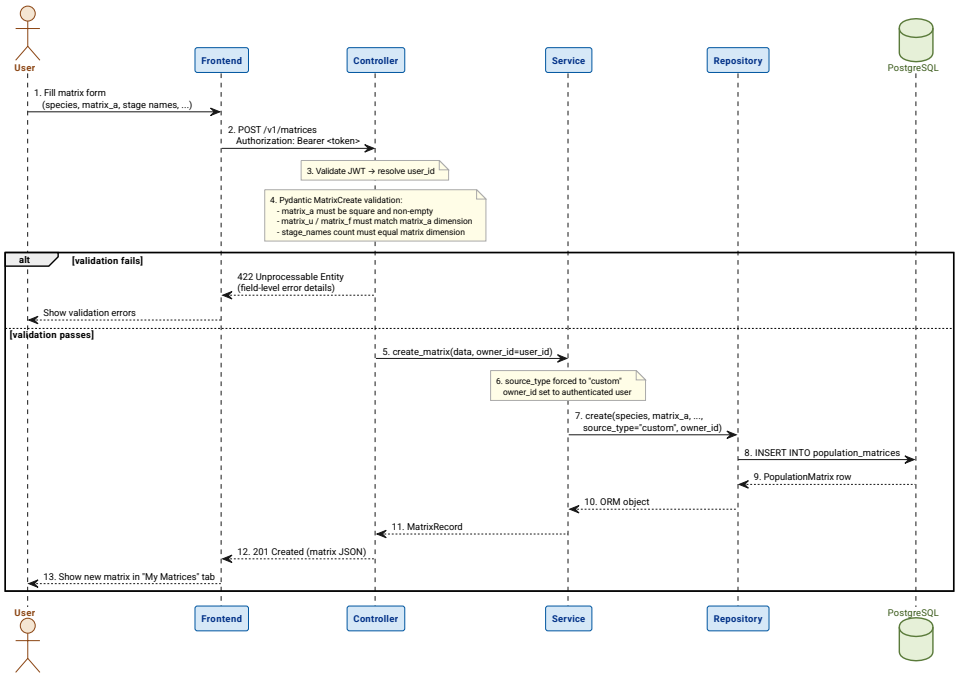

Figure 58: Sequence diagram: create a custom matrix. Ownership is set at creation time.

Save a simulation (Figure 59). The server runs the projection algorithm and stores the complete trajectory (`result_history`) alongside a snapshot of the matrix stage names.

Storing server-side guarantees reproducibility and uniform results across all client devices.

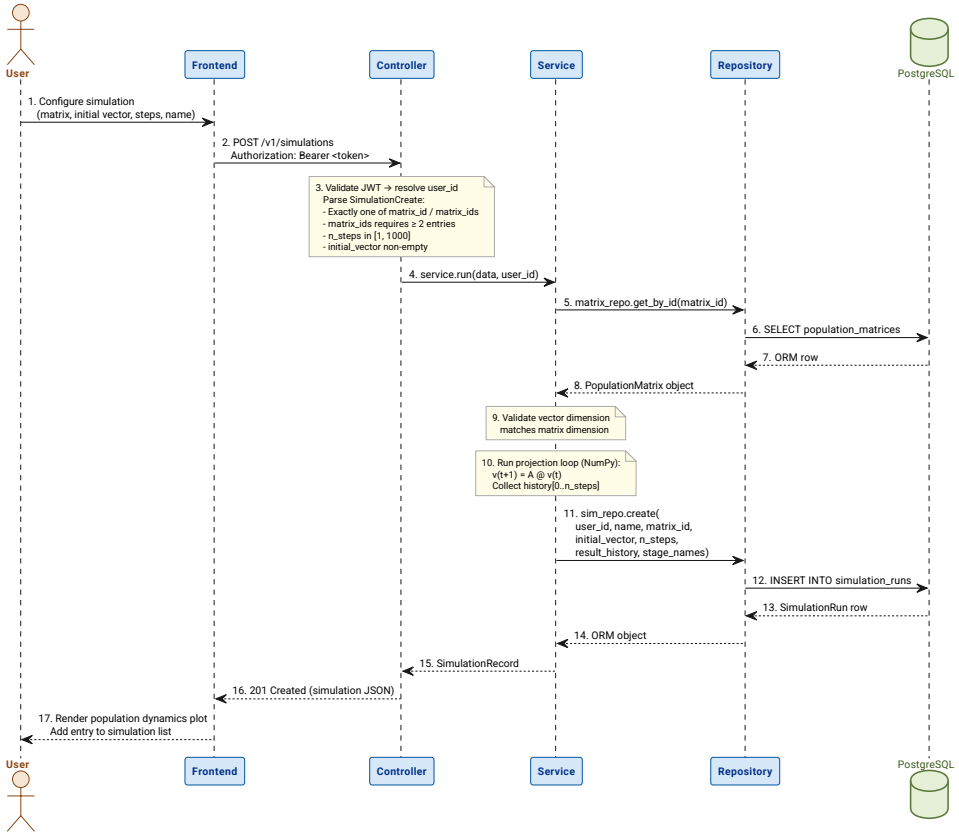

Figure 59: Sequence diagram: save a simulation. The full trajectory is computed and stored server-side.

Open a saved simulation (Figure 60). The service verifies that the requesting user owns the simulation before returning the full history. An unauthorised request receives `HTTP 403`.

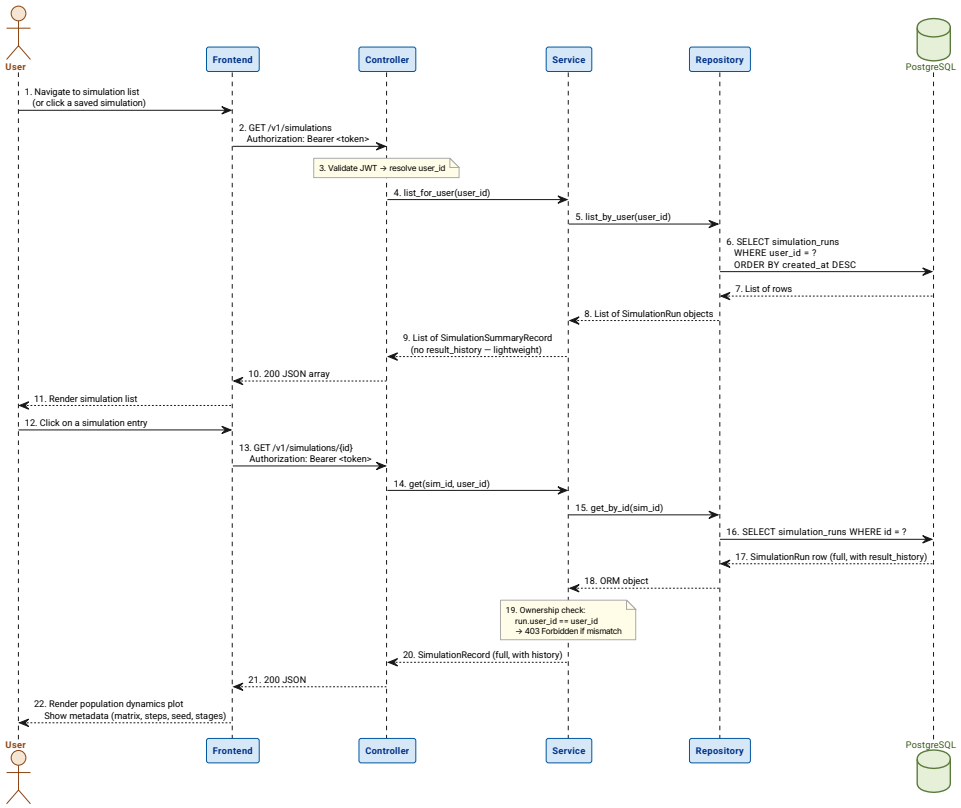

Figure 60: Sequence diagram: open a saved simulation. Ownership is checked before the trajectory is returned.

Export a simulation (Figure 61). The stored trajectory is serialised to a JSON payload and returned to the client. No recomputation occurs.

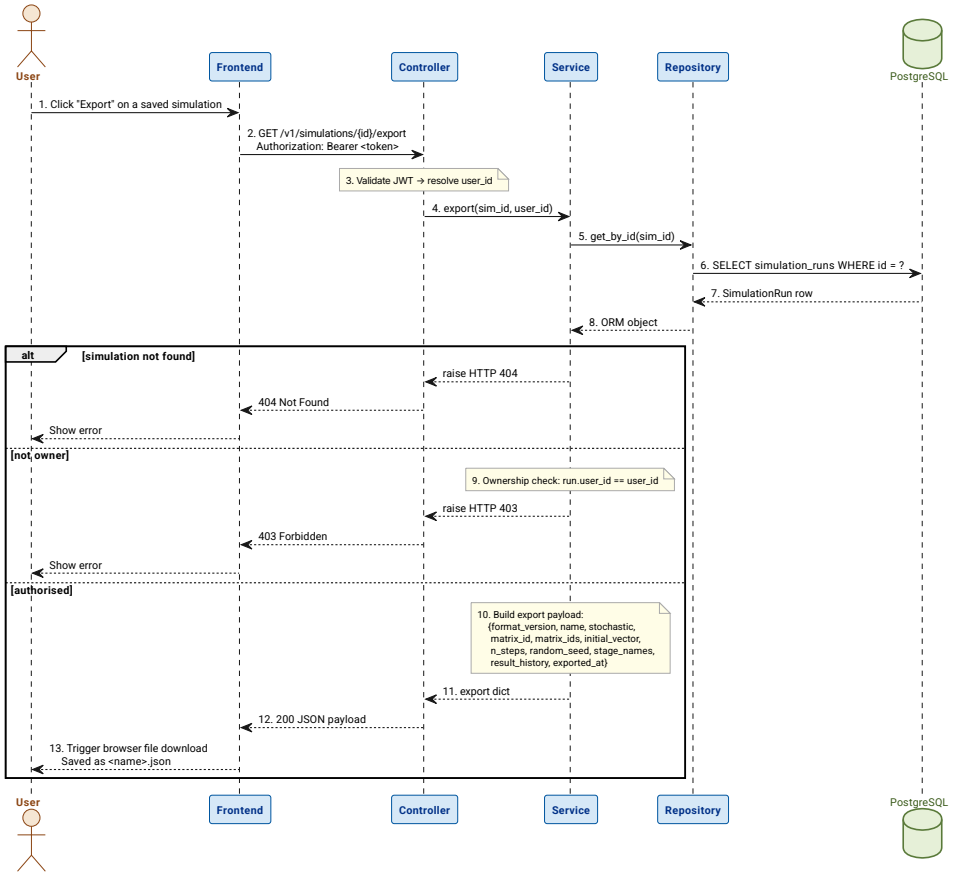

Figure 61: Sequence diagram: export a simulation as JSON. The stored result is serialised without recomputation.

Import a simulation (Figure 62). The client uploads a previously exported JSON file. The service deserialises the payload and re-persists it as a new simulation record owned by the authenticated user, without re-running the algorithm.

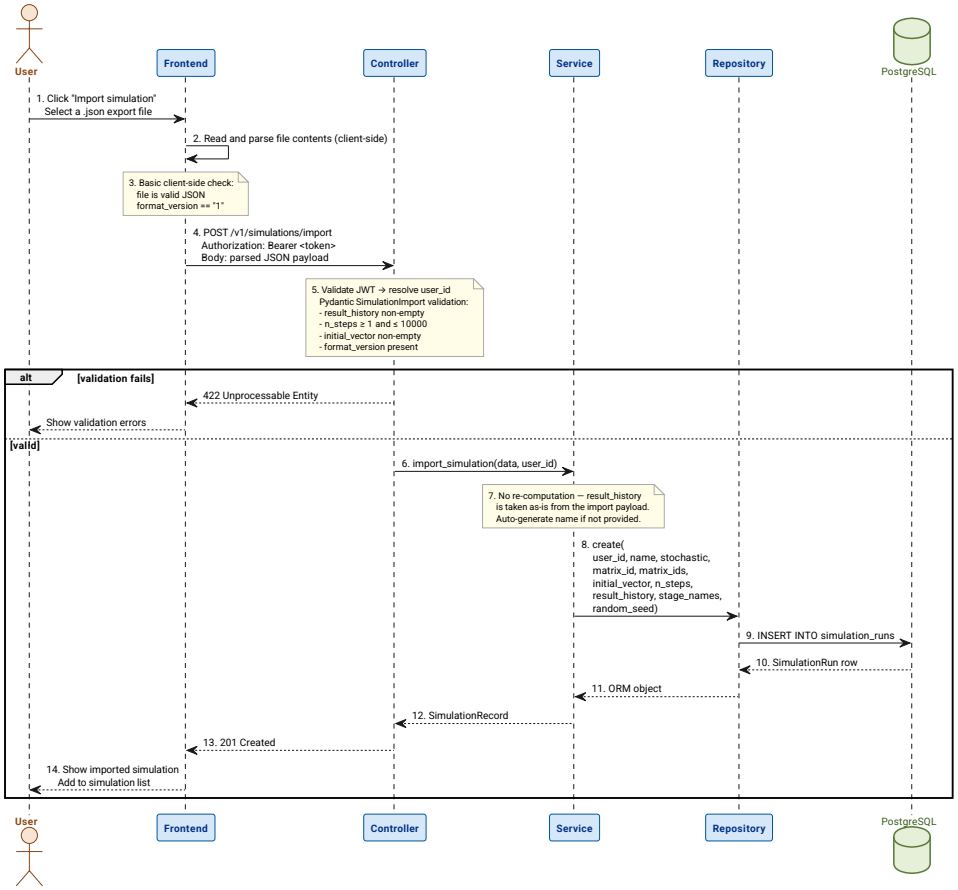

Figure 62: Sequence diagram: import a simulation from JSON. The trajectory is restored from the file without rerunning the algorithm.

5.2.2 Persistent Data Model

The database contains three tables managed by SQLAlchemy ORM models defined in `db/models.py`. Figure 63 shows the entities, their attributes, and the foreign-key relationships between them.

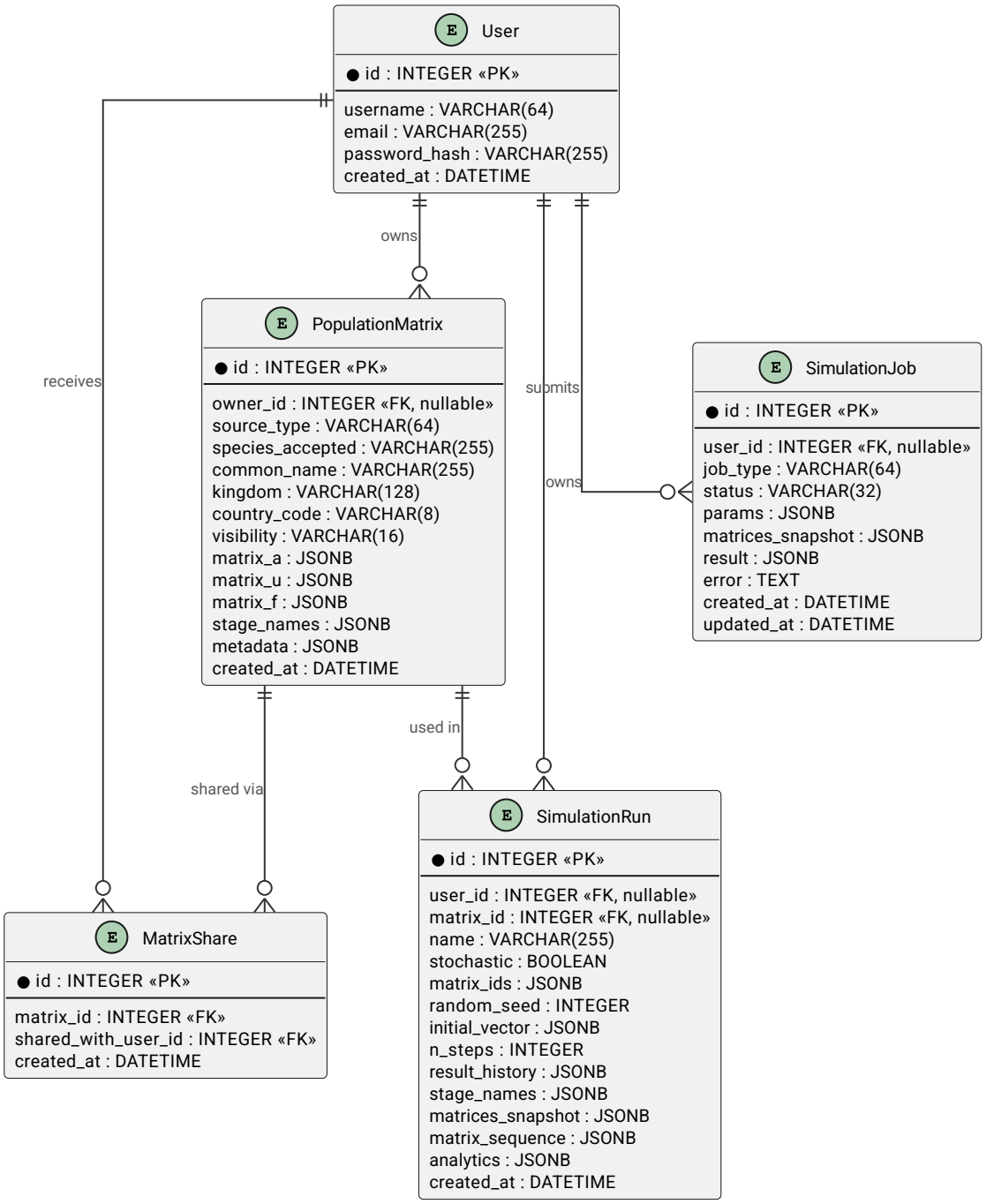

Figure 63: Entity-relationship diagram. Three tables store users, matrices, and simulation trajectories.

`users`. Stores registered user accounts. Passwords are stored as bcrypt hashes; the plaintext password exists only for the duration of the registration or login request and is never persisted. The `username` and `email` columns carry unique constraints enforced at the database level as a second line of defence after the service-layer uniqueness check.

`population_matrices`. The central reference entity. Two kinds of records coexist in this table, distinguished by `source_type`.

COMPADRE matrices (`source_type = "compadre"`) are seeded from the COMPADRE Plant Matrix Database at startup. They have `owner_id = NULL` and are read-only: any attempt to

modify them through the API is rejected with `HTTP 403`. Custom matrices (`source_type = "custom"`) are created by authenticated users and can be updated by their owner.

Matrix data is stored as JSONB columns (`matrix_a`, `matrix_u`, `matrix_f`, `stage_names`, `metadata_`). The ORM column is named `metadata_` (with a trailing underscore) to avoid a collision with SQLAlchemy's internal `metadata` attribute; the API serialises it as `"metadata"` through a Pydantic `validation_alias`.

`simulation_runs`. Records a complete simulation result. The `result_history` JSONB column stores the full list of population vectors from step 0 (the initial vector) to step `n_steps`. For stochastic runs, this is the mean population vector across all N runs at each step, making retrieval self-contained without re-running the algorithm.

For deterministic runs, `matrix_id` is set and `matrix_ids` is null. For stochastic runs, `matrix_ids` is set (as a JSONB array of integers) and `matrix_id` is null. The `random_seed` column enables exact reproduction of stochastic runs. The `stage_names` column is a snapshot of the stage labels at run time, so that historical simulations remain interpretable even if the source matrix is later updated or deleted.

Four additional JSONB columns support the multi-run stochastic model: `n_runs` records how many independent runs were executed; `result_variance`, `result_min_history`, and `result_max_history` store the per-stage ensemble statistics (variance, minimum, and maximum population values) at each time step across all runs. The `matrix_sequence` JSONB column holds one committed matrix index per run - not one per step - so the full run-level matrix attribution is reproducible. All four columns are nullable and set to null for deterministic runs.

Schema evolution is managed by Alembic. The migration chain runs from `0001_initial_schema` (users and population_matrices) through `0007_add_stochastic_stats` (multi-run stochastic columns on simulation_runs). Migrations are applied automatically by `entrypoint.sh` on container startup and are idempotent.

5.2.3 Design Patterns Applied

The following design patterns are applied throughout the codebase. Each is documented with its intent and the concrete classes that fulfil each role.

Repository Pattern. Intent: decouple persistence from business logic, making services independently testable. Roles: abstract interface → implicit Python duck typing; concrete repositories → `MatrixRepository` (`api/repositories/matrix_repository.py`), `SimulationRepository`, and `UserRepository`. The service layer never accesses SQLAlchemy sessions directly; it always goes through a repository. In the test suite, repositories are replaced with `unittest.mock.MagicMock` objects so that service logic can be verified without a database.

Dependency Injection. Intent: provide service and repository instances to controllers without coupling them to instantiation logic. Roles: injector → FastAPI's `Depends` mechanism; provider → `api/deps.py`; consumers → all controller route functions. Each

request receives a fresh DB session scoped to its lifetime; services and repositories are constructed within `deps.py` and injected into the controllers that need them.

DTO / Record Split. Intent: maintain a strict boundary between input validation and output serialisation. Roles: input DTO → Pydantic Schema classes in `api/schemas.py`; output model → Record classes in `api/records.py`. This separation prevents the accidental exposure of internal ORM fields in responses and makes the API surface explicit and auditable.

Immutability Guard. Intent: protect the integrity of the COMPADRE reference dataset regardless of the requesting user's identity. Roles: guard location → `MatrixService.update_matrix` in `api/services/matrix_service.py`; trigger → `source_type == "compadre"`; response → `HTTPException(status_code=403)`. The rule is enforced entirely in the service layer with no reliance on a database constraint, making it fully testable with mocked repositories.

5.3 Development Workflow and Tooling

5.3.1 Repository and Version Control

The project is hosted on GitHub. Figure 64 shows the branching model used throughout development.

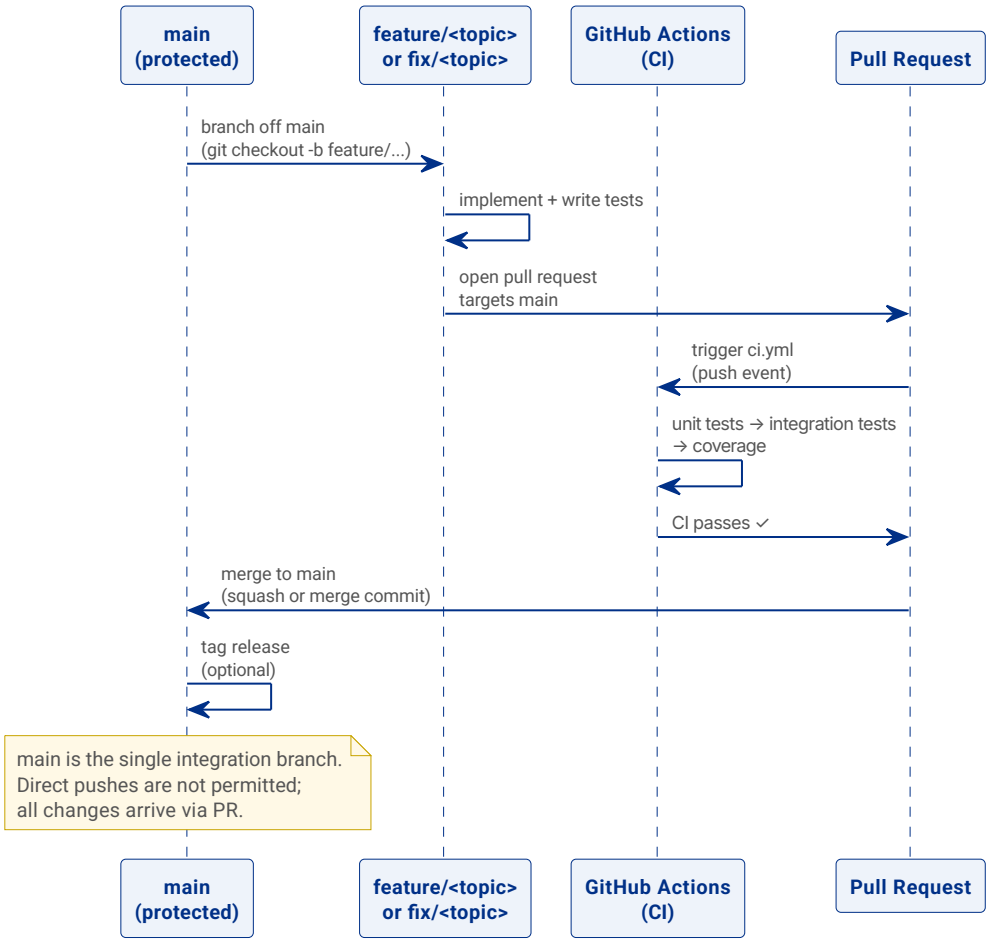

Figure 64: Branch strategy. All changes arrive on `main` through a pull request; direct pushes are not permitted.

The `main` branch is the single integration branch and is kept in a deployable state at all times. All new work is developed on short-lived feature or fix branches named `feature/<topic>` or `fix/<topic>`. A pull request targeting `main` is required before any branch can be merged; the CI workflow (Section 5.4.2) must pass before the merge is allowed. This guarantees that `main` never contains code that fails the test suite.

5.3.2 Team Workflow

This is a single-developer project. The development cycle follows a lightweight agile-inspired loop: identify a task or defect, create a feature branch from `main`, implement the change together with its tests, open a pull request, wait for CI to pass, and merge. Commit messages follow the imperative-mood convention (e.g. `Add stochastic simulation endpoint`, `Fix COMPADRE immutability check`), keeping the git log readable as a chronological record of intent.

5.4 Continuous Integration and Deployment (CI/CD)

5.4.1 Pipeline Overview

Figure 65 maps every trigger event to the workflows it activates. Table 61 lists all four automation files and their purposes. The following subsections detail each workflow's internal job structure (Figure 66, Figure 67, Figure 68, Figure 75, Figure 77).

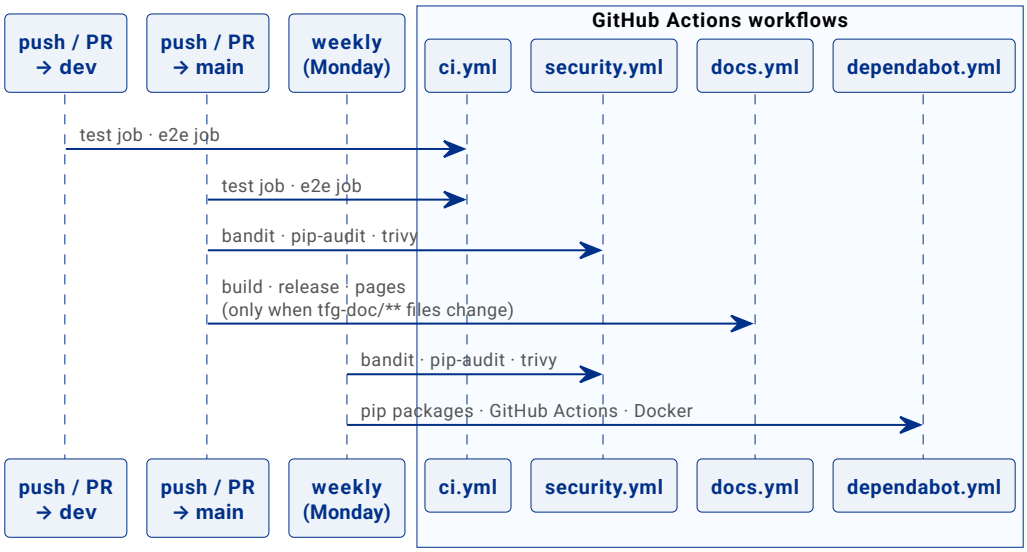

Figure 65: Pipeline trigger map. Each event column shows which workflows and jobs it activates.

File	Trigger	Purpose
`ci.yml`	Push / PR to `main`, `dev`, `master`	Correctness: tests, coverage, E2E
`security.yml`	Push / PR to `main`; weekly (Monday 08:00 UTC)	Security: Bandit SAST, pip-audit CVEs, Trivy container scan
`docs.yml`	Push / PR to `main` (`tfg-doc/**`); manual	Documentation: compile Typst PDF, publish GitHub Release and GitHub Pages
`dependabot.yml`	Weekly (Monday)	Dependency updates: pip packages, GitHub Actions, Docker base image

Table 61: GitHub Actions workflow files and Dependabot configuration.

The workflows are kept separate deliberately. A security job failure (for example, a newly disclosed CVE in a transitive dependency) does not block a feature merge, because the fix may require an upstream release that is outside the developer's immediate control. Conversely, the weekly schedule of the security workflow surfaces vulnerabilities in base images and dependencies that no code commit could have detected.

5.4.2 Continuous Integration

The CI workflow (`ci.yml`) runs on every push and pull request targeting `main`. It provisions a PostgreSQL 16 service container within the CI job so that integration tests can run against a real database without any external infrastructure.

The workflow executes the following steps in order:

1. **Checkout** - `actions/checkout@v4` fetches the repository at the triggering commit.
2. **Setup Python 3.13** - `actions/setup-python@v5` installs Python and enables pip dependency caching.
3. **Install dependencies** - installs `requirements.txt` plus the test packages (`pytest`, `pytest-cov`).
4. **Apply migrations** - `python -m alembic upgrade head` brings the CI database schema to the current version.
5. **Unit tests** - `pytest tests/unit/ -v` runs the fast, database-free unit tests first. Failures here abort the pipeline before slower integration tests are attempted.
6. **Integration tests** - `pytest tests/ --ignore=tests/unit/ -v` runs the full HTTP integration suite against the provisioned PostgreSQL instance.
7. **Generate coverage report** - a combined `--cov=api --cov=db` run produces both a terminal summary and an XML report.
8. **Upload to Codecov** - the XML report is sent to Codecov for trend tracking. `fail_ci_if_error: false` prevents a Codecov service outage from blocking the pipeline.

Table 62 lists the environment variables configured for the CI job. The `JWT_SECRET_KEY` value is a non-production placeholder used only within the isolated CI environment.

Variable	Value in CI
`POSTGRES_USER`	`postgres`
`POSTGRES_PASSWORD`	`postgres`
`POSTGRES_DB`	`matrix_db`
`POSTGRES_HOST`	`localhost`
`POSTGRES_PORT`	`5432`
`JWT_SECRET_KEY`	`ci-test-secret-key-not-used-in-production`

Table 62: Environment variables configured for the CI job.

Figure 66 and Figure 67 show the two parallel jobs that make up the CI workflow.

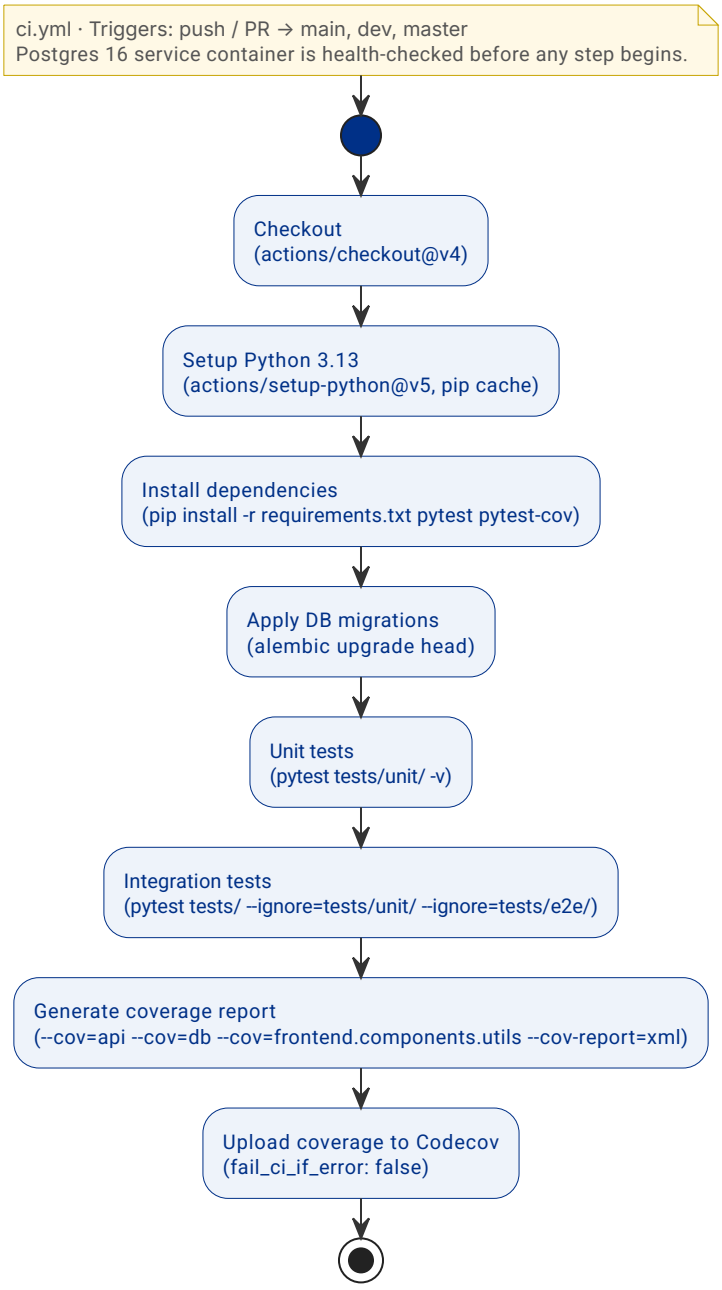

Figure 66: CI pipeline - `test` job. A PostgreSQL 16 service container is health-checked before any step begins.

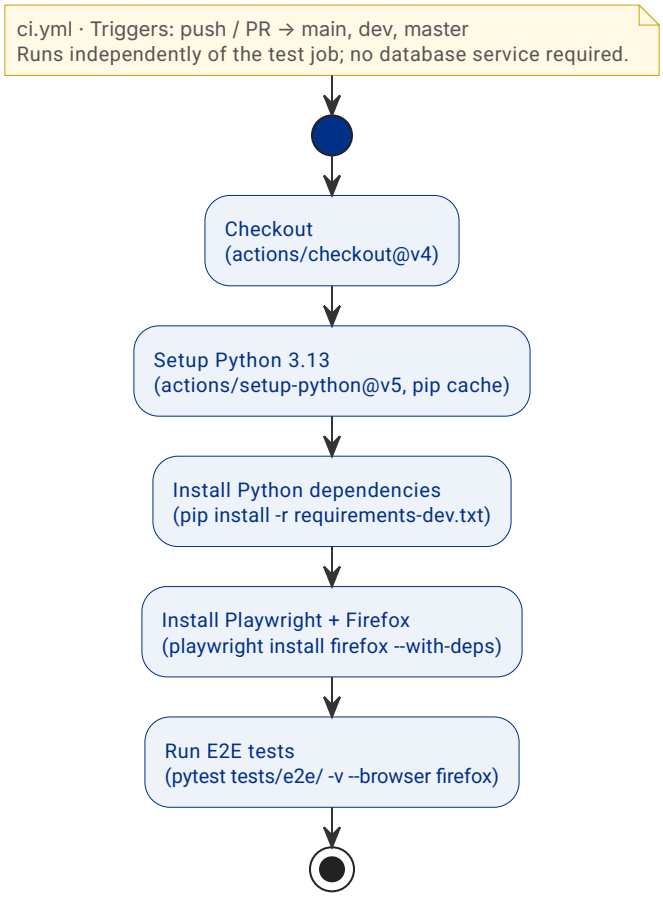

Figure 67: CI pipeline - `e2e` job. Playwright drives Firefox against the Shiny frontend with no database required.

5.4.3 Security Scanning

The security workflow (`security.yml`) runs three independent jobs in parallel: Bandit for Python SAST, pip-audit for dependency CVE scanning, and Trivy for container image scanning. It is triggered on every push and pull request to `main` and additionally on a weekly schedule (Mondays at 08:00 UTC), so that newly disclosed CVEs are surfaced even when no code change occurs.

Bandit uses a two-pass approach: the first pass writes a JSON artifact at medium severity (for review), and the second pass gates the job at high severity (blocking the workflow on critical findings). pip-audit scans `requirements.txt` against the OSV database. Trivy scans the built API Docker image for OS-level and library vulnerabilities; it always reports findings but never fails the workflow (`exit-code: 0`), since fixing base-image vulnerabilities may depend on an upstream release.

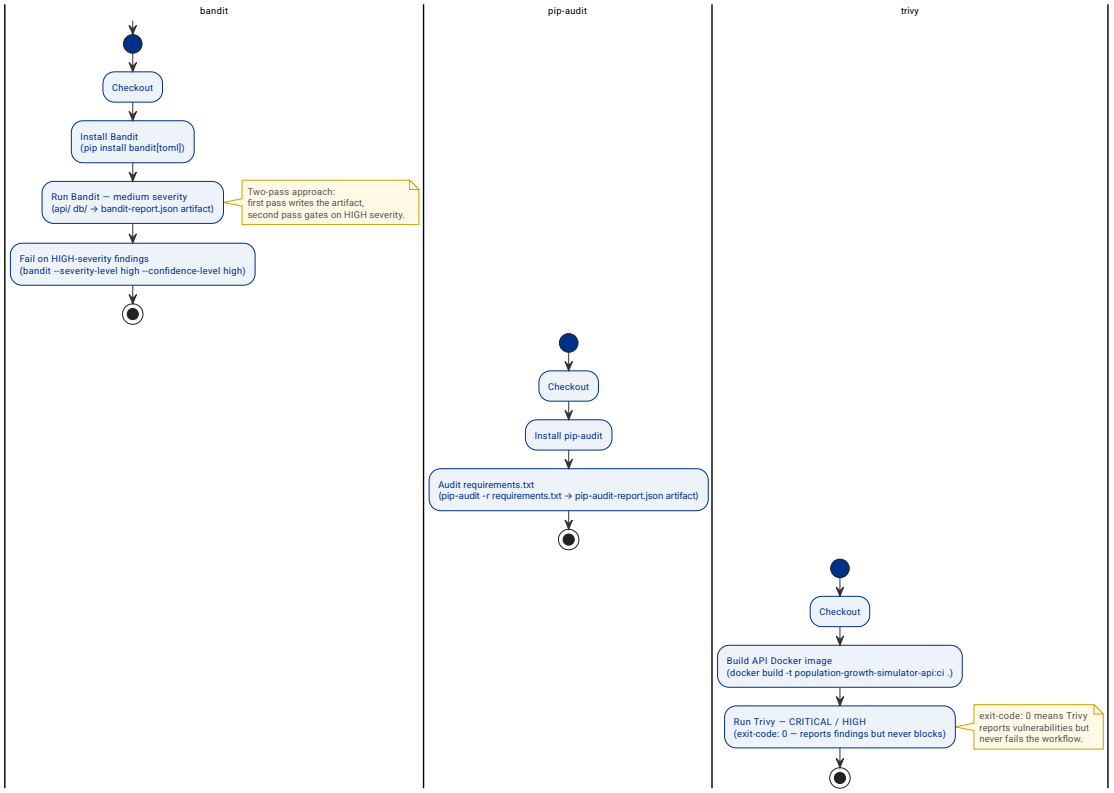

Figure 68: Security pipeline. Three independent jobs run in parallel on push/PR to `main` and on a weekly schedule.

5.4.4 Continuous Deployment

No automated deployment pipeline exists in the current implementation. Deployment is performed manually by running `docker compose up -d --build` on the target host after pulling the latest `main` branch. Automating this step could be done triggering a CD pipeline to deploy over a remote machine when changes are introduced to `dev` or `main`. Right now, the deployment is performed automatically using Railway. Information about this deployment can be found in Section 7.2.3.

5.5 Monitoring Design

5.5.1 Observability Strategy

A lightweight observability approach was chosen, appropriate to the educational and research focus of the application. Two mechanisms are in place.

Uvicorn emits structured access logs by default for every HTTP request, capturing the method, path, response status code, and processing time. These logs are written to standard output and collected by Docker, making them visible via `docker compose logs -f api` during local development and available to any log aggregation tool in a cloud deployment.

Test coverage is tracked via Codecov on every push to `main`. The CI pipeline uploads an XML coverage report for `api/` and `db/`, giving a continuous, visible measure of how much of the codebase is exercised by the test suite and surfacing coverage regressions in pull request reviews.

5.5.2 Monitoring Architecture

The `GET /health` endpoint returns a static `200 OK` response with no authentication required. It serves as a liveness probe for Docker Compose health checks: the `api` container is only considered ready after this endpoint responds successfully, which gates the `frontend` container from starting before the API is available. The same endpoint can be polled by an external load balancer or uptime monitor in a production deployment.

The `db` container is configured with a PostgreSQL-specific health check (`pg_isready`) that gates the `api` and `frontend` containers through Docker Compose `depends_on` conditions. This prevents the application from starting against a database that has not yet finished initialising.

5.5.3 Dashboard and Alert Design

Prometheus metrics collection, Grafana dashboards, and alert rules are not implemented in the current release. This is a deliberate scope decision: adding a Prometheus exporter (e.g. `prometheus-fastapi-instrumentator`), a Prometheus scrape configuration, and Grafana dashboard definitions would meaningfully increase the operational surface area without adding educational or research value in this phase. Integrating full observability infrastructure is identified as future work in Section 8.2.

5.6 Test Design

The test suite is organised around a two-speed philosophy. Fast unit tests with no external dependencies provide immediate feedback during development; thorough integration tests against a real PostgreSQL instance validate the full HTTP contract, SQL correctness, and migration state.

Unit tests (`tests/unit/`) use `unittest.mock.MagicMock` to substitute repository implementations, allowing service business logic and Pydantic schema validators to be exercised in complete isolation from the database. The suite covers `MatrixService`, `SimulationService`, `AuthService`, and the schema validation edge cases defined in `api/schemas.py`. Unit tests complete in seconds and are run first in CI; a failure here aborts the pipeline before integration tests are attempted.

Integration tests (`tests/`) use FastAPI's `TestClient` against a dedicated `matrix_db_test` database. Shared pytest fixtures (`alice`, `bob`, `compadre_matrix_id`, `alice_matrix`) create and tear down test data per session, keeping tests isolated from one another. This layer validates full request-to-database-to-response round-trips, including authentication flows, ownership rules, and the simulation computation path.

End-to-end tests (`tests/e2e/`) use Playwright for Python to drive the Shiny frontend in a real browser. Two run modes are supported: `RUN_MODE=mock` serves canned fixture data through a lightweight mock API server (no database required); `RUN_MODE=real` exercises the full stack. E2E tests are currently run manually and are not part of the CI pipeline.

The test plan (what is tested and why) is described in Section 4.3. Section 6.3 presents the implementation results and coverage numbers.

Chapter 6

IMPLEMENTATION

6.1 APPLICATION STRUCTURE

The application follows the three-tier architecture described in Section 5.1.2 and is deployed as three Docker containers as shown in Figure 54. Each tier maps to a top-level directory in the repository:

- **`frontend/`** - Python Shiny single-page application (`app.py` as entry point). Components for each tab live in `frontend/components/`; shared utilities (API client, Plotly helpers) in `frontend/components/utils.py`. All data flows through the REST API; no computation is performed in the frontend.
- **`api/`** - FastAPI application (`main.py` registers routers and CORS middleware). The internal structure enforces the controller → service → repository separation: `api/controllers/` for HTTP parsing, `api/services/` for business logic and algorithms, `api/repositories/` for database access. Input validation lives in `api/schemas.py`; response serialisation in `api/records.py`.
- **`db/`** - SQLAlchemy ORM models (`db/models.py`) and an Alembic migration history (`alembic/versions/`). The seed script (`db/seed_compadre.py`) populates COMPADRE matrices on first start-up and is idempotent on subsequent restarts.

6.1.1 SIMULATION ALGORITHMS

The simulation logic resides entirely in `api/services/simulation_service.py` and is invoked by both the ephemeral endpoint (`POST /v1/simulations/run`) and the stored endpoint (`POST /v1/simulations`).

Deterministic algorithm. A single Leslie/Lefkovitch matrix $\boldsymbol{A}$ is applied iteratively for T steps:

$$\boldsymbol{v}(t+1) = \boldsymbol{A} \cdot \boldsymbol{v}(t), \quad t = 0, 1, ..., T-1$$

The result is the full trajectory $[\boldsymbol{v}(0), \boldsymbol{v}(1), ..., \boldsymbol{v}(T)]$ - a list of $T+1$ population vectors.

Stochastic algorithm. Given K matrices $\boldsymbol{A}_0, ..., \boldsymbol{A}_{\{K-1\}}$ and a run count N (parameter `n_runs`, range 10-50 000, default 100), the algorithm executes N independent runs. Before each run, a single matrix index i is sampled uniformly at random from the master RNG (`numpy.random.default_rng`). That matrix is committed for all T steps of the run:

$$i_r \sim \text{Uniform}(\{0, ..., K-1\}), \quad \boldsymbol{v}_{r(t+1)} = \boldsymbol{A}_{\{i_r\}} \cdot \boldsymbol{v}_{r(t)}, \quad r = 1, ..., N$$

After all runs complete, four aggregate trajectories are computed across the $N \times (T+1) \times S$ tensor (where S is the number of stages):

```
all_histories = np.zeros((n_runs, n_steps + 1, n_stages))
# ... fill tensor ...
mean_h = all_histories.mean(axis=0)   # returned as result_history
var_h  = all_histories.var(axis=0)    # result_variance
min_h  = all_histories.min(axis=0)    # result_min_history
max_h  = all_histories.max(axis=0)    # result_max_history
```

The frontend renders the mean trajectory with a shaded min/max band (15 % opacity) using Plotly `fill="toself"` traces.

Quasi-extinction algorithm. Quasi-extinction does not require that a population reach zero; instead, a run is considered extinct once any tracked stage falls below a user-supplied threshold. The analysis is implemented in `api/services/quasi_extinction_service.py` and runs as an asynchronous background job (HTTP 202 → poll `GET /v1/jobs/{id}`).

Each stage s carries two parameters: a threshold $q_s \geq 0$ and an exclusion flag. Excluded stages are never checked. A run goes extinct at the first step t^* where $v_{s(t^*)} < q_s$ for some non-excluded stage s. With the default $q_s = 0$, we're measuring actual extinction (0 individuals), so in practice the user sets q_s to an ecologically meaningful minimum viable population for each stage.

The same per-run commitment pattern as the stochastic simulation applies: the master RNG (`numpy.random.default_rng`, optionally seeded for reproducibility) draws one matrix index i_r uniformly at the start of each run, and that matrix is used for all T steps:

$$i_r \sim \text{Uniform}(\{0, ..., K-1\}), \quad \boldsymbol{v}_r(t+1) = \boldsymbol{A}_{i_r} \cdot \boldsymbol{v}_r(t), \quad r = 1, ..., N$$

After N runs the service computes the *quasi-extinction probability*

$$\hat{p}_{\text{qe}} = \frac{N_{\text{extinct}}}{N}$$

where N_{extinct} is the number of runs that went extinct within the T-step horizon. The complementary quantity $1 - \hat{p}_{\text{qe}}$ is the empirical probability that a population starting from the given initial vector survives beyond the horizon under the stochastic matrix environment. The result also includes the time-to-extinction distribution (a sparse map from step t^* to run count), which stages triggered extinction, and the per-run stochastic growth rate λ_s (the geometric mean of step-wise norm ratios). Population trajectories (mean, min, max, variance) across all N runs are returned alongside the summary statistics.

6.2 CI/CD Pipeline Implementation

6.2.1 Repository Configuration

The project is hosted as a single GitHub repository. All automation configuration lives under the `.github/` directory at the repository root:

- **`.github/workflows/ci.yml`**: The continuous integration workflow (unit tests, integration tests, coverage upload).
- **`.github/workflows/security.yml`**: Security scanning workflow (Bandit SAST, pip-audit CVE scan, Trivy container scan).
- **`.github/workflows/docs.yml`**: Documentation publishing workflow (compile Typst PDF, publish to GitHub Releases and GitHub Pages).
- **`.github/dependabot.yml`**: Dependabot configuration for automated dependency updates (Section 6.2.5).

SonarCloud static analysis is configured through `sonar-project.properties` at the repository root. It points to `api/`, `db/`, and `frontend/` as source directories and to `coverage.xml` (generated by CI) as the coverage report path. Two GitHub Actions secrets are required: `CODECOV_TOKEN` for uploading coverage reports to Codecov, and `SONAR_TOKEN` for SonarCloud analysis.

6.2.2 Branch Strategy and Protection Rules

The branching model is described in Section 5.3.1. In summary: `main` is the single long-lived branch and is kept in a deployable state at all times. New work is developed on short-lived branches named `feature/<topic>` or `fix/<topic>`. A pull request targeting `main` is required before merging; the CI workflow must pass before the merge is allowed.

This is enforced through GitHub's branch protection rules configured on `main`:

- **Require a pull request before merging**: Direct pushes to `main` are blocked.
- **Require status checks to pass**: The `Tests (Python 3.13)` and `E2E tests (Firefox)` CI jobs must succeed before a pull request can be merged.
- **Require branches to be up to date before merging**: The head branch must be current with `main` at the time of merge.

6.2.3 Implemented Pipeline Stages

Two workflows constitute the active quality gates: `ci.yml`, triggered on every pull request targeting `main`, and `security.yml`, triggered weekly on a schedule and on every push to `main`.

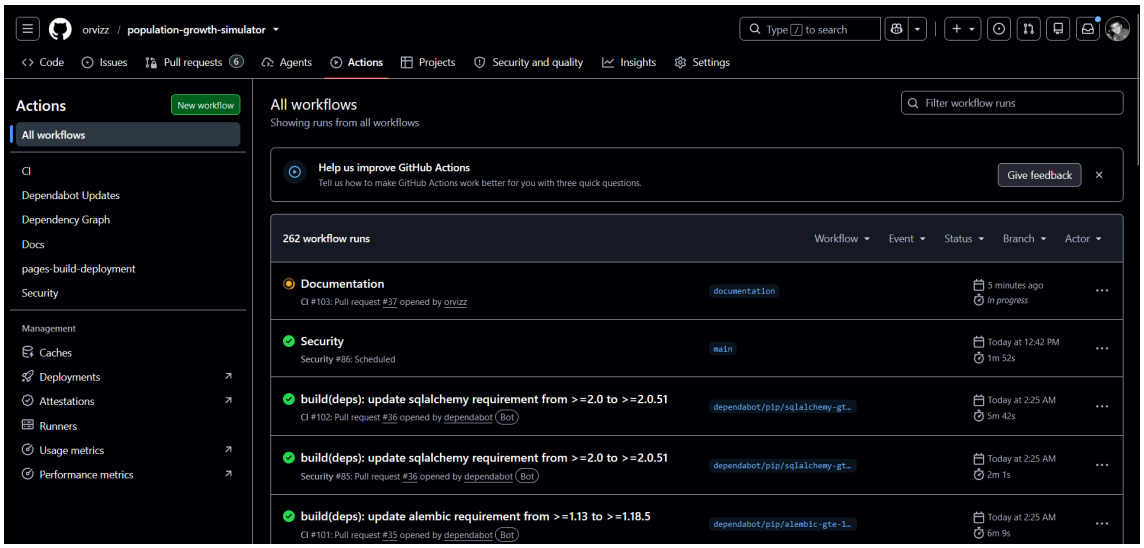

Figure 69: GitHub Actions workflows tab. Five workflows are active: CI, Security, Docs, Dependabot Updates, and the built-in pages-build-deployment.

The `ci.yml` workflow runs two jobs in parallel on every pull request. The `Tests (Python 3.13)` job initializes a PostgreSQL service container, checks out the repository, installs dependencies, applies Alembic migrations against the test database, runs unit tests with `pytest tests/unit/`, runs integration tests with `pytest tests/`, generates a coverage report, and uploads it to Codecov. The `E2E tests (Firefox)` job sets up Python 3.13, installs Playwright alongside the Firefox browser binary, and runs end-to-end tests against the Shiny frontend. A complete CI run finishes in approximately six minutes.

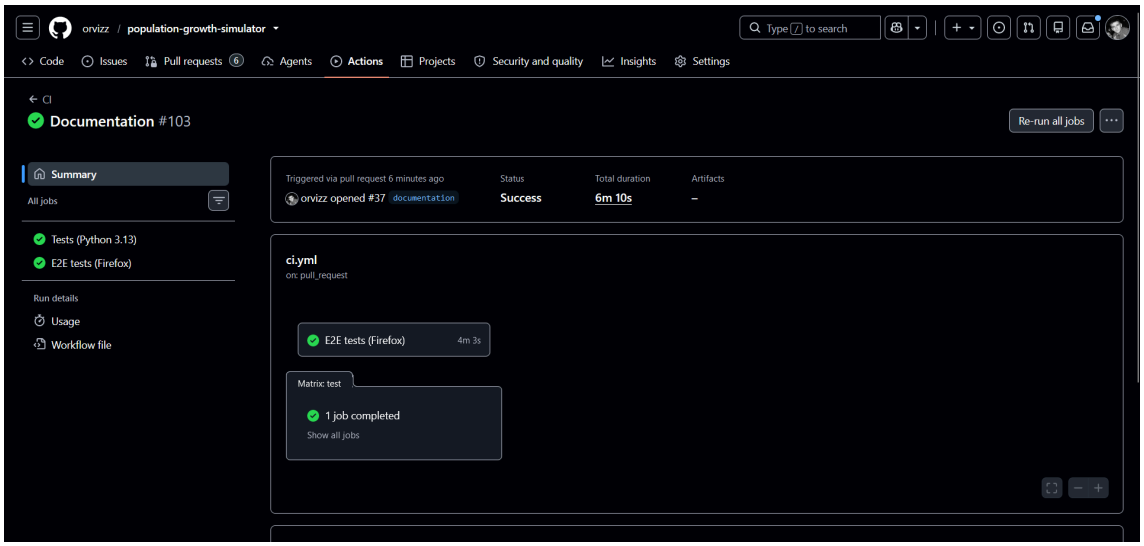

Figure 70: CI run summary. Both the `Tests (Python 3.13)` and `E2E tests (Firefox)` jobs completed successfully in 6 minutes 10 seconds.

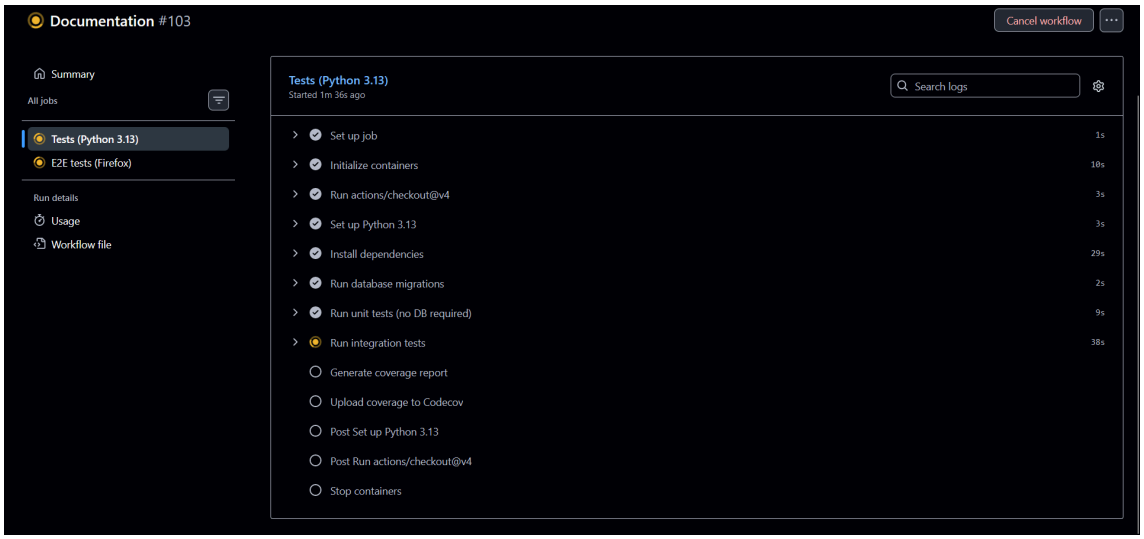

Figure 71: `Tests (Python 3.13)` job expanded, showing the full step sequence: container initialization, dependency installation, database migrations, unit tests, integration tests, and coverage upload.

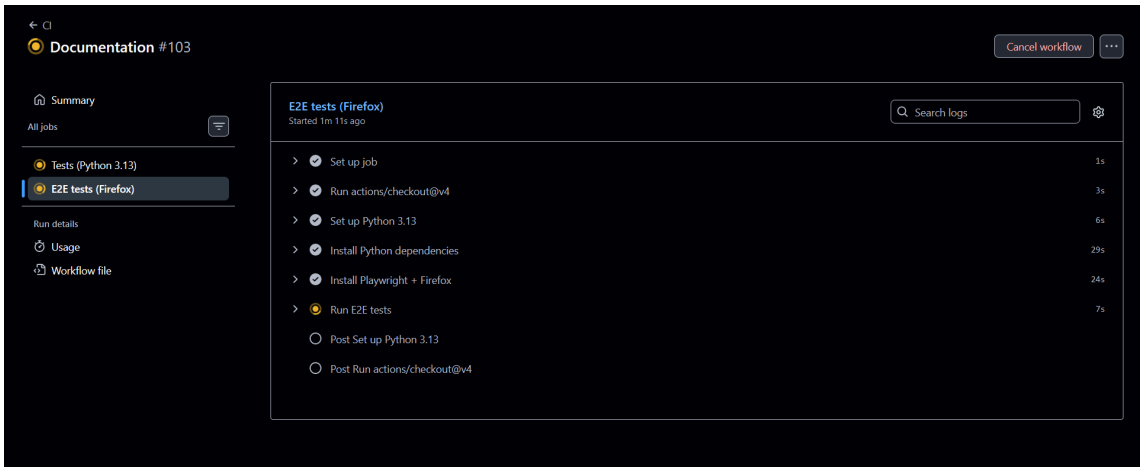

Figure 72: `E2E tests (Firefox)` job expanded, showing the steps to set up Playwright and the Firefox binary before running the end-to-end test suite.

The `security.yml` workflow runs three jobs in parallel, each targeting a distinct threat surface:

- **`Bandit SAST`**: scans Python source code for common security antipatterns; fails on any high-severity finding (14 s).
- **`pip-audit (dependency CVEs)`**: audits `requirements.txt` against the Python Vulnerability Database for known CVEs (37 s).
- **`Trivy (container scan)`**: builds the API Docker image and scans it for OS-level CVEs (1 m 47 s).

Because the jobs run in parallel, total wall-clock time is bounded by the slowest job, approximately two minutes per run.

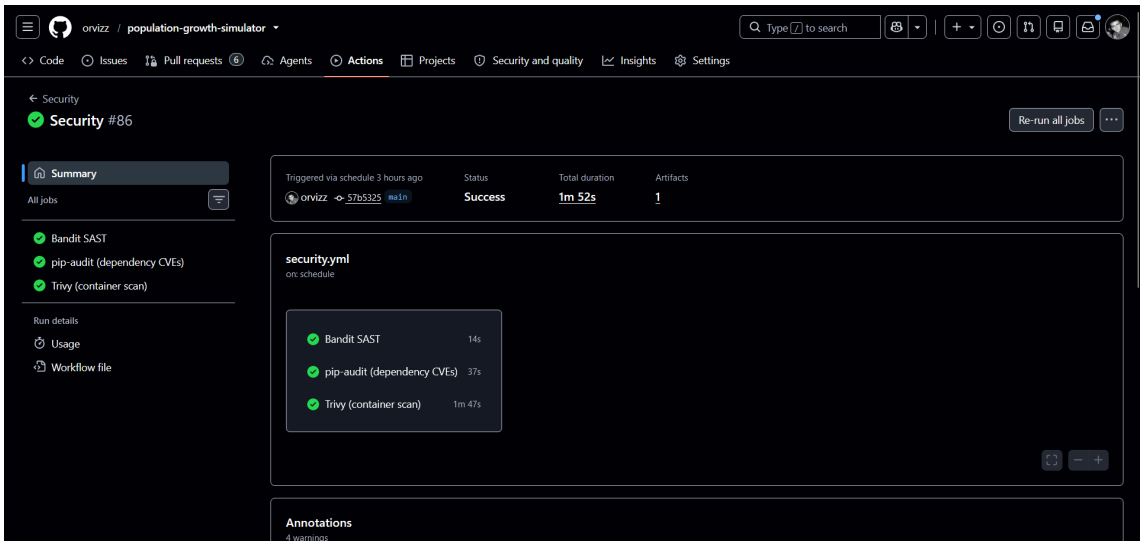

Figure 73: Security workflow run. The three jobs execute in parallel; the entire workflow completes in under two minutes.

Branch protection rules on `main` require both `Tests (Python 3.13)` and `E2E tests (Firefox)` to pass before a pull request can be merged. The merge button is disabled while checks are pending or failed.

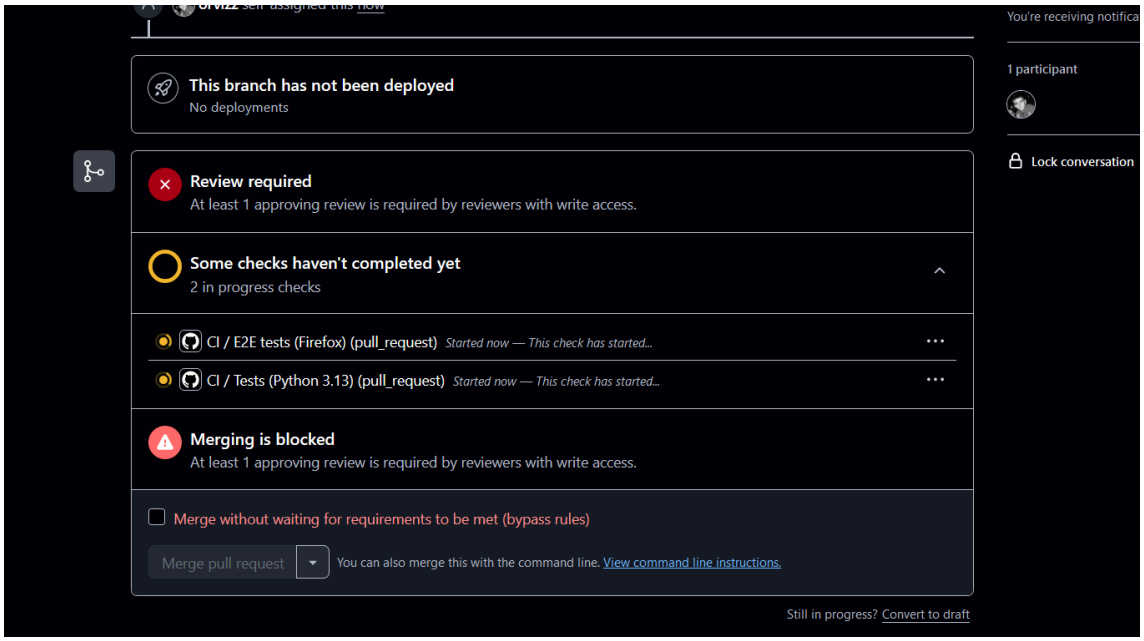

Figure 74: Pull request gate. GitHub blocks the merge action until both required CI checks pass and at least one approving review is recorded.

6.2.4 Documentation Publishing Pipeline

Alongside `ci.yml` and `security.yml`, a third workflow (`docs.yml`) automates publishing the compiled TFG document. It triggers on every push to `main` that touches `tfg-doc/**`, plus manual dispatch, and runs three jobs:

1. **`build`** - installs Typst and runs `typst compile tfg-doc/main.typ tfg-doc/main.pdf`. The resulting PDF is uploaded as a workflow artifact so the two downstream jobs do not each recompile it.

2. **`release`** - downloads the artifact and publishes it as the asset of a single rolling GitHub Release tagged `docs-latest`. Because the tag is reused on every run rather than incremented, the download URL never changes between revisions.
3. **`pages`** - downloads the artifact alongside a generated redirect `index.html` and deploys both to GitHub Pages, so the root Pages URL opens the PDF directly.

The `release` and `pages` jobs both depend only on `build`, not on each other, so a failure in one does not block the other from publishing. The `pages` job additionally requires a one-time manual step that cannot be scripted: enabling Settings → Pages → Source: "GitHub Actions" on the repository. On a private repository this can still be configured, but the published site only becomes publicly reachable once the repository itself is made public.

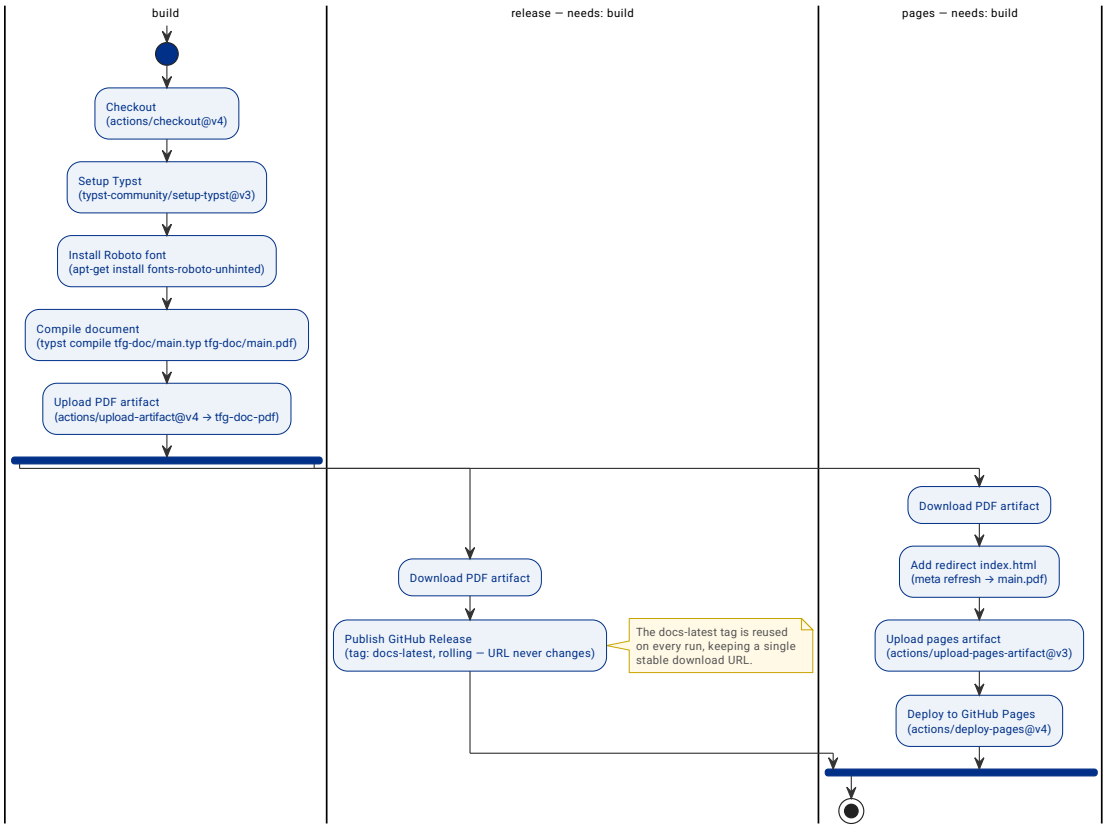

Figure 75: Documentation pipeline. The `build` job compiles the PDF; `release` and `pages` run in parallel once the artifact is ready.

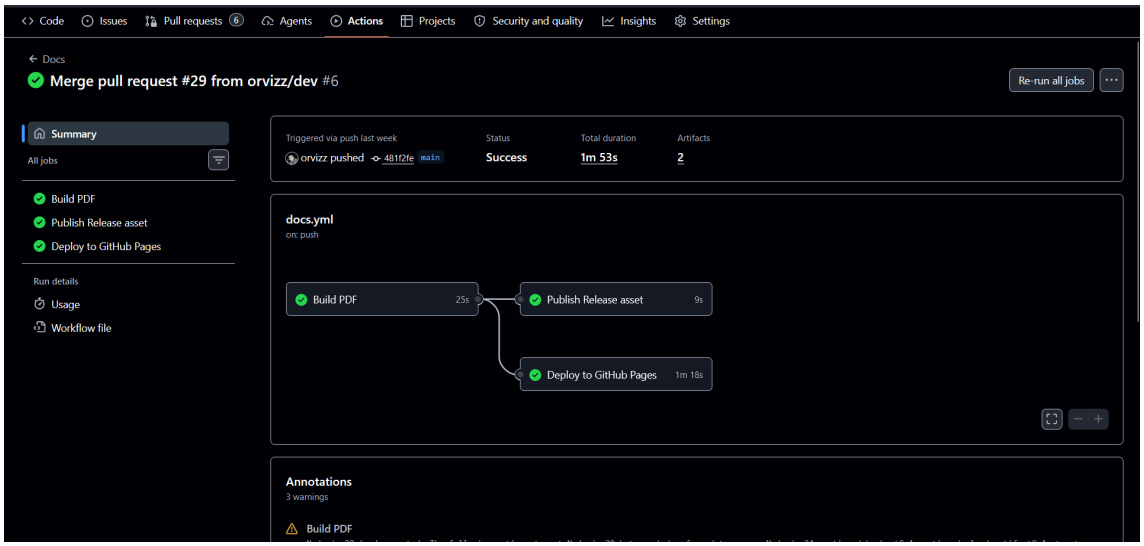

Figure 76: A completed `docs.yml` run triggered by a push to `main`. The `build` job finished in 25 seconds; `release` and `pages` ran in parallel and completed the full workflow in under two minutes.

6.2.5 Automated Dependency Updates

Dependabot is configured in `.github/dependabot.yml` to open pull requests every Monday for three dependency ecosystems: Python packages (`pip`), GitHub Actions version pins, and the Docker base image. Minor and patch updates are grouped into a single PR per ecosystem to reduce review noise; major version bumps arrive as individual PRs. A maximum of five open PRs per ecosystem prevents queue build-up. Every Dependabot PR triggers the CI workflow (`ci.yml`), so updates are automatically validated before they can be merged.

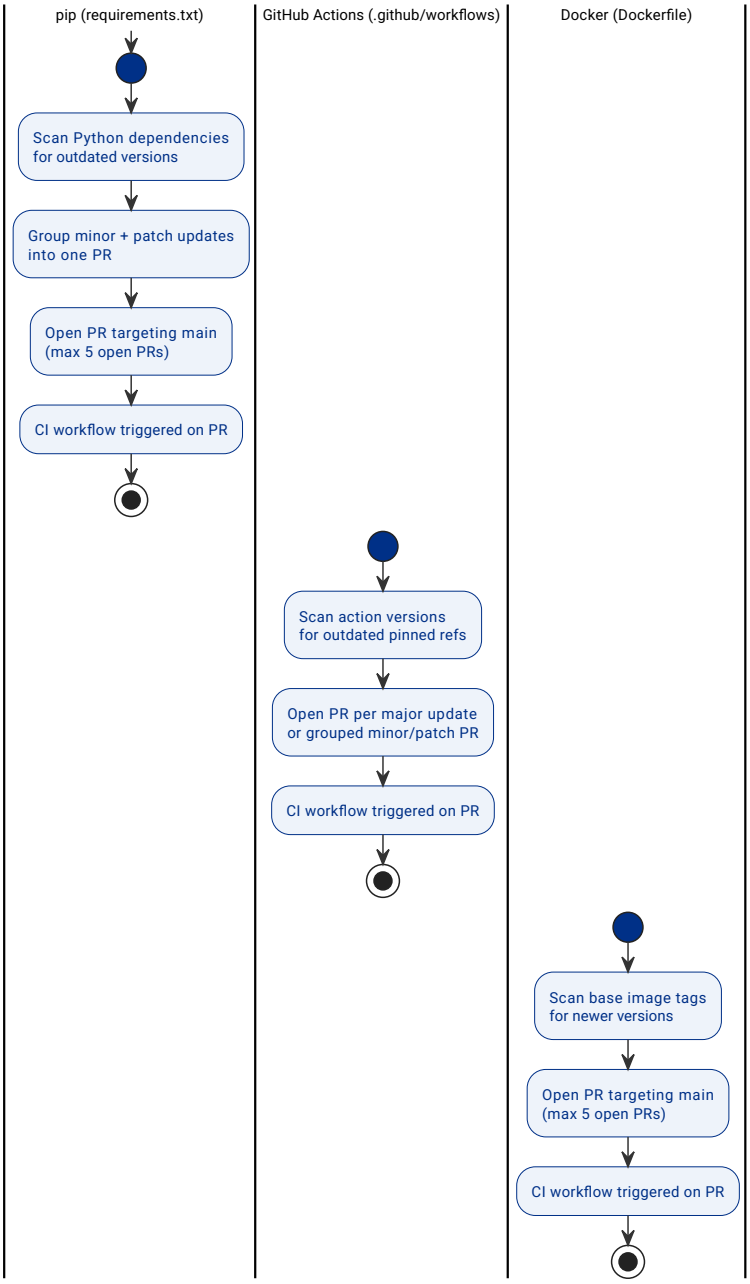

Figure 77: Dependabot automation. Three ecosystems are scanned weekly; each opens a PR that is automatically validated by CI.

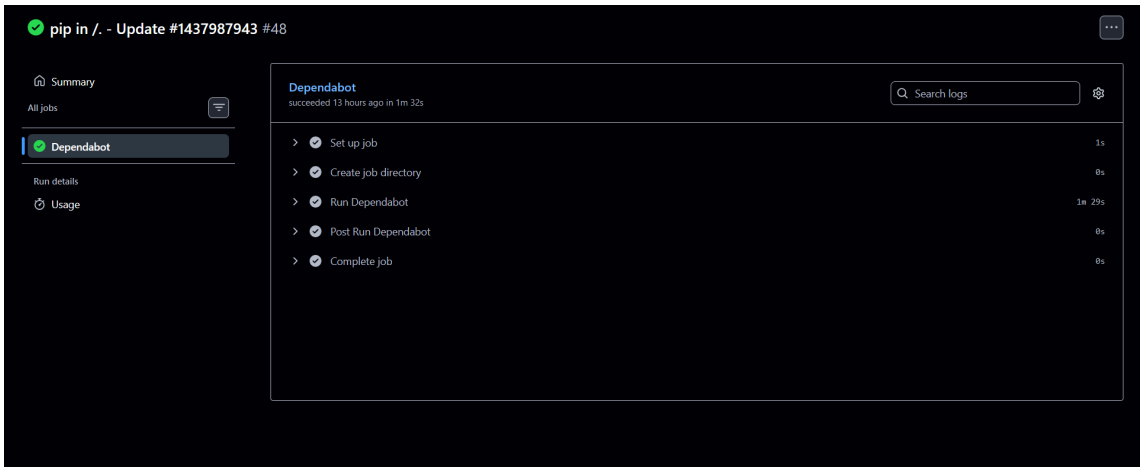

Figure 78: A Dependabot `pip` update run. The single `Dependabot` job completes in under two minutes and triggers a CI run on the resulting pull request.

6.3 Implementation of Tests

The project is tested at three levels. Unit tests (`tests/unit/`) use `unittest.mock.MagicMock` to replace all repositories and require no running database; they can be executed standalone with `python -m pytest tests/unit/ -v`. Integration tests (`tests/`) exercise the full request-response cycle against a live PostgreSQL instance. End-to-end tests (`tests/e2e/`) drive the Shiny frontend with Playwright, either against a mock API (CI default) or the real stack.

Test module	Tests	Passed	Focus
`test_schemas.py`	56	56	Input validation (Pydantic schemas)
`test_simulation_service.py`	55	55	Simulation algorithms and service logic
`test_quasi_extinction_service.py`	38	38	Quasi-extinction Monte Carlo and job lifecycle
`test_analytics_service.py`	22	22	Eigenvalue analytics (λ_1, λ_s, elasticities)
`test_matrix_service.py`	56	56	Matrix CRUD, visibility, sharing
`test_auth_service.py`	10	10	Registration, login, JWT
`test_frontend_utils.py`	7	7	Frontend `api()` HTTP helper: empty-body handling, error sanitisation
`test_matrix_grid.py`	5	5	Shared matrix cell-grid editor (`render_matrix_grid`, `read_matrix`)
Total	**249**	**249**	

Table 63: Unit test results by module (`python -m pytest tests/unit/ -v`, 249 collected, 249 passed)

Test module	Tests	Focus
`test_auth.py`	10	Registration, login, token validation
`test_health.py`	1	Health check endpoint
`test_jobs.py`	43	Quasi-extinction job lifecycle, polling, deletion
`test_matrices.py`	59	Matrix CRUD, filtering, pagination, PATCH (incl. US-11 edit fields), export/import

Test module	Tests	Focus
`test_matrix_visibility.py`	35	Private/shared/public visibility transitions, sharing
`test_simulations.py`	60	Ephemeral and stored simulations, export/import, deletion
Total	**208**	

Table 64: Integration test results by module (`python -m pytest tests/ --ignore=tests/unit --ignore=tests/e2e -v`, requires PostgreSQL)

Test module	Tests	Focus
`test_auth.py`	5	Login/registration modals, auth state
`test_browse.py`	10	Matrix browser search and filtering
`test_my_matrices.py`	11	Create, delete, and edit flows, the edit-flow tests (pre-population, species/kingdom save, matrix cell save, stage add + grid resize) were added alongside the US-11 frontend fix
`test_navigation.py`	2	Tab navigation, UI state
`test_quasi_extinction.py`	10	Quasi-extinction analysis UI
`test_simulate.py`	9	Simulation UI flow: search, add matrices, run, save, delete
Total	**47**	

Table 65: End-to-end test results by module (`python -m pytest tests/e2e/ -v --browser firefox`, mock-API mode)

Across all three levels, **504 tests** pass (249 unit + 208 integration + 47 E2E). Running the unit and integration suites together with `--cov=api --cov=db --cov=frontend.components.utils` (the scope tracked by Codecov in CI) reports **83%** line coverage; the lowest-covered areas are the COMPADRE/COMADRE seeding scripts (`db/seed_compadre.py`, `db/seed_comadre.py`, 0%, exercised only by the one-time container startup path, not by the test suite) and `frontend/components/utils.py`'s error-formatting branches for less common HTTP failure shapes.

Key test classes related to the stochastic simulation rework:

- `TestComputeStochastic` (6 tests) - verifies the multi-run algorithm shape, variance, reproducibility with seed, and the bimodal commitment property (`test_all_runs_commit_to_single_matrix`: with a doubling matrix and a zeroing matrix, after 3 steps the minimum is 0.0 and the maximum is 8.0 - no intermediate values are possible because each run commits to a single matrix).
- `TestPerRunMatrixSelection` (1 test) - verifies that quasi-extinction probability with a doubling/zeroing pair is 0.5 (per-run commitment), not 0.97 (which per-step selection would produce over 5 steps).

Chapter 7

MANUALS

7.1 User Manual

The Population Growth Simulator is a web application for exploring structured population models. It requires no installation: open a browser and navigate to the application URL at https://popgrowthsim.marioorviz.dev.
The interface is organised into four tabs, **Browse Matrices**, **Simulate**, **Quasi-Extinction**, and **My Matrices**, visible in the header on every page. Most features are freely accessible without an account. Creating an account unlocks saving and managing your own work.

7.1.1 Getting Started

Open the application in a browser. The **Browse Matrices** tab loads automatically, showing the full catalogue of population matrices from the COMPADRE and COMADRE databases (Figure 79). No login is required to explore the catalogue or run an ephemeral simulation.

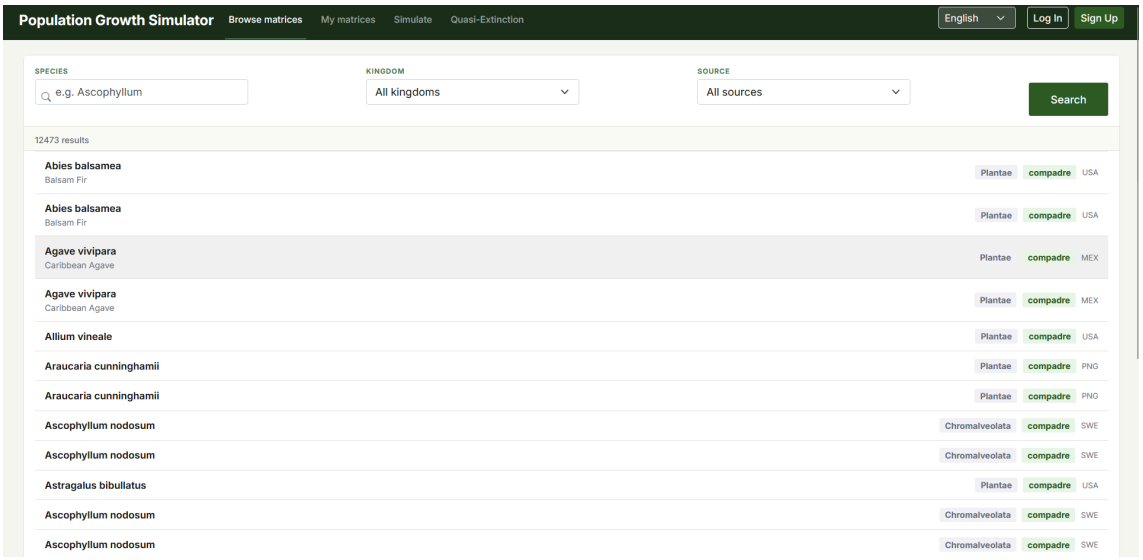

Figure 79: Application landing page: Browse Matrices tab with the full matrix catalogue and filter controls.

7.1.2 Creating an Account and Logging In

An account is required to save simulations, manage custom matrices, and run quasi-extinction analyses. To register, click **Sign Up** in the header, fill in a username, email address, and a password of at least eight characters containing at least one letter and one number, then submit the form.

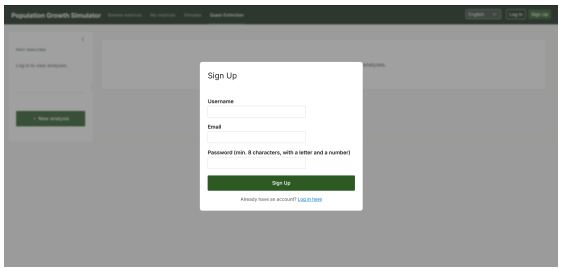

Figure 80: Sign-Up form. Enter username, email, and password to create an account.

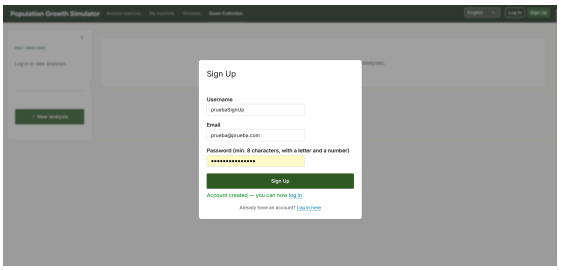

Figure 81: Confirmation message shown after successful registration.

Once registered, click **Log In**, enter your username and password, and submit. The header updates to show your username and a **Sign Out** link (Figure 82). To end the session, click **Sign Out** and the application returns to the unauthenticated state without losing your browsing context.

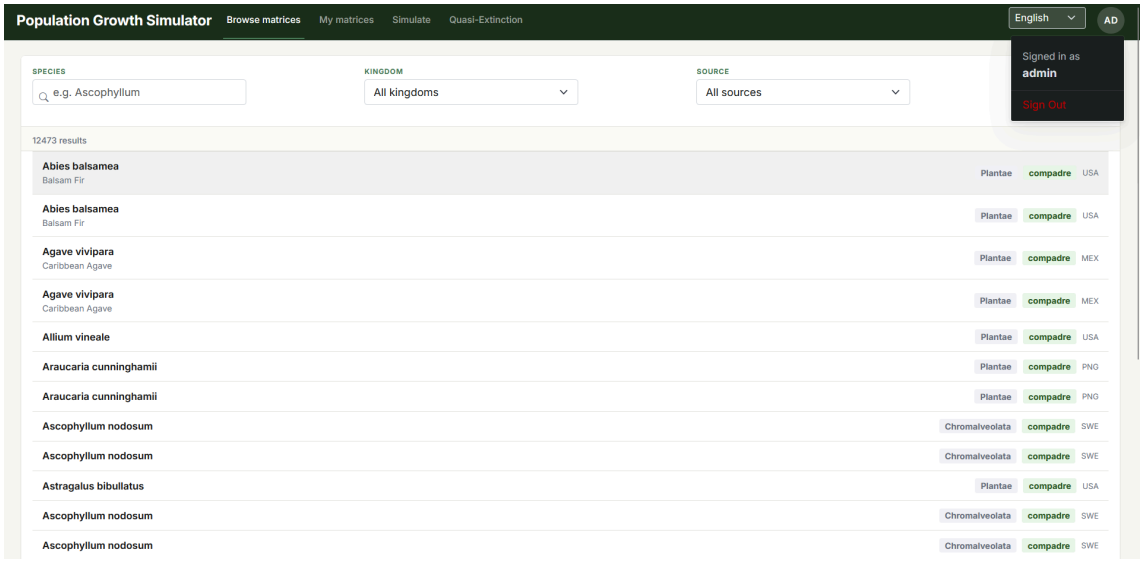

Figure 82: Header after a successful login: auth buttons are replaced by the signed-in username and a Sign Out link.

7.1.3 Browsing the Population Matrix Catalogue

The **Browse Matrices** tab is accessible to everyone. It lists all matrices from COMPADRE (plants) and COMADRE (animals), plus any public custom matrices created by registered users (over 15 000 entries visible in the default unfiltered view).

To narrow the list, use the filter bar:

- Type a species name in the search field; results update as you type.
- Use the **Kingdom** dropdown to restrict to plants or animals.
- Use the **Source** dropdown to restrict to a specific database.

Active filters combine as AND conditions. Click any row to open the **Matrix detail panel** for that species (Figure 83). The panel shows the projection matrix A, the survival matrix U, and the fecundity matrix F as labelled grids, together with an interactive life-cycle network diagram. Click the info button to view publication metadata such as author, study

duration, and geographic origin. Use the **Export JSON** or **Export CSV** buttons to download the matrix data.

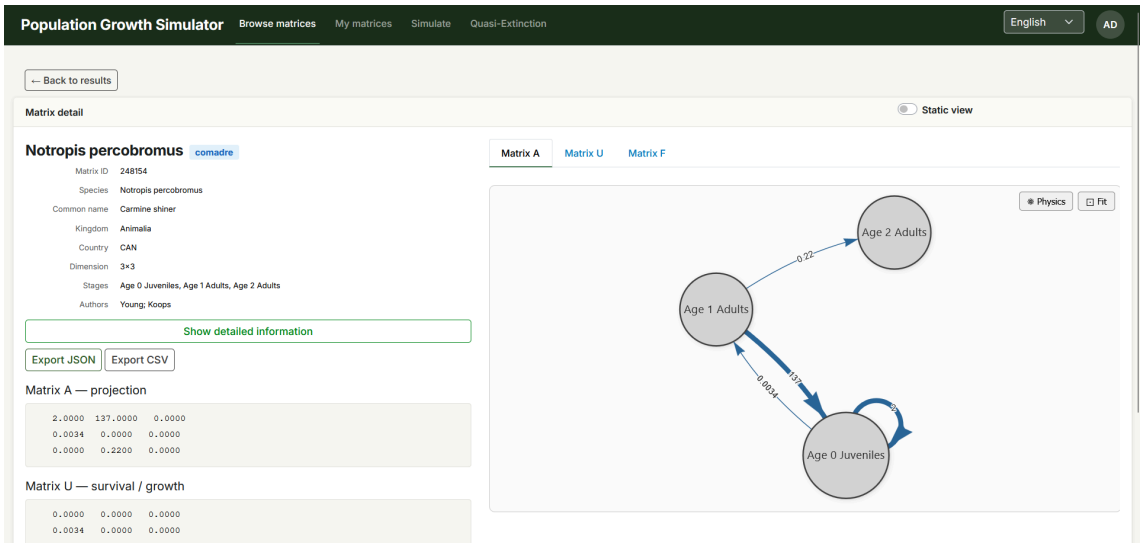

Figure 83: Matrix detail panel. Projection matrices, life-cycle network diagram, and export controls.

7.1.4 Managing Your Custom Matrices

The **My Matrices** tab lets registered users build and maintain a personal library of population matrices. Open the tab and log in if prompted.

Creating a matrix: click **New matrix**, choose the number of stages, assign names to each stage, then fill in the numerical values for the A, U, and F sub-matrices. Save when finished.

Editing and deleting: use the action buttons next to any matrix in the list to open the edit form or delete the matrix (deletion requires confirmation to prevent accidental loss).

Visibility: set each matrix to **private** (visible only to you), **shared** (accessible to specific users you invite by username), or **public** (visible to all users in the Browse catalogue).

Importing: click **Import** to upload a JSON file or a ZIP archive containing multiple JSON files; all matrices are added to your library immediately.

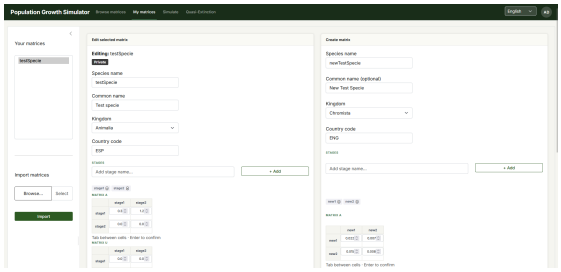

Figure 84: My Matrices tab. Authenticated view with the matrix list and management controls.

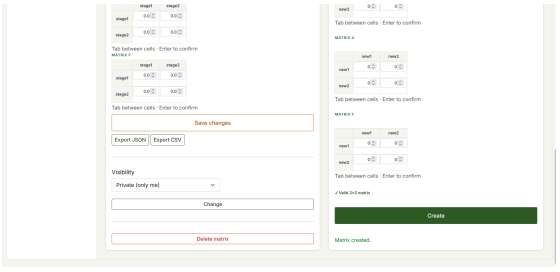

Figure 85: Stage-name configuration step during matrix creation or editing.

7.1.5 Running Population Simulations

The **Simulate** tab computes population trajectories from a selected matrix or set of matrices. Unauthenticated users can run ephemeral simulations; saving results requires login.

7.1.5.1 Setting Up a Simulation

The left sidebar guides you through four steps:

1. **Matrix**: Type a species name in the search field and click the species you want to use.
2. **Mode**: Choose **Deterministic** (a single fixed matrix applied at every time step) or **Stochastic** (the model randomly selects one matrix per step from a set you provide, capturing environmental variability across years or seasons).
3. **In Simulation**: Review the selected matrix; for stochastic runs, add further matrices using the **Add** button.
4. **Parameters**: Enter one initial population value per life-history stage, set the number of time steps (1–50,000), and optionally a random seed for reproducibility. Give the run a name if you intend to save it.

Click **Run** to compute the trajectory (Figure 86).

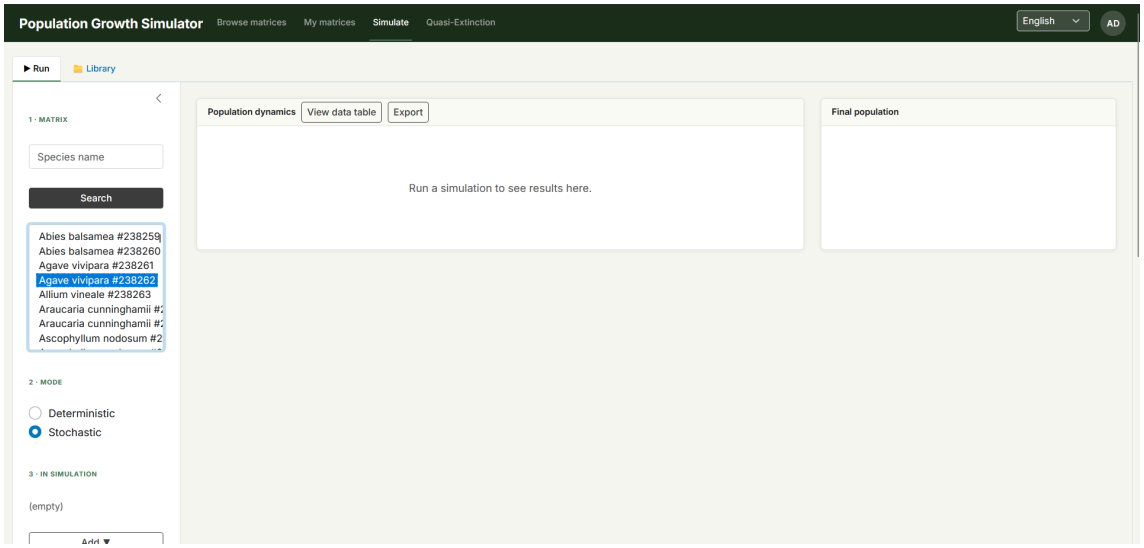

Figure 86: Simulation setup sidebar: Matrix search, mode selection (stochastic shown), and parameter entry.

7.1.5.2 Interpreting Results

The results panel displays the **Population dynamics** chart, plotting the abundance of each life-history stage over time. Switch to the **Analytics** section to see:

- **λ (lambda)**: The asymptotic population growth rate. Values above 1 indicate a growing population; values below 1 indicate decline.
- **Stable stage distribution**: The long-term proportion of individuals expected in each stage.
- **Elasticities**: How sensitive λ is to proportional changes in each matrix element, shown as a colour-coded heatmap. High-elasticity entries flag the demographic rates that most strongly influence population fate.

- **Average matrix A**: The mean projection matrix across all matrices (stochastic runs only).

Click **View data table** to open a modal with the full numerical trajectory for every stage at every recorded time step.

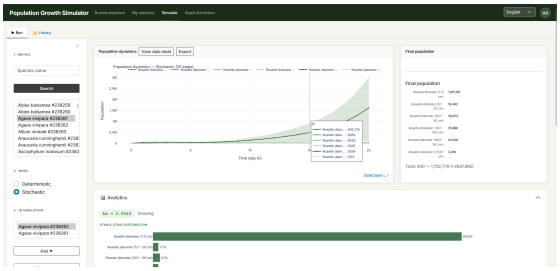

Figure 87: Results panel: Population dynamics chart and final population breakdown by stage.

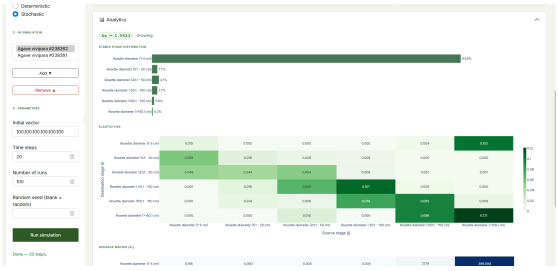

Figure 88: Analytics panel: Growth rate λ, stable stage distribution, and elasticity matrix.

7.1.5.3 Saving and Revisiting Runs

To save the current run, enter a name in the **Parameters** step and click **Save as new** (login required). Saved runs appear in the **Library** view, toggled from the top of the sidebar. Opening a past run restores its dynamics chart and analytics. To remove a run from the Library, click the delete button and confirm.

7.1.6 Quasi-Extinction Analysis

The **Quasi-Extinction** tab estimates the probability that a stochastic population will fall below a critical threshold within a given time horizon. The computation runs as a background job, so you can navigate elsewhere and return once it completes.

7.1.6.1 Configuring a New Analysis

Click **New analysis** in the left sidebar. The configuration form has three parts:

1. **Organism selection**: Search for and add the matrices to use (the same multi-matrix interface as stochastic simulation). Using matrices from different years or environmental conditions captures temporal variability.
2. **Stage configuration** (optional, Figure 90): Open the stage modal to set a per-stage minimum abundance threshold below which that stage is considered locally extinct, or to exclude stages (such as juveniles) that naturally fluctuate near zero.
3. **Simulation parameters**: Set the global extinction threshold (total population count), the time horizon, the number of Monte Carlo runs, and an optional random seed.

When all fields are filled, click **Run analysis**. The job starts immediately and a progress indicator appears in the sidebar; you do not need to wait on the page.

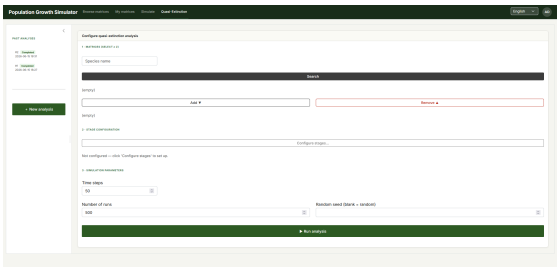

Figure 89: New analysis form. Matrix selection and simulation parameter entry.

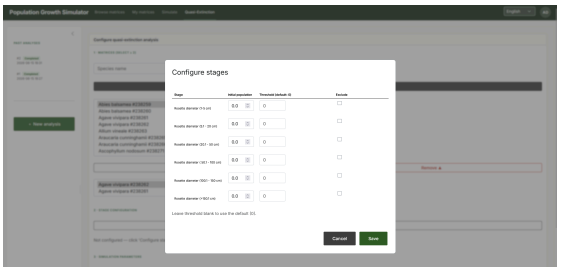

Figure 90: Stage-threshold configuration modal. Per-stage thresholds and stage exclusion controls.

7.1.6.2 Reading the Results

When the job finishes the panel populates with several views (Figure 91, Figure 92):

- **Mean population trajectory**: The average population path across all Monte Carlo runs, with a clickable stage-name legend.
- **Cumulative quasi-extinction probability**: The proportion of runs in which the population crossed the threshold by each point in time. A curve that rises steeply early signals high near-term extinction risk.
- **Time to extinction distribution**: How often each time step was the first moment of extinction across runs.
- **Which stage triggered extinction**: Which life-history stage most frequently caused the population to cross the threshold, helping identify demographic bottlenecks.
- **Stochastic growth rate (λ_s) distribution**: The spread of long-run growth rates across Monte Carlo runs.
- **Data table**: The full numerical mean trajectory for every stage at every time step.

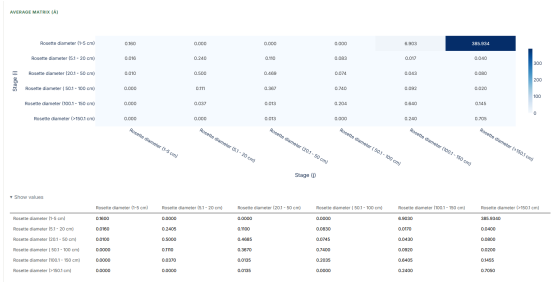

Figure 91: Cumulative quasi-extinction probability curve over the analysis time horizon.

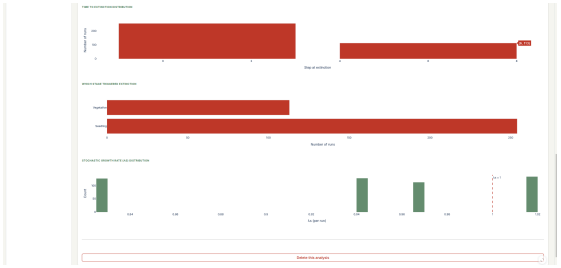

Figure 92: Supplementary results: extinction timing distribution, causation by life stage, and stochastic growth rate distribution.

Past analyses are listed in the left sidebar. To remove one, click the delete button next to its entry and confirm.

7.1.7 Selecting the Interface Language

The language selector is available in the header on every tab. Click it to open a dropdown and choose one of the six supported locales: English, Spanish (Español), Asturian (Asturianu), Galician (Galego), Basque (Euskara), or Catalan (Català). The selected language takes effect immediately across all labels, buttons, and messages, and is remembered across sessions.

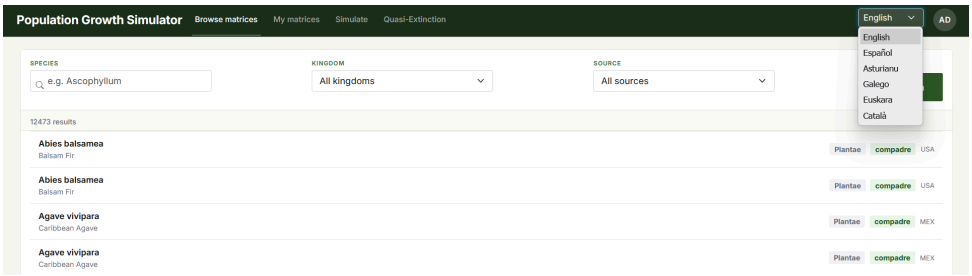

Figure 93: Language selector dropdown. Six locales available from the header on any tab.

7.2 Installation and Configuration Manual

The project can be run in three ways: executing each component manually on the host machine, running the full stack with Docker Compose, or deploying to production on Railway. The first is best suited to active development and debugging (hot-reload, no container overhead); the second reproduces the full multi-service environment locally with a single command; the third is how the live deployment is operated.

7.2.1 Manual Execution (Without Docker)

Prerequisites: Python 3.13+ and PostgreSQL 16+ installed locally.

1. **Install dependencies** - `pip install -r requirements.txt`.
2. **Set up the database** - create the database and bring the schema up to date:

   ```
   createdb matrix_db
   python -m alembic upgrade head
   python -m db.seed_compadre   # one-time COMPADRE seed
   ```

   Unlike the Docker entrypoint (Section 7.2.2), this seed command has no idempotency guard - re-running it manually would re-insert duplicate matrices, so it should only be run once against a fresh database.
3. **Start the services**, each in its own terminal:

   ```
   # Terminal 1 - API
   python -m uvicorn api.main:app --reload --port 8000

   # Terminal 2 - Frontend
   cd frontend
   python -m shiny run app.py --reload --port 8080
   ```

   The `--reload` flag enables hot-reload on source changes, which is the main reason to prefer this method during development.

7.2.2 Docker Deployment

Running `make up` (`docker compose up -d --build`) builds and starts three containers: `db`, `api`, and `frontend`. Figure 94 shows the three containers running locally, each printing connection logs as the frontend and API exchange requests.

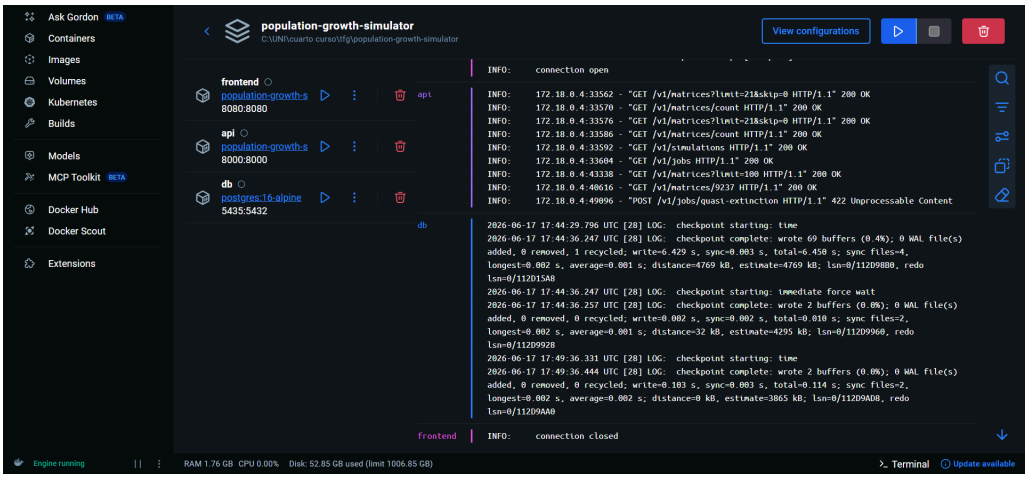

Figure 94: Docker Desktop view of the three running containers (`frontend`, `api`, `db`) after `make up`.

- **`db`** uses the official `postgres:16-alpine` image with no custom build. Data persists across restarts in the named volume `postgres_data`. A `pg_isready` healthcheck gates startup of the other two services, so they only start once the database is actually accepting connections.
- **`api`** is built from the root `Dockerfile`: a `python:3.13-slim` base image installs `requirements.txt`, copies the source tree, and sets `entrypoint.sh` as the container `ENTRYPOINT`. On every start, the entrypoint runs `alembic upgrade head`, then seeds COMPADRE only if zero COMPADRE rows already exist, then seeds COMADRE only if zero COMADRE rows already exist, and finally `exec`s into the container's `CMD` (Uvicorn). This makes startup idempotent - restarting the container never re-seeds or fails on already-applied migrations.
- **`frontend`** is, perhaps counter-intuitively, built from the **same root `Dockerfile`** as `api` (`docker-compose.yml` declares `build: .` with no `dockerfile:` override) - it only overrides the container `command:` to launch Shiny instead of Uvicorn. In practice this means the frontend container also runs the same idempotent migration/seed check on startup, which is harmless but redundant. A separate, entrypoint-less `Dockerfile.frontend` exists specifically for the Railway deployment (Section 7.2.3), so that the frontend service there does not redundantly attempt migrations in production.

Host-to-container port mapping is `5435` → `5432` for the database (the host port avoids colliding with a locally installed PostgreSQL on the default port) and a direct `8000/8080` mapping for `api`/`frontend`. Table 66 lists the available Makefile shortcuts.

Target	Underlying command	Purpose
`make up`	`docker compose up -d --build`	Build images and start all services
`make down`	`docker compose down`	Stop containers; volumes preserved

Target	Underlying command	Purpose
`make logs`	`docker compose logs -f`	Follow logs from all services
`make logs-api`	`docker compose logs -f api`	Follow logs from the API service only
`make clean`	`docker compose down -v`	Stop containers and delete all volumes (full reset)

Table 66: Makefile targets and their underlying Docker Compose commands.

7.2.3 Railway Deployment

The application is deployed on Railway using its GitHub integration, across two environments (Section 7.2.3.2). Each environment is composed of three services that mirror the Docker Compose setup: a managed PostgreSQL database, an `api` service (FastAPI, built from the root `Dockerfile`), and a `frontend` service (Python Shiny, built from `Dockerfile.frontend`).

7.2.3.1 Architecture

Railway provisions PostgreSQL automatically when added to the project; no manual database configuration is required. The `api` and `frontend` services are each connected to the same GitHub repository as a separate service, with the Dockerfile path set per service in its Settings → Build configuration. Services reference one another through Railway's `${{ }}` variable interpolation rather than hard-coded URLs - for example, the API's CORS configuration points at the frontend's public domain, and the frontend's API client points back at the API's public domain. This means a service's public URL can change (e.g. on a custom domain change) without requiring a manual update to the other service.

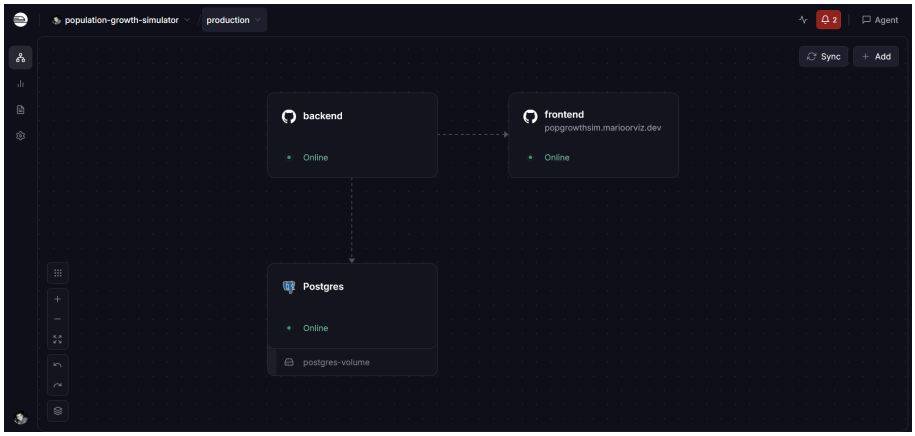

Figure 95: Railway `production` environment project canvas: the database service feeds the API service (labelled `backend` in this deployment), which in turn serves the `frontend` service at its public domain.

7.2.3.2 Environments

The Railway project defines two environments, each provisioning its own independent copy of the three services and database:

- **`production`** tracks the `main` branch and is routed to the custom domain https://popgrowthsim.marioorviz.dev.
- **`staging`** tracks the `dev` branch and is reachable only at its Railway-generated `*.railway.app` domain - no custom domain is configured for it.

Because the two environments are fully isolated, pushing to `dev` redeploys and verifies a change on `staging` without any risk to the live `production` environment; merging `dev` into `main` is what triggers the corresponding `production` redeploy.

7.2.3.3 Environment Variables

Variable	Service	Value
`DATABASE_URL`	`api`	`${{Postgres.DATABASE_URL}}`
`JWT_SECRET_KEY`	`api`	32-byte hex string (`openssl rand -hex 32`)
`CORS_ORIGINS`	`api`	`https://${{frontend.RAILWAY_PUBLIC_DOMAIN}}`
`API_BASE_URL`	`frontend`	`https://${{api.RAILWAY_PUBLIC_DOMAIN}}`

Table 67: Environment variables configured per Railway service.

The `JWT_SECRET_KEY` value is a 32-byte hex string generated once per environment (`openssl rand -hex 32`) and must be kept confidential - unlike the CI environment, it is never a placeholder in production.

7.2.3.4 Operational Notes

On the very first deploy, `entrypoint.sh` seeds around 18,000 COMPADRE and COMADRE records into the database, which takes 2-3 minutes; the API does not respond until seeding completes. The check is idempotent, so subsequent deploys skip it and start in seconds. Railway automatically redeploys each service on every push to its connected branch, running Alembic migrations (`alembic upgrade head`) before the API starts. To roll back, the **Deployments** tab of a service lists prior deploys with a one-click **Rollback** action.

7.2.4 Deployment Verification

Regardless of which method was used, the following checks confirm the system is running correctly (substitute the appropriate base URL - local host/port or the public Railway domain):

1. **API health check** - `GET /health` returns `{"status": "ok"}`.
2. **API docs** - the Swagger UI at `/docs` loads and lists all `/v1/` endpoints.
3. **Frontend** - the Shiny app loads and the **Browse Matrices** tab lists COMPADRE data.
4. **End-to-end flow** - register a user, log in, run a simulation from the **Simulate** tab, and confirm the result is saved.

7.3 Operations and Monitoring Manual

Target audience: a developer or operator maintaining a running instance, whether the local Docker Compose stack (Section 7.2.2) or a Railway environment (Section 7.2.3). Railway-specific rollback and first-deploy behaviour are already covered in Section 7.2.3.4; this section covers day-to-day operation of the Docker Compose stack, plus monitoring that applies to both.

7.3.1 Logs and Service Status

View logs for a single service, following new output as it arrives:

```
docker compose logs -f api
docker compose logs -f frontend
docker compose logs -f db
```

`docker compose ps` lists all three containers with their current status and exposed ports; `docker compose logs --tail=100` without `-f` prints the last 100 lines across every service without following, useful for a quick post-mortem.

7.3.2 Restarting a Service

```
docker compose restart api
```

Restarting `api` or `frontend` re-runs `entrypoint.sh`, which re-checks (idempotently) that migrations are applied and COMPADRE/COMADRE are seeded before starting the process - safe to do at any time. Restarting `db` is safe too: data persists in the `postgres_data` volume regardless of container restarts (only `docker compose down -v` discards it).

7.3.3 Database Backup and Restore

```
# Backup
docker compose exec db pg_dump -U $POSTGRES_USER $POSTGRES_DB > backup_$(date +
%F).sql

# Restore (into an existing, empty database)
docker compose exec -T db psql -U $POSTGRES_USER $POSTGRES_DB < backup.sql
```

For Railway, the managed PostgreSQL instance exposes its own connection string in the service's **Variables** tab, which `pg_dump`/`psql` can target directly using the `-h`/`-p`/`-d` flags instead of `docker compose exec`.

7.3.4 Applying New Migrations

`entrypoint.sh` applies all pending Alembic migrations automatically on container start. To apply a migration to an already-running container without restarting it (for example while debugging):

```
docker compose exec api python -m alembic upgrade head
```

7.3.5 Updating Images

```
docker compose pull   # no-op for locally built images; relevant if a base image moved
docker compose up -d --build
```

Because `api` and `frontend` are built from the local `Dockerfile`/`Dockerfile.frontend` rather than pulled from a registry, "updating" in practice means rebuilding after a `git pull` - `docker compose up -d --build` picks up both source changes and any updated `requirements.txt` dependencies.

7.3.6 Monitoring

- **Test coverage**: the Codecov badge in `README.md` reflects the latest `main` build; click through to codecov.io for line-by-line coverage detail.
- **Code quality**: once SonarCloud is activated in CI (see Section 8.2), its dashboard adds code-smell, duplication, and security-hotspot tracking alongside coverage.
- **Security scanning**: the `security.yml` and `codeql.yml` workflows (Section 5.4) run on every push/PR and weekly on a schedule; their results appear under the repository's **Security** tab on GitHub.
- **Railway**: each service's **Metrics** tab shows CPU/memory/network usage and recent deploy history; the **Observability** view aggregates logs across all three services in one place.

Chapter 8

Conclusions and Future Work

8.1 Conclusions

8.1.1 Requirements Coverage

The functional requirements identified in Section 3.2 have all been delivered: authentication (register US-01, log in US-02, log out US-03) with secure password storage; the COMPADRE and COMADRE matrix catalogue with browsing (US-04), search (US-05), and export (US-07), seeded automatically per US-08; custom matrices with full CRUD support (create US-10, edit US-11, delete US-12), including, as of this revision, complete editing of cell values and metadata (see the US-11 note below); visibility and sharing controls (US-13, US-14); deterministic and stochastic simulations (US-16, US-17) with their associated ecological analytics (US-20); and asynchronous quasi-extinction analysis with per-stage threshold configuration (US-21, US-22). The non-functional goals stated in Chapter 1 (Section 1, a web-based, zero-installation tool, freely accessible to students and researchers regardless of institution) are also met: the application requires only a

browser, and is deployed publicly with no account needed to browse the catalogue or run an ephemeral simulation.

One requirement was delivered later than the others. US-11 ("edit the cell values and metadata of a custom matrix I own") had its backend fully implemented and tested from early on, but the frontend only exposed `common_name` and `country_code` for editing (species, kingdom, matrix cell values, and stage names were not editable through the UI), despite the acceptance criteria requiring it. This was found during the documentation phase, while auditing the user stories against the live application, and fixed with the same test-driven discipline used elsewhere in the project: failing unit tests for the extracted shared grid-editor module, then failing end-to-end tests for the full edit flow, before writing the implementation. It is a useful case study in why specifying acceptance criteria is not the same as verifying them.

8.1.2 Technical Lessons

The strict controller → service → repository layering proved its value most clearly in testing: every service can be unit-tested against mocked repositories with no database required, which kept the unit suite (Table 63, 249 tests) running in seconds and made it the natural place to put business-rule tests. FastAPI's automatic OpenAPI generation and Pydantic's validation caught contract mismatches early and gave the frontend a reliable, self-documenting API to build against. Python Shiny enabled a fully reactive UI without a separate JavaScript build step, which suited a solo-developer timeline well, though it comes with real constraints: the US-11 fix above surfaced a subtle Shiny pitfall (reading and writing the same reactive value within one render execution creates a self-invalidation loop), and a second bug where a parent container re-render silently discarded a child output's pending update, both invisible without end-to-end browser testing. Unit and integration tests alone would not have caught either.

8.1.3 Interdisciplinary Dimension

Implementing the analytics service required engaging directly with population ecology literature, eigenvalue-based metrics (dominant growth rate, stable stage distribution, reproductive value, sensitivity and elasticity) come from [1], and the quasi-extinction threshold concept from [2]. Integrating the COMPADRE and COMADRE databases exposed practical data-wrangling challenges (heterogeneous matrix dimensions, missing values, and an R-native `.RData`/JSON distribution format that needed careful, defensive parsing). This collaboration between the School of Engineering and the Faculty of Biology is the project's defining characteristic, and shaped technical decisions (such as keeping COMPADRE/COMADRE matrices read-only and immutable in stored simulations) that a purely software-engineering brief would not have surfaced on its own.

8.1.4 Process Reflection

Containerisation with Docker Compose eliminated environment-specific bugs almost entirely; the one exception, the `entrypoint.sh` CRLF issue (risk T-06, materialised early

and resolved quickly), was itself an argument for containerising sooner rather than later. The CI/CD pipeline (unit and integration tests, Bandit, pip-audit, Trivy, CodeQL, soon SonarCloud) made it possible to merge with confidence throughout development. Test-driven development was not applied with full consistency in the earliest implementation sprints, which is part of why the US-11 gap went undetected for as long as it did; the fix applied strict red-green-refactor discipline throughout, and that contrast is itself a useful, concrete lesson about the cost of skipping it.

8.2 Future Work

Several items that were originally scoped as future work during earlier planning (COMADRE seeding, internationalisation (US-24), average-matrix display for stochastic and quasi-extinction runs, per-stage quasi-extinction thresholds and exclusion (US-22), and the per-run-commitment stochastic simulation rework) have since been fully implemented and shipped, and are described in their respective chapters rather than here. What remains:

- **Real-time collaboration**: concurrent editing of the same matrix by multiple users is explicitly out of scope (Section 3.2) and would require optimistic concurrency control or operational transforms not currently designed for.
- **Celery/Redis migration**: quasi-extinction jobs currently run as FastAPI `BackgroundTasks` against a DB-backed `SimulationJob` record. The architecture was designed with this migration in mind (job state already lives in the database, not in process memory), but moving to a proper task queue would be needed to support many concurrent users running Monte Carlo analyses at once.
- **Observability infrastructure**: as noted in Section 5.5.3, Prometheus metrics collection and Grafana dashboards were deliberately left out of the current release (observability today is limited to Uvicorn's structured access logs and the `/health` liveness probe). Adding a Prometheus exporter (e.g. `prometheus-fastapi-instrumentator`), a scrape configuration, and Grafana dashboards for request latency, error rate, and quasi-extinction job throughput would matter most if the institutional-deployment item below materialises and the application starts serving concurrent classroom-scale traffic.
- **Faster first-deploy seeding**: seeding the full COMPADRE and COMADRE catalogue (18,000 records, Section 7.2.3.4) takes 2-3 minutes on first deploy, during which the API does not respond. A pre-baked database image or a lazy/background seeding strategy would remove this cold-start cost.
- **Institutional deployment**: the application is already publicly deployed (Section 7.2.3), but formal adoption into a Biology course's curriculum (with instructor accounts, classroom-scale usage, and direct faculty feedback) has not yet happened and remains the project's original long-term goal.
- **Usability testing**: no formal usability study has been conducted. A structured evaluation — recruit representative users (biology students and researchers), administer

task-based scenarios covering the main workflows, and collect both think-aloud observations and SUS (System Usability Scale) questionnaire responses — would surface friction points in the interface and provide evidence-based input for a redesign iteration.

- **Load and performance testing**: the application has not been subjected to systematic load testing. Given that quasi-extinction jobs are CPU-bound Monte Carlo simulations running as FastAPI `BackgroundTasks`, concurrent classroom-scale submissions could saturate a single worker. A load-test suite (e.g. Locust or k6) targeting the `/v1/jobs/quasi-extinction` endpoint under simultaneous users would quantify throughput limits and validate whether the Celery/Redis migration noted above is needed before any institutional deployment.

Chapter 9

Appendices

9.1 Risk Management Plan

Risk management on this project follows a Probability-Impact methodology adapted from the PMBOK Guide for a single-developer academic project, covering both **threats** (negative risks) and **opportunities** (positive risks) with the same machinery. It has four steps: (1) classify candidate risks using a Risk Breakdown Structure (RBS), (2) rate each one's probability and impact against fixed qualitative scales, (3) compute an exposure score and place it on a Probability-Impact Matrix to prioritise it, and (4) define a contingency plan for every threat (or an exploitation plan for every opportunity) that lands in the matrix's red zone. The resulting risk identification and Risk Register live in Section 2.1.5, and the opportunity identification and Opportunity Register live in Section 2.1.6, both in the planning chapter.

9.1.1 Risk Breakdown Structure

Risks and opportunities are grouped into the same four top-level categories, each split into subcategories that narrow down the likely source of the risk. Not every subcategory has a risk or opportunity assigned to it yet; the RBS is the fixed classification scheme used throughout the project, not a list of risks itself.

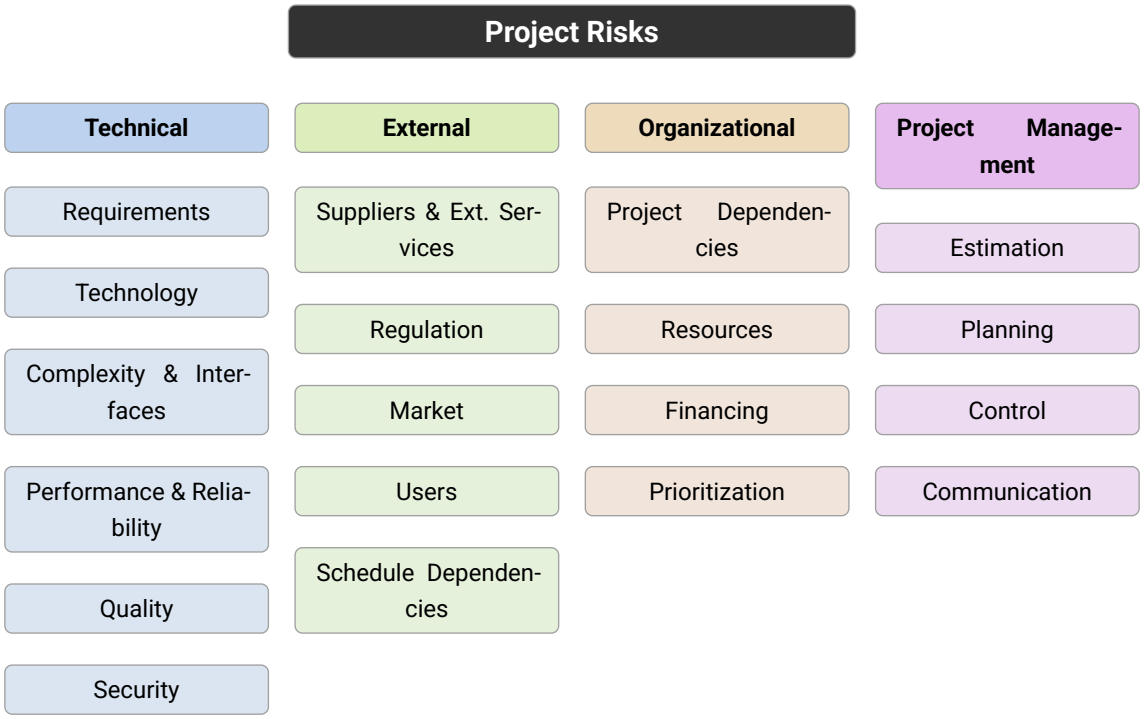

Figure 96: Risk Breakdown Structure (RBS).

9.1.2 Probability and Impact Scales

Probability is rated on a five-level qualitative scale (Table 68). Impact is rated independently against four project objectives (cost, schedule, scope, and quality) using the scale in Table 69, so that a risk threatening, say, the scientific validity of the simulation results (quality) is not understated just because it has no cost or schedule effect.

Label	Range	Value used for calculations
Very Low	[0%-20%]	10%
Low	(20%-40%]	30%
Medium	(40%-60%]	50%
High	(60%-80%]	70%
Very High	(80%-100%]	90%

Table 68: Risk probability scale.

Objective	Very Low / 5%	Low / 10%	Moderate / 20%	High / 40%	Very High / 80%
Cost	Insignificant cost increase	Cost increase < 10%	Cost increase 10-20%	Cost increase 20-40%	Cost increase > 40%
Schedule	Insignificant time increase	Time increase < 5%	Time increase 5-10%	Time increase 10-20%	Time increase > 20%
Scope	Barely noticeable scope reduction	Minor areas of scope affected	Major areas of scope affected	Scope reduction unacceptable to the stakeholders	The delivered scope is effectively useless

Quality	Barely noticeable quality degradation	Only very demanding applications are affected	Quality reduction requires stakeholder acceptance	Quality reduction unacceptable to the stakeholders	The end result is effectively useless

Table 69: Risk impact scale across the four project objectives.

Table 69 describes negative consequences, for rating threats. The same numeric thresholds apply to opportunities, but represent a **positive** contribution to the equivalent objective instead (e.g. a schedule reduction rather than a schedule increase, or a quality improvement rather than a quality degradation).

9.1.3 Probability-Impact Matrix

For each risk, the exposure score is **probability x impact**, where impact is the **highest-rated objective** for that risk (its worst-case dimension). Table 70 maps every probability/impact combination to a priority zone: green risks are accepted and simply monitored, yellow risks are actively mitigated, and red risks additionally require a documented contingency plan, provided below for every red-zone risk identified in Section 9.1.

Probability \ Impact	0.05 Very Low	0.10 Low	0.20 Moderate	0.40 High	0.80 Very High
0.90 Very High	0.05	0.09	0.18	0.36	0.72
0.70 High	0.04	0.07	0.14	0.28	0.56
0.50 Medium	0.03	0.05	0.10	0.20	0.40
0.30 Low	0.02	0.03	0.06	0.12	0.24
0.10 Very Low	0.01	0.01	0.02	0.04	0.08

Table 70: Probability-Impact Matrix. Green: low priority (score ≤ 0.05). Yellow: moderate priority (0.06-0.14). Red: high priority, requires a contingency plan (score ≥ 0.18).

9.1.4 Opportunity Probability-Impact Matrix

Opportunities are scored the same way (**probability x impact**, using the best-rated objective) and placed on the same matrix shape (Table 71). The zones carry the opposite meaning: red is the highest priority to actively **exploit**, not the highest priority to mitigate. Green opportunities are accepted and simply monitored; yellow opportunities are actively enhanced; red opportunities get a dedicated exploitation plan, provided below.

Probability \ Impact	0.05 Very Low	0.10 Low	0.20 Moderate	0.40 High	0.80 Very High
0.90 Very High	0.05	0.09	0.18	0.36	0.72
0.70 High	0.04	0.07	0.14	0.28	0.56

0.50 Medium	0.03	0.05	0.10	0.20	0.40
0.30 Low	0.02	0.03	0.06	0.12	0.24
0.10 Very Low	0.01	0.01	0.02	0.04	0.08

Table 71: Opportunity Probability-Impact Matrix. Green: low priority, simply monitored. Yellow: moderate priority, actively enhanced. Red: high priority (score $\geq$ 0.18), requires a dedicated exploitation plan.

9.1.5 Contingency Plans

Five identified risks fall in the matrix's red zone: T-01 (simulation and analytics correctness errors), O-01 (sole-developer key-person risk), O-02 (competing coursework workload), PM-01 (effort estimation / schedule slippage), and PM-02 (scope creep from stakeholder requests). Each has a contingency plan below, to be activated if the trigger condition occurs.

Contingency Plan - T-01 - Simulation and analytics correctness errors	
Trigger	A unit test or a co-tutor review reveals that computed analytics (λ_1, sensitivities, elasticities) or simulation output disagrees with a known reference case from Caswell (2001) or with the co-tutor's manual calculation.
Owner	Sole developer (Mario Orviz Viesca), with biological validation from the co-tutor (SI-5, Mario Quevedo de Anta).
Immediate actions	Freeze the affected endpoint behind the test suite. Do not present the corresponding result as validated in this document until corrected; write a unit test that reproduces the discrepancy.
Short-term actions	Re-derive the formula to correct the discrepancy; add a regression test pinned to the corrected reference values; request a follow-up review from the co-tutor.
Reserve usage	Use up to one documentation buffer day to re-verify and rewrite the affected analytics section.

Table 72: Contingency Plan - T-01 - Simulation and analytics correctness errors

Contingency Plan - O-01 - Sole-developer key-person risk	
Trigger	The developer is unavailable (illness, personal emergency) for more than three consecutive working days during an active phase.
Owner	Sole developer; both tutors are informed as soon as the absence is identified.
Immediate actions	Notify both tutors of the interruption and the expected return date; commit and push all work-in-progress so no work is at risk of loss.
Short-term actions	Re-baseline the schedule on return, consuming available buffer days first; if the absence exceeds one week, descope Should/Could-priority user stories (MoSCoW priorities, Section 4.1.1.4) to protect the Must-priority core.
Reserve usage	Consume the schedule's buffer days (Day 13-14) and, if exhausted, request a submission deadline extension through the academic tutor.

Table 73: Contingency Plan - O-01 - Sole-developer key-person risk

Contingency Plan - O-02 - Competing coursework workload	
Trigger	Weekly tracking shows actual progress more than three days behind the planned schedule for two consecutive weeks.
Owner	Sole developer.
Immediate actions	Re-prioritise the current week using MoSCoW priorities; pause Could/Should items in favour of Must items.
Short-term actions	Reallocate planned buffer days (Day 13-14) to the affected chapter; if slippage persists, move Should-priority future-work items (Section 8.2) out of the current scope.
Reserve usage	Up to two buffer days reallocated from the documentation review phase.

Table 74: Contingency Plan - O-02 - Competing coursework workload

Contingency Plan - PM-01 - Effort estimation / schedule slippage	
Trigger	A phase's actual duration exceeds its planned duration by more than 50% (per the Tracking Plan, Section 2.2.1).
Owner	Sole developer.
Immediate actions	Log the deviation in the Project Issue Log with its cause and the estimated extra effort required.
Short-term actions	Re-estimate the remaining phases using the observed velocity rather than the original plan; trim Should/Could-priority scope first.
Reserve usage	Use the schedule's built-in buffer days before requesting any deadline extension.

Table 75: Contingency Plan - PM-01 - Effort estimation / schedule slippage

Contingency Plan - PM-02 - Scope creep from stakeholder requests	
Trigger	A stakeholder (Biology Faculty) formally requests a feature not present in the SR-F baseline (Section 3.2).
Owner	Sole developer, with acceptance authority resting with the academic tutor (SI-4 is Accountable for requirements decisions per the RACI matrix, Table 4).
Immediate actions	Log the request in the Project Issue Log; do not implement it without baseline approval.
Short-term actions	Evaluate the request against current Must-priority backlog capacity; if accepted, add it to Future Work (Section 8.2) rather than the current release, unless a Must-priority item is descoped to compensate.
Reserve usage	None by design, scope changes do not consume schedule reserve; they are deferred instead.

Table 76: Contingency Plan - PM-02 - Scope creep from stakeholder requests

9.1.6 Opportunity Exploitation Plans

One identified opportunity falls in the matrix's red zone: OPP-E-01 (institutional adoption interest from the Biology Faculty). It has an exploitation plan below, to be activated if the opportunity arises.

Exploitation Plan - OPP-E-01 - Institutional adoption interest from the Biology Faculty	
Trigger	A Faculty of Biology stakeholder (SI-7) or the co-tutor (SI-5) expresses interest, during a demo or the TFG defense, in using the platform beyond the scope of this TFG.
Owner	Sole developer, with the academic tutor (SI-4) and co-tutor (SI-5) as the natural points of contact for any institutional follow-up.
Immediate actions	Acknowledge the interest and note it in the Project Issue Log; avoid making delivery commitments beyond what the Installation and Operations manuals (Section 7.2) already support.
Short-term actions	Walk the interested party through the Installation manual (Section 7.2) for a self-hosted deployment, or scope a minimal institutional deployment (per the Future Work item, Section 8.2) as a separate, explicitly out-of-TFG-scope effort.
Resourcing	No additional resourcing within the TFG's timeline; any real deployment work is treated as Future Work (Section 8.2), pursued after submission.

Table 77: Exploitation Plan - OPP-E-01 - Institutional adoption interest from the Biology Faculty

9.2 Content Delivered

The physical submission consists of the two files listed in Table 78. Because the project artefacts (source code, database seed data, Docker configuration, and CI pipelines) are hosted in a public GitHub repository, no source archive is bundled with the PDF. The `README.txt` file contains the two URLs needed to access the live application and to inspect or clone the full project.

File	Description
`TFG_Mario_Orviz.pdf`	This document (technical report).
`README.txt`	Plain-text file containing the URL of the public GitHub repository and the URL of the live deployed application.

Table 78: Files included in the physical submission.

9.2.1 Repository Structure

The public repository at https://github.com/orvizz/population-growth-simulator is organised as a monorepo containing the backend API, the frontend web application, the database layer, automated tests, and the CI/CD workflow definitions. Table 79 describes the top-level directories and key files.

Path	Description
`api/`	FastAPI backend.
`api/controllers/`	HTTP layer: route handlers; no business logic.
`api/services/`	Business logic: simulation algorithm, analytics, authentication.
`api/repositories/`	Database access layer (SQLAlchemy queries).
`api/schemas.py`	Input DTOs (Pydantic); all input validation lives here.
`api/records.py`	Output domain records returned by the API.
`api/main.py`	App factory; registers routers and CORS middleware.
`frontend/`	Python Shiny web application.
`frontend/app.py`	Main entry point; all four UI tabs.
`frontend/components/`	Reusable UI widgets (matrix editor, simulation panel, etc.).
`frontend/locales/`	i18n translation files (English, Spanish, Asturian, Galician, Basque, Catalan).
`db/`	ORM models, session factory, and COMPADRE/COMADRE seed scripts.
`alembic/`	Database migration files; applied automatically on container start.
`tests/`	Integration tests (require a running PostgreSQL instance).
`tests/unit/`	Unit tests with no database required; use `unittest.mock`.
`tests/e2e/`	End to end tests
`.github/workflows/`	CI/CD pipelines: unit + integration tests, Bandit/pip-audit security scans, and CodeQL static analysis.
`docker-compose.yml`	Local multi-service stack (`api` + `frontend` + `db`).
`Dockerfile`	Docker image for the API service (also used locally for the frontend).
`Dockerfile.frontend`	Entrypoint-less image for the Railway frontend service.
`Makefile`	Developer shortcuts (`make up`, `make down`, `make logs`).
`requirements.txt`	Python runtime dependencies.
`tfg-doc/`	Typst source files for this document.

Table 79: Structure of the public GitHub repository (https://github.com/orvizz/population-growth-simulator).

BIBLIOGRAPHY

[1] H. Caswell, *Matrix Population Models: Construction, Analysis, and Interpretation*, 2nd ed. Sunderland, MA: Sinauer Associates, 2001.

[2] D. F. D. William F. Morris, *Quantitative conservation biology : theory and practice of population viability analysis*. 2002.

[3] M. Quevedo, "eco3r: Population ecology with R." [Online]. Available: https://github.com/quevedomario/eco3r

[4] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA: Project Management Institute, 2021.

[5] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[6] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[7] R. Salguero-Gómez *et al.*, "The COMPADRE Plant Matrix Database: an open online repository for plant demography," *Journal of Ecology*, vol. 103, no. 1, pp. 202–218, 2015, doi: 10.1111/1365-2745.12334.

[8] PyInstaller Development Team, "PyInstaller." [Online]. Available: https://pyinstaller.org/

[9] Posit PBC, "Shiny for Python." [Online]. Available: https://shiny.posit.co/py/

[10] Docker, Inc., "Docker." [Online]. Available: https://www.docker.com/

[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[12] R Core Team, "R: A Language and Environment for Statistical Computing." Vienna, Austria, 2024. [Online]. Available: https://www.r-project.org/

[13] OpenJS Foundation, "Node.js." [Online]. Available: https://nodejs.org/

[14] Python Software Foundation, "Python Programming Language." [Online]. Available: https://www.python.org/

[15] Streamlit Inc., "Streamlit." [Online]. Available: https://streamlit.io/

[16] Plotly Technologies Inc., "Dash." [Online]. Available: https://dash.plotly.com/

[17] S. Ramírez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com/

[18] Pallets Projects, "Flask." [Online]. Available: https://flask.palletsprojects.com/

[19] T. Christie, "Django REST Framework." [Online]. Available: https://www.django-rest-framework.org/

[20] Django Software Foundation, “Django.” [Online]. Available: https://www.djangoproject.com/

[21] S. Colvin, “Pydantic.” [Online]. Available: https://docs.pydantic.dev/

[22] T. Christie, “Starlette.” [Online]. Available: https://www.starlette.io/

[23] Oracle Corporation, “MySQL.” [Online]. Available: https://www.mysql.com/

[24] D. R. Hipp, “SQLite.” [Online]. Available: https://www.sqlite.org/

[25] MongoDB, Inc., “MongoDB.” [Online]. Available: https://www.mongodb.com/

[26] The PostgreSQL Global Development Group, “PostgreSQL.” [Online]. Available: https://www.postgresql.org/

[27] Tortoise ORM Contributors, “Tortoise ORM.” [Online]. Available: https://tortoise.github.io/

[28] Psycopg Team, “Psycopg – PostgreSQL database adapter for Python.” [Online]. Available: https://www.psycopg.org/

[29] M. Bayer, “SQLAlchemy.” [Online]. Available: https://www.sqlalchemy.org/

[30] Redgate Software, “Flyway.” [Online]. Available: https://flywaydb.org/

[31] Liquibase Inc., “Liquibase.” [Online]. Available: https://www.liquibase.org/

[32] M. Bayer, “Alembic.” [Online]. Available: https://alembic.sqlalchemy.org/